

Chương 1. NGÔN NGỮ LẬP TRÌNH VÀ PHƯƠNG PHÁP LẬP TRÌNH

Mục tiêu: Sau khi hoàn tất bài này học viên sẽ hiểu và vận dụng các kiến thức kỹ năng cơ bản sau:

- Ý nghĩa, các bước lập trình.
- Xác định dữ liệu vào, ra.
- Phân tích các bài toán đơn giản.
- Khái niệm so sánh, lặp.
- Thể hiện bài toán bằng lưu đồ.

1.1. NGÔN NGỮ LẬP TRÌNH (*Programming Language*)

Phần này chúng ta sẽ tìm hiểu một số khái niệm căn bản về thuật toán, chương trình, ngôn ngữ lập trình. Thuật ngữ "thuật giải" và "thuật toán" dĩ nhiên có sự khác nhau song trong nhiều trường hợp chúng có cùng nghĩa.

1.1.1. Thuật giải (Algorithm)

Là một dãy các thao tác xác định trên một đối tượng, sao cho sau khi thực hiện một số hữu hạn các bước thì đạt được mục tiêu. Theo R.A.Kowalski thì bản chất của thuật giải:

Thuật toán = Logic + Điều khiển

* **Logic**: Đây là phần khá quan trọng, nó trả lời câu hỏi "Thuật toán làm gì, giải quyết vấn đề gì?", những yếu tố trong bài toán có quan hệ với nhau như thế nào v.v... Ở đây bao gồm những kiến thức chuyên môn mà bạn phải biết để có thể tiến hành giải bài toán.

Ví dụ 1: Để giải một bài toán tính diện tích hình cầu, mà bạn không còn nhớ công thức tính hình cầu thì bạn không thể viết chương trình cho máy để giải bài toán này được.

* **Điều khiển**: Thành phần này trả lời câu hỏi: giải thuật phải làm như thế nào?. Chính là cách thức tiến hành áp dụng thành phần logic để giải quyết vấn đề.

Ví dụ 2: thuật toán để giải phương trình bậc nhất $P(x): ax + b = c$, (a, b, c là các số thực), trong tập hợp các số thực có thể là một bộ các bước sau đây:

Nếu $a = 0$

$b = c$ thì $P(x)$ có nghiệm bất kì

$b \neq c$ thì $P(x)$ vô nghiệm

Nếu $a \neq 0$

$P(x)$ có duy nhất một nghiệm $x = (c - b)/a$

Lưu ý:

Khi một thuật toán đã hình thành thì ta không xét đến việc chứng minh thuật toán đó mà chỉ chú trọng đến việc áp dụng các bước theo sự hướng dẫn sẽ có kết quả đúng. Việc chứng minh tính đầy đủ và tính đúng của các thuật toán phải được tiến hành xong trước khi có thuật toán. Nói rõ hơn, thuật toán có thể chỉ là việc áp dụng các công thức hay qui tắc, qui trình đã được công nhận là đúng hay đã được chứng minh về mặt toán học.

"Thuật toán" hiện nay thường được dùng để chỉ thuật toán giải quyết các vấn đề tin học. Hầu hết các thuật toán tin học đều có thể viết thành các chương trình máy tính mặc dù chúng thường có một vài hạn chế (vì khả năng của máy tính và khả năng của người lập trình). Trong nhiều trường hợp, một chương trình khi thiết kế bị thất bại là do lỗi ở các thuật toán mà người lập trình đưa vào là không chính xác, không đầy đủ, hay không ước định được trọn vẹn lời giải của vấn đề. Tuy nhiên cũng có một số bài toán mà hiện nay

người ta chưa tìm được lời giải triệt để, những bài toán ấy gọi là những bài toán NP-completed.

Một thuật toán có các tính chất sau:

Tính chính xác: để đảm bảo kết quả tính toán hay các thao tác mà máy tính thực hiện được là chính xác.

Tính rõ ràng: Thuật toán phải được thể hiện bằng các câu lệnh minh bạch; các câu lệnh được sắp xếp theo thứ tự nhất định.

Tính khách quan: Một thuật toán dù được viết bởi nhiều người trên nhiều máy tính vẫn phải cho kết quả như nhau.

Tính phổ dụng: Thuật toán không chỉ áp dụng cho một bài toán nhất định mà có thể áp dụng cho một lớp các bài toán có đầu vào tương tự nhau.

Tính kết thúc: Thuật toán phải gồm một số hữu hạn các bước tính toán.

1.1.2. Chương trình (Program)

Là một tập hợp các mô tả, các phát biểu, nằm trong một hệ thống qui ước về ý nghĩa và thứ tự thực hiện, nhằm điều khiển máy tính làm việc. Theo Niklaus Wirth thì:

Chương trình = Thuật toán + Cấu trúc dữ liệu

Các thuật toán và chương trình đều có cấu trúc dựa trên **3 cấu trúc điều khiển cơ bản**:

* **Tuần tự** (Sequential): Các bước thực hiện tuần tự một cách chính xác từ trên xuống, mỗi bước chỉ thực hiện đúng một lần. Kết quả của bước giải trước là tiền đề cho bước giải sau.

* **Chọn lọc** (Selection): Chọn 1 trong 2 hay nhiều thao tác để thực hiện. Việc thực hiện lệnh phụ thuộc vào một điều kiện nào đó.

* **Lặp lại** (Repetition): Một hay nhiều bước được thực hiện lặp lại một số lần. Cho phép thực hiện nhiều lần một quá trình dựa trên một điều kiện cho trước. Có 2 dạng cơ bản của cấu trúc lặp là cấu trúc lặp cho phép lặp lại một quá trình xử lý một số lần xác định trước. Loại còn lại cho phép lặp quá trình xử lý cho đến lúc điều kiện kiểm tra không còn đúng nữa. Cấu trúc lặp chiếm khoảng 98% khối lượng chương trình.

Muốn trở thành lập trình viên chuyên nghiệp bạn hãy làm đúng trình tự để có thói quen tốt và thuận lợi sau này trên nhiều mặt của một người làm máy tính. Bạn hãy làm theo các bước sau:

 Tìm, xây dựng thuật giải (trên giấy) → viết chương trình trên máy
 → dịch chương trình → chạy và thử chương trình

1.1.3. Ngôn ngữ lập trình (Programming language)

Ngôn ngữ lập trình là hệ thống các ký hiệu tuân theo các qui ước về ngữ pháp và ngữ nghĩa, dùng để xây dựng thành các chương trình cho máy tính.

Một chương trình được viết bằng một ngôn ngữ lập trình cụ thể (ví dụ Pascal, C...) gọi là chương trình nguồn, chương trình dịch làm nhiệm vụ dịch chương trình nguồn thành chương trình thực thi được trên máy tính.

1.2. CÁC BƯỚC GIẢI BÀI TOÁN TRÊN MÁY TÍNH

Việc sử dụng máy tính điện tử (MTĐT) để giải quyết một vấn đề nào đó thường được quan niệm một cách không chuẩn xác, đơn giản đó chỉ là việc lập trình thuần túy. Thực ra, đó là cả một quá trình phức tạp bao gồm nhiều giai đoạn phát triển mà lập trình chỉ là một trong các giai đoạn đó (thậm chí chưa chắc đã là phần việc quan trọng nhất). Các bước quan trọng của toàn bộ quá trình được liệt kê dưới đây:

Bước 1. Xác định vấn đề - bài toán. (xác định I-P-O)

Bước đầu tiên của bước phân tích hệ thống là nhằm phát biểu chính xác vấn đề - bài toán, làm rõ những yêu cầu mà người sử dụng đòi hỏi. Sau khi nghiên cứu vấn đề được đặt ra, người phân tích viên thiết lập mối phụ thuộc giữa các dữ kiện và kết quả phải tìm. Trên cơ sở có được mô hình vấn đề - bài toán, người phân tích viên sẽ đánh giá, nhận định tính khả thi của vấn đề - bài toán được đặt ra có đáng phải giải quyết không?

Bước 2. Lựa chọn phương pháp giải.

Có thể có nhiều cách khác nhau để giải quyết vấn đề - bài toán đã thiết lập ở bước 1. Các phương pháp có thể khác nhau về thời gian thực hiện, chi phí lưu trữ dữ liệu, độ chính xác.... Nói chung không có phương pháp tối ưu về mọi phương diện. Tùy theo nhu cầu cụ thể mà lựa chọn phương pháp thích hợp. Việc lựa chọn trên cũng cần căn cứ vào khả năng xử lý tự động mà ta sẽ sử dụng.

Bước 3. Xây dựng thuật toán hoặc thuật giải.

Xây dựng mô hình chặt chẽ, chính xác hơn và chi tiết hóa hơn phương pháp đã lựa chọn. Xác định rõ ràng dữ liệu vào, ra cho các bước thực hiện cơ bản và trật tự thực hiện các bước cơ bản đó. Nên áp dụng phương pháp thiết kế có cấu trúc, từ thiết kế tổng thể tiến hành làm mịn dần từng bước.

Bước 4. Cài đặt chương trình.

Mô tả thuật giải bằng chương trình. Dựa vào thuật giải đã được xây dựng, căn cứ quy tắc của một ngôn ngữ lập trình để soạn thảo ra chương trình thể hiện giải thuật thiết lập ở bước 3.

Bước 5. Hiệu chỉnh chương trình.

Ở bước 4, nói chung chúng ta không tránh khỏi sai sót. Ở bước 5 này chúng ta cho chương trình chạy thử để phát hiện và điều chỉnh các sai sót nếu tìm thấy.

Có hai loại lỗi:

✚Lỗi cú pháp là lỗi do không tuân thủ đúng các quy tắc viết chương trình trên một ngôn ngữ lập trình cụ thể.

✚Lỗi ngữ nghĩa là lỗi làm sai lạc ý nghĩa hoặc dẫn đến bế tắc của chương trình. Lỗi cú pháp thường dễ phát hiện và hiệu chỉnh hơn lỗi ngữ nghĩa. Cần phải nói rằng việc hiệu chỉnh chương trình khá phức tạp, mất nhiều thời gian và công sức. Việc xây dựng tốt, phù hợp, đầy đủ các bộ dữ liệu để kiểm chứng chương trình là hết sức quan trọng, giúp phát hiện ra các lỗi ngữ nghĩa của chương trình cũng như có thể có vấn đề gì đó bị bỏ sót.

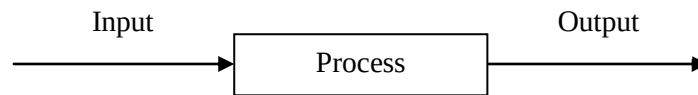
Bước 6. Thực hiện chương trình.

Cho MTĐT thực hiện chương trình. Tiến hành phân tích kết quả thu được. Việc phân tích kết quả nhằm khẳng định kết quả đó có phù hợp hay không. Nếu không, cần kiểm tra lại toàn bộ các bước một lần nữa. Nói chung, dù thận trọng đến mức nào đi nữa thì sau mỗi bước thực hiện nêu trên cũng không khẳng định được kết quả thực hiện từng bước là đúng đắn tuyệt đối. Hơn nữa, như ở bước 5, ta chỉ hiệu chỉnh tất cả các lỗi đã được phát hiện. Còn có thể có sai sót khác của chương trình với một bộ dữ liệu nào khác phức tạp hơn mà ta chưa có cơ hội để phát hiện trước đó. Do đó, ta không thể khẳng định được rằng, chương trình đúng đắn tuyệt đối, không còn sai sót nữa. Như vậy, việc giải quyết một vấn đề cụ thể thực hiện qua hai giai đoạn. Giai đoạn đầu là giai đoạn quan niệm, gồm các bước phân tích, lựa chọn mô hình, xây dựng thuật giải, cài đặt chương trình. Giai đoạn sau là khai thác và bảo trì chương trình. Trong quá trình sử dụng, nói chung thường có nhu cầu về cải tiến, mở rộng chương trình do các yếu tố của bài toán ban đầu có thể thay đổi.

1.3. KỸ THUẬT LẬP TRÌNH

1.3.1. I-P-O Cycle (Input-Process-Output Cycle) (Quy trình nhập-xử lý-xuất)

Quy trình xử lý cơ bản của máy tính gồm I-P-O.



Ví dụ 1. Xác định Input, Process, Output của việc làm 1 ly nước chanh nóng

Input : ly, đường, chanh, nước nóng, muỗng.

Process : - cho hỗn hợp đường, chanh, nước nóng vào ly.
- dùng muỗng khuấy đều.

Output : ly chanh nóng đã sẵn sàng để dùng.

Ví dụ 2. Xác định Input, Process, Output của chương trình tính tiền lương công nhân tháng 10/2002 biết rằng lương = lương căn bản * ngày công

Input : lương căn bản, ngày công

Process : nhân lương căn bản với ngày công

Output : lương

Ví dụ 3. Xác định Input, Process, Output của chương trình giải phương trình bậc nhất $ax + b = 0$

Input : hệ số a, b

Process : chia $-b$ cho a

Output : nghiệm x

Ví dụ 4. Xác định Input, Process, Output của chương trình tìm số lớn nhất của 2 số a và b.

Input : a, b

Process : Nếu $a > b$ thì *Output* = a lớn nhất

Ngược lại *Output* = b lớn nhất

Thuật toán là một phương pháp thể hiện lời giải bài toán nên cũng phải tuân theo một số quy tắc nhất định. Để có thể truyền đạt thuật toán cho người khác hay chuyển thuật toán thành chương trình máy tính, ta phải có phương pháp biểu diễn thuật toán. Có 3 phương pháp biểu diễn thuật toán :

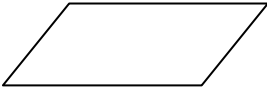
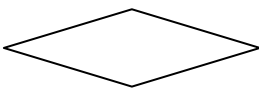
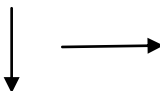
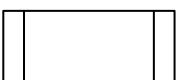
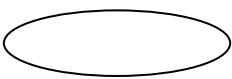
1. Dùng ngôn ngữ tự nhiên.
2. Dùng lưu đồ-sơ đồ khối (flowchart).
3. Dùng mã giả (pseudocode).

1.3.2. Ngôn ngữ tự nhiên (Ngôn ngữ mẹ đẻ)

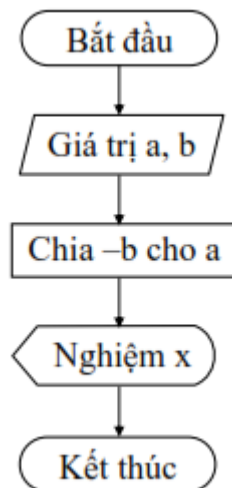
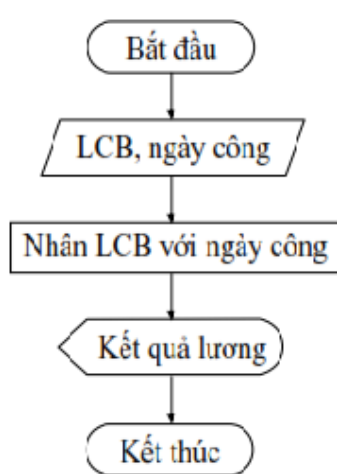
Trong cách biểu diễn thuật toán theo ngôn ngữ tự nhiên, người ta sử dụng ngôn ngữ thường ngày để liệt kê các bước của thuật toán (Các ví dụ về thuật toán trong mục 1 của chương sử dụng ngôn ngữ tự nhiên). Phương pháp biểu diễn này không yêu cầu người viết thuật toán cũng như người đọc thuật toán phải nắm các quy tắc. Tuy vậy, cách biểu diễn này thường dài dòng, không thể hiện rõ cấu trúc của thuật toán, đôi lúc gây hiểu lầm hoặc khó hiểu cho người đọc. Gần như không có một quy tắc cố định nào trong việc thể hiện thuật toán bằng ngôn ngữ tự nhiên. Tuy vậy, để dễ đọc, ta nên viết các bước con lù vào bên phải và đánh số bước theo quy tắc phân cấp như 1, 1.1, 1.1.1, ... Bạn có thể tham khảo lại ví dụ trong mục 1 của chương để hiểu cách biểu diễn thuật toán theo ngôn ngữ tự nhiên.

1.3.3. Lưu đồ - sơ đồ khối

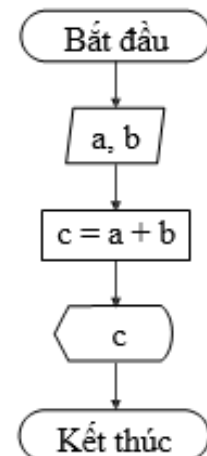
Để dễ hơn về quy trình xử lý, các nhà lập trình đưa ra dạng lưu đồ để minh họa từng bước quá trình xử lý một vấn đề (bài toán).

Hình dạng (symbol)	Hành động (Activity)
	Dữ liệu vào (Input)
	Xử lý (Process)
	Dữ liệu ra (Output)
	Quyết định (Decision), sử dụng điều kiện
	Luồng xử lý (Flow lines)
	Gọi CT con, hàm... (Procedure, Function...)
	Bắt đầu, kết thúc (Begin, End)
	Điểm ghép nối (Connector)

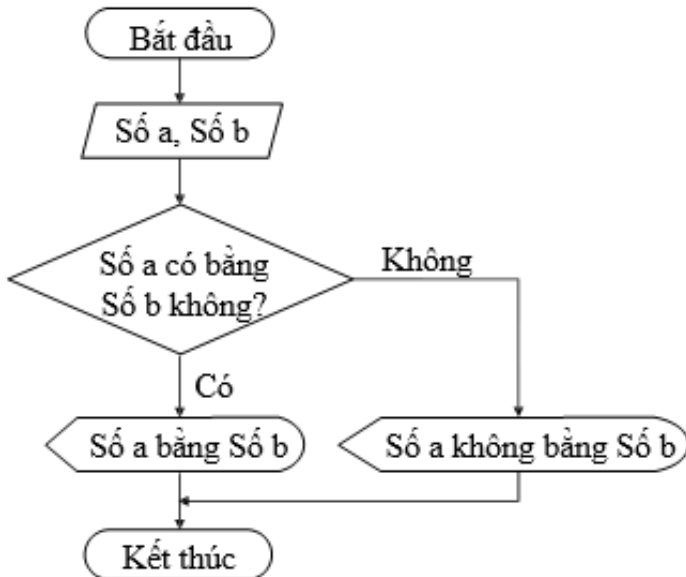
Ví dụ 5. Mô tả bằng sơ đồ khối ví dụ 2,3 như sau:



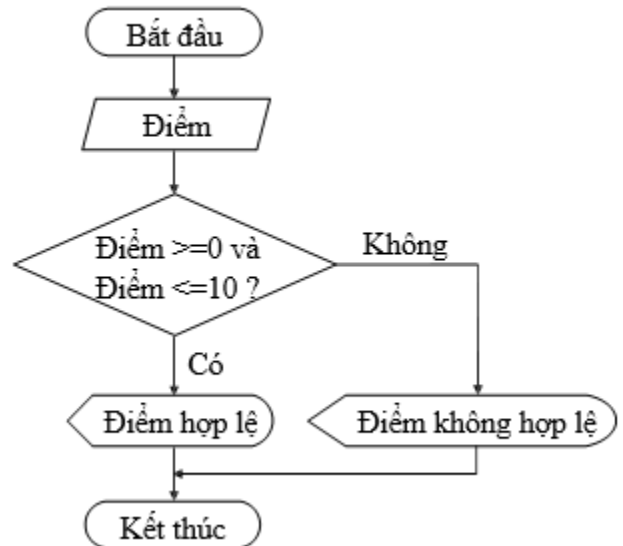
Ví dụ 6: Cộng 2 số



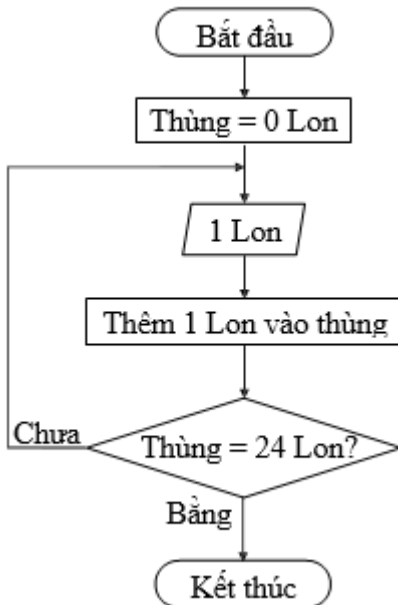
Ví dụ 7: so sánh 2 số



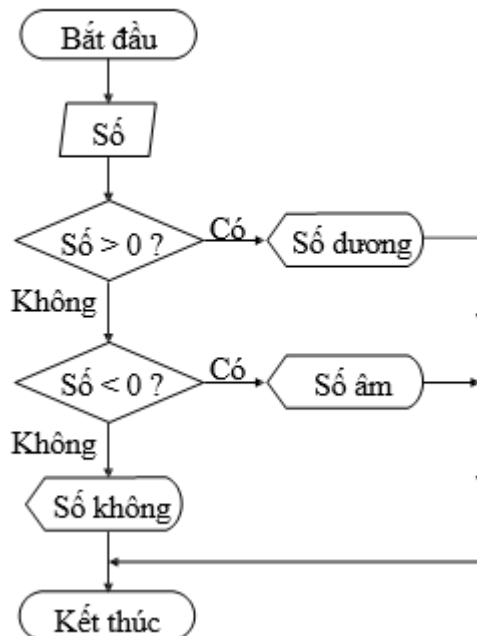
Ví dụ 8: Kiểm tra tính hợp lệ của điểm



Ví dụ 9: Xếp lon vào thùng



Ví dụ 10: Kiểm tra loại số



1.3.4. Ngôn ngữ lập trình (Mã giả)

Tuy sơ đồ khối thể hiện rõ quá trình xử lý và sự phân cấp các trường hợp của thuật toán nhưng lại cồng kềnh. Để mô tả một thuật toán nhỏ ta phải dùng một không gian rất lớn. Hơn nữa, lưu đồ chỉ phân biệt hai thao tác là rẽ nhánh (chọn lựa có điều kiện) và xử lý mà trong thực tế, các thuật toán còn có thêm các thao tác lặp (Chúng ta sẽ tìm hiểu về thao tác lặp trong các bài sau).

Khi thể hiện thuật toán bằng mã giả, ta sẽ vay mượn các cú pháp của một ngôn ngữ lập trình nào đó để thể hiện thuật toán. Tất nhiên, mọi ngôn ngữ lập trình đều có những thao tác cơ bản là xử lý, rẽ nhánh và lặp. Dùng mã giả vừa tận dụng được các khái niệm trong ngôn ngữ lập trình, vừa giúp người cài đặt dễ dàng nắm bắt nội dung thuật toán. Tất nhiên là trong mã giả ta vẫn dùng một phần ngôn ngữ tự nhiên. Một khi đã vay mượn cú pháp và khái niệm của ngôn ngữ lập trình thì chắc chắn mã giả sẽ bị phụ thuộc vào ngôn ngữ lập trình đó. Chính vì lý do này, chúng ta chưa vội tìm hiểu về mã giả trong bài này (vì chúng ta chưa biết gì về ngôn ngữ lập trình!). Sau khi tìm hiểu xong bài về thủ tục - hàm bạn sẽ hiểu mã giả là gì !

Một đoạn mã giả của thuật toán giải phương trình bậc hai

```
if Delta > 0 then begin
    x1=(-b-sqrt(delta))/(2*a)
    x2=(-b+sqrt(delta))/(2*a)
    xuất kết quả : phương trình có hai nghiệm là x1 và x2
end
else
    if delta = 0 then
        xuất kết quả : phương trình có nghiệm kép là -b/(2*a)
    else {trường hợp delta < 0 }
        xuất kết quả : phương trình vô nghiệm
```

CÂU HỎI ÔN TẬP CHƯƠNG 1

Xác định Input, Process, Output và Về lưu đồ cho các chương trình sau:

1. Đổi từ tiền VND sang tiền USD.(1USD=20865đ).
2. Tính điểm trung bình của học sinh gồm các môn Toán, Lý, Hóa.
3. Giải phương trình bậc 2: $ax^2 + bx + c = 0$
4. Đổi từ độ sang radian và đổi từ radian sang độ
(công thức $\alpha/\pi = a/180$, với α : radian, a: độ)
5. Kiểm tra 2 số a, b giống nhau hay khác nhau.
6. Kiểm tra tính hợp lệ của điểm.

Chương 2. LÀM QUEN LẬP TRÌNH C QUA CÁC VÍ DỤ ĐƠN GIẢN

Sau khi hoàn tất chương này viên sẽ hiểu và vận dụng các kiến thức kỹ năng cơ bản sau:

- Ngôn ngữ C.
- Một số thao tác cơ bản của trình soạn thảo C.
- Cách lập trình trên C.
- Tiếp cận một số lệnh đơn giản thông qua các ví dụ.
- Nắm bắt được một số kỹ năng đơn giản.

2.1. KHỞI ĐỘNG VÀ THOÁT KHỎI CHƯƠNG TRÌNH C

C là ngôn ngữ lập trình cấp cao, được sử dụng rất phổ biến để lập trình hệ thống cùng với Assembler và phát triển các ứng dụng.

Vào những năm cuối thập kỷ 60 đầu thập kỷ 70 của thế kỷ XX, Dennis Ritchie (làm việc tại phòng thí nghiệm Bell) đã phát triển ngôn ngữ lập trình C dựa trên ngôn ngữ BCPL (do Martin Richards đưa ra vào năm 1967) và ngôn ngữ B (do Ken Thompson phát triển từ ngôn ngữ BCPL vào năm 1970 khi viết hệ điều hành UNIX đầu tiên trên máy PDP-7) và được cài đặt lần đầu tiên trên hệ điều hành UNIX của máy DEC PDP-11.

Năm 1978, Dennis Ritchie và B.W Kernighan đã cho xuất bản quyển “Ngôn ngữ lập trình C” và được phổ biến rộng rãi đến nay.

Lúc ban đầu, C được thiết kế nhằm lập trình trong môi trường của hệ điều hành Unix nhằm mục đích hỗ trợ cho các công việc lập trình phức tạp. Nhưng về sau, với những nhu cầu phát triển ngày một tăng của công việc lập trình, C đã vượt qua khuôn khổ của phòng thí nghiệm Bell và nhanh chóng hội nhập vào thế giới lập trình để rồi các công ty lập trình sử dụng một cách rộng rãi. Sau đó, các công ty sản xuất phần mềm lần lượt đưa ra các phiên bản hỗ trợ cho việc lập trình bằng ngôn ngữ C và chuẩn ANSI C cũng được khai sinh từ đó.

Ngôn ngữ lập trình C là một ngôn ngữ lập trình hệ thống rất mạnh và rất “mềm dẻo”, có một thư viện gồm rất nhiều các hàm (function) đã được tạo sẵn. Người lập trình có thể tận dụng các hàm này để giải quyết các bài toán mà không cần phải tạo mới. Hơn thế nữa, ngôn ngữ C hỗ trợ rất nhiều phép toán nên phù hợp cho việc giải quyết các bài toán kỹ thuật có nhiều công thức phức tạp. Ngoài ra, C cũng cho phép người lập trình tự định nghĩa thêm các kiểu dữ liệu trừu tượng khác. Tuy nhiên, điều mà người mới vừa học lập trình C thường gặp “rắc rối” là “hơi khó hiểu” do sự “mềm dẻo” của C. Dù vậy, C được phổ biến khá rộng rãi và đã trở thành một công cụ lập trình khá mạnh, được sử dụng như là một ngôn ngữ lập trình chủ yếu trong việc xây dựng những phần mềm hiện nay.

Ngôn ngữ C có những đặc điểm cơ bản sau:

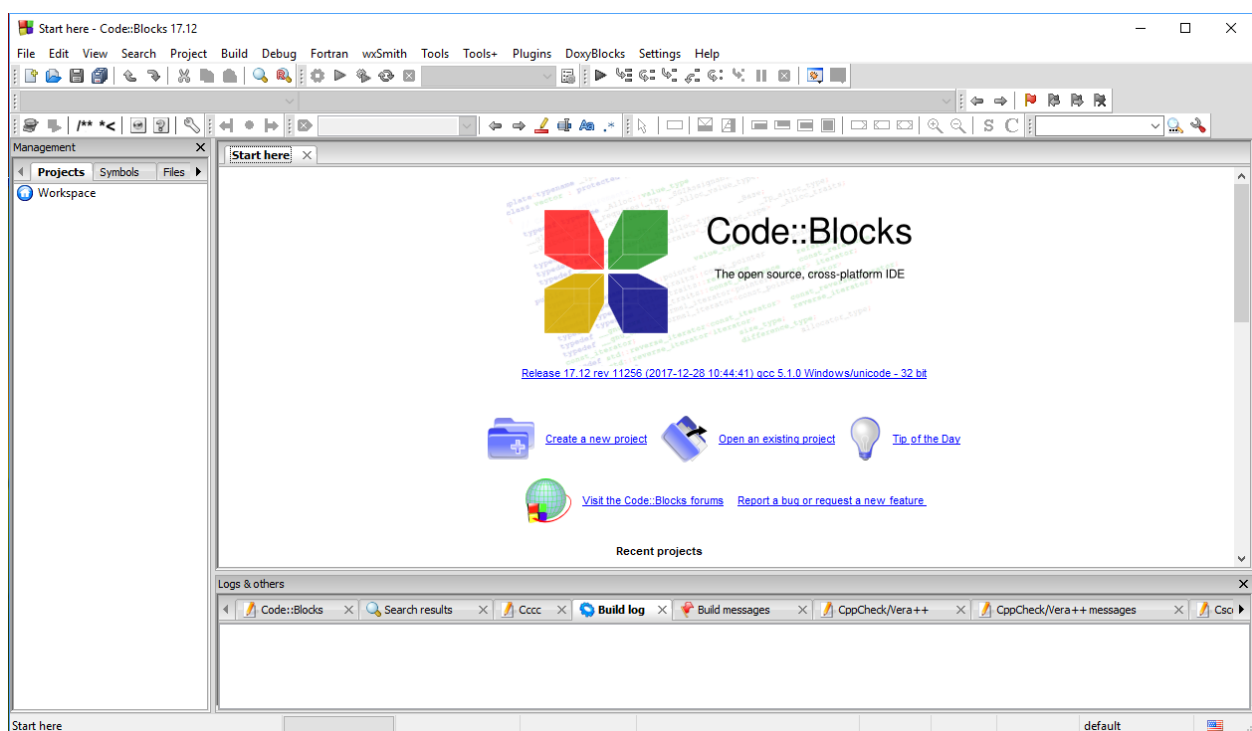
- *Tính cô đọng (compact)*: C chỉ có 32 từ khóa chuẩn và 40 toán tử chuẩn, nhưng hầu hết đều được biểu diễn bằng những chuỗi ký tự ngắn gọn.

- *Tính cấu trúc (structured)*: C có một tập hợp những chỉ thị của lập trình như cấu trúc lựa chọn, lặp... Từ đó các chương trình viết bằng C được tổ chức rõ ràng, dễ hiểu.
- *Tính tương thích (compatible)*: C có bộ tiền xử lý và một thư viện chuẩn vô cùng phong phú nên khi chuyển từ máy tính này sang máy tính khác các chương trình viết bằng C vẫn hoàn toàn tương thích.
- *Tính linh động (flexible)*: C là một ngôn ngữ rất uyển chuyển và cú pháp, chấp nhận nhiều cách thể hiện, có thể thu gọn kích thước của các mã lệnh làm chương trình chạy nhanh hơn.
- *Biên dịch (compile)*: C cho phép biên dịch nhiều tập tin chương trình riêng rẽ thành các tập tin đối tượng (object) và liên kết (link) các đối tượng đó lại với nhau thành một chương trình có thể thực thi được (executable) thống nhất.

2.1.1. Khởi động C



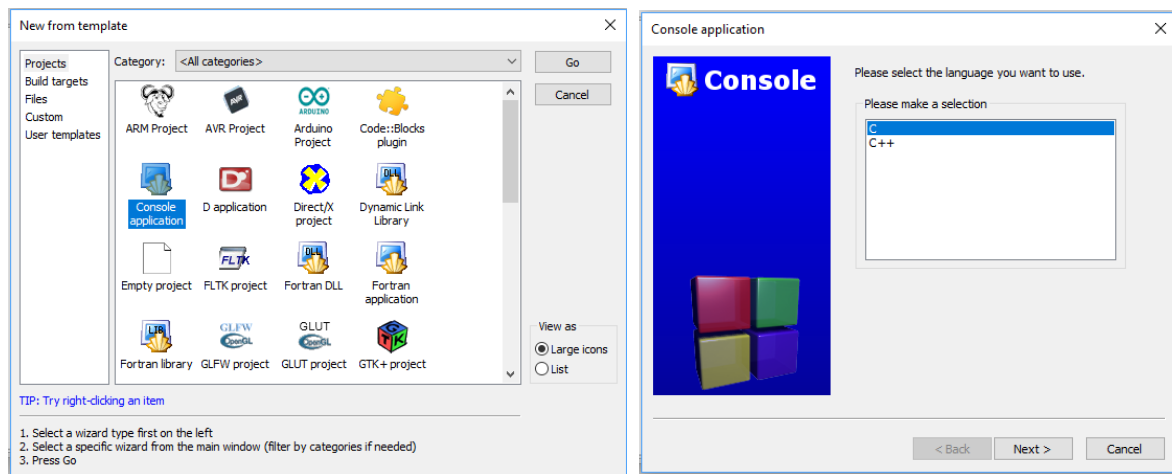
Chạy C cũng giống như chạy các chương trình khác trong môi trường Windows, màn hình sẽ xuất hiện menu của CodeBlock có dạng như sau:



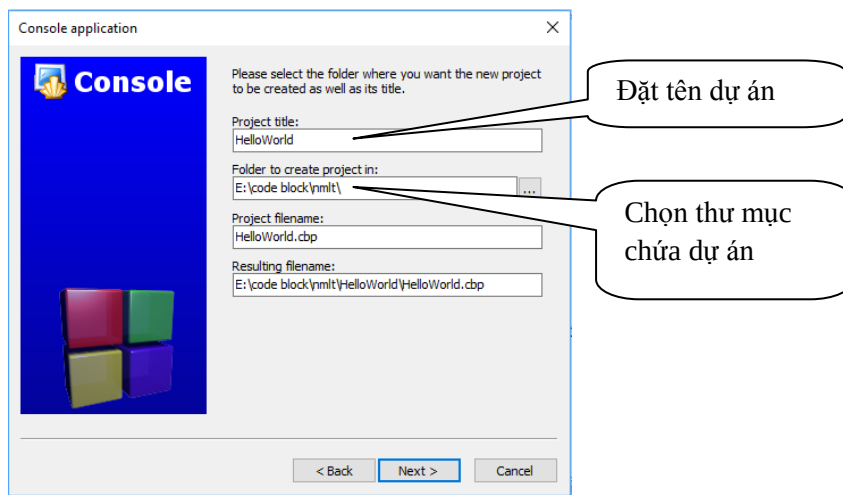
Dòng trên cùng gọi là thanh menu (menu bar). Mỗi mục trên thanh menu lại có thể có nhiều mục con nằm trong một menu kéo xuống.

2.1.2. Bắt đầu một dự án C

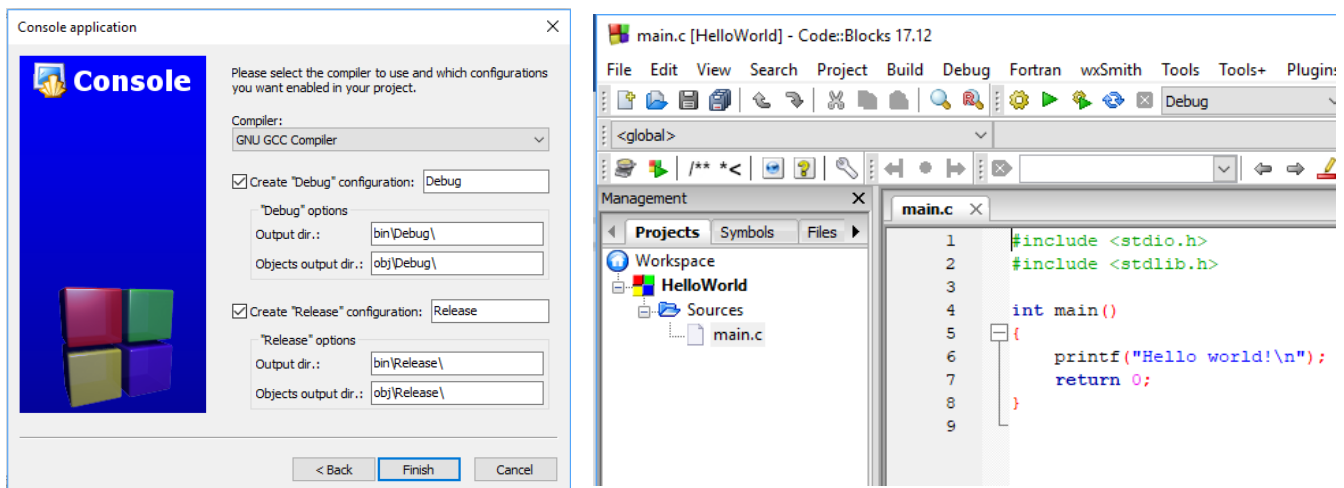
Chọn **File->New->Project...** để bắt đầu dự án mới. Bạn có thể chọn nhiều loại templates để bắt đầu dự án dễ dàng hơn. Với dự án đầu tiên, hãy chọn **Console application** (đây là tùy chọn phổ biến nhất cho những mục đích thông thường). Nhấp **Go**.



Chọn C -> Next.



Chọn trình biên dịch -> Finish, xuất hiện cửa sổ soạn thảo.



2.1.3. Lưu dự án

File -> Save (Ctrl + S).

2.1.4. Thoát khỏi CodeBlock

Nhấn nút Close  trên góc phải hoặc chọn File -> Quit (Ctrl + Q).

2.2. CÁC VÍ DỤ C ĐƠN GIẢN

Ví dụ 1

Dòng	
1	/* Chương trình in ra câu bài học C đầu tiên */
2	#include <stdio.h>
3	int main()
4	{
5	printf("Bài học C đầu tiên.");
6	return 0;
7	}

→ **Kết quả in ra màn hình**

Bài học C đầu tiên.

- Dòng thứ 1: bắt đầu bằng /* và kết thúc bằng */ cho biết hàng này là hàng diễn giải (chú thích). Khi dịch và chạy chương trình, dòng này không được dịch và cũng không thi hành lệnh gì cả. Mục đích của việc ghi chú này giúp chương trình rõ ràng hơn. Sau này bạn đọc lại chương trình biết chương trình làm gì.
- Dòng thứ 2: chứa phát biểu tiền xử lý **#include <stdio.h>**. Vì trong chương trình này ta sử dụng hàm thư viện của C là **printf**, do đó bạn cần phải có khai báo của hàm thư viện này để báo cho trình biên dịch C biết. Nếu không khai báo chương trình sẽ báo lỗi.
- Dòng thứ 3: **int main()** là thành phần chính của mọi chương trình C (bạn có thể viết main() hoặc void main() hoặc main(void)). Mọi chương trình C đều bắt đầu thi hành từ hàm main. Cặp dấu ngoặc () cho biết đây là khối hàm (function).
- Dòng thứ 4 và 7: cặp dấu ngoặc móc {} giới hạn thân của hàm. Thân hàm bắt đầu bằng dấu { và kết thúc bằng dấu }.
- Dòng thứ 5: **printf("Bài học C đầu tiên.");**, chỉ thị cho máy in ra chuỗi ký tự nằm trong nháy kép (""). Hàng này được gọi là một câu lệnh, kết thúc một câu lệnh trong C phải là dấu chấm phẩy (;).

Dòng thứ 6: **return 0 ;** trả về giá trị nguyên cho hàm int main().

Chú ý:

- Các từ include, stdio.h, void, main, printf phải viết bằng chữ thường.
- Chuỗi trong nháy kép cần in ra "Bạn có thể viết chữ HOA, thường tùy, ý".
- Kết thúc câu lệnh phải có dấu chấm phẩy.
- Kết thúc tên hàm không có dấu chấm phẩy hoặc bất cứ dấu gì.
- Ghi chú phải đặt trong cặp /* */.
- Thân hàm phải được bao bởi cặp { }.
- Các câu lệnh trong thân hàm phải viết thụt vào.

Bạn nhập đoạn chương trình trên vào máy. Dịch, chạy và quan sát kết quả.



Build and run : Dịch và chạy chương trình.

Sau khi bạn nhập xong đoạn chương trình vào máy. Bạn gõ F9 để dịch và chạy chương trình. Khi đó sẽ xuất hiện cửa sổ kết quả.

Bây giờ bạn sửa lại dòng thứ 5 bằng câu lệnh printf("Bài học C đầu tiên.\n");, sau đó dịch và chạy lại chương trình, quan sát kết quả.

→ **Kết quả in ra màn hình**

Ở dòng bạn vừa sửa có thêm \n, \n là ký hiệu xuống dòng sử dụng trong lệnh printf. Sau đây là một số ký hiệu khác.

+ Các ký tự điều khiển:

\n : Nhảy xuống dòng kế tiếp canh về cột đầu tiên.
\t : Canh cột tab ngang.
\r : Nhảy về đầu hàng, không xuống hàng.
\a : Tiếng kêu bip.

+ Các ký tự đặc biệt:

\\ : In ra dấu \
\" : In ra dấu "
\' : In ra dấu '

Bây giờ bạn sửa lại dòng thứ 5 bằng câu lệnh printf("\tBai hoc C dau tien.\a\n");, sau đó dịch và chạy lại chương trình, quan sát kết quả.

→ Kết quả in ra màn hình

Bai hoc C dau tien.

Khi chạy chương trình bạn nghe tiếng bip phát ra từ loa.

Dòng	
1	/* Chuong trinh in ra cau bai hoc C dau tien */
2	#include <stdio.h>
3	int main()
4	{
5	printf("\t\tBai hoc C \rdau tien.\n");
6	return 0;
7	}

→ Kết quả in ra màn hình

dau tien. Bai hoc C

Ví dụ 2

Dòng	
1	/* Chuong trinh nhap va in ra man hinh gia tri bien*/
2	#include <stdio.h>
3	#include <conio.h>
4	
5	int main()
6	{
7	int i;
8	printf("Nhap vao mot so: ");
9	scanf("%d", &i);
10	printf("So ban vua nhap la: %d.\n", i);
11	return 0;
12	}

→ **Kết quả in ra màn hình**

Nhap vao mot so: 15
So ban vua nhap la: 15.
—

- Dòng thứ 7: **int i;** là lệnh khai báo, mẫu tự i gọi là tên biến. Biến là một vị trí trong bộ nhớ dùng lưu trữ giá trị nào đó mà chương trình sẽ lấy để sử dụng. Mỗi biến phải thuộc một kiểu dữ liệu. Trong trường hợp này ta sử dụng biến i kiểu số nguyên (integer) viết tắt là int.
- Dòng thứ 9: **scanf("%d", &i);** Sử dụng hàm scanf để nhận từ người sử dụng một trị nào đó. Hàm scanf trên có 2 đối mục. Đối mục **"%d"** được gọi là chuỗi định dạng, cho biết loại dữ kiện mà người sử dụng sẽ nhập vào. Chẳng hạn, ở đây phải nhập vào là số nguyên. Đối mục thứ 2 **&i** có dấu & đi đầu gọi là address operator, dấu & phối hợp với tên biến cho hàm scanf biến đem trị gõ từ bàn phím lưu vào biến i.
- Dòng thứ 10: **printf("So ban vua nhap la: %d.\n", i);** Hàm này có 2 đối mục. Đối mục thứ nhất là một chuỗi định dạng có chứa chuỗi văn bản **So ban vua nhap la:** và **%d** (ký hiệu khai báo chuyển đổi dạng thức) cho biết số nguyên sẽ được in ra. Đối mục thứ 2 là i cho biết giá trị lấy từ biến i để in ra màn hình.

```
1  /* Chuong trinh nhap vao 2 so a, b in ra tong*/
2  #include <stdio.h>
3  #include <conio.h>
4  int main()
5  {
6      int a, b;
7      printf("Nhap vao so a: ");
8      scanf("%d", &a);
9      printf("Nhap vao so b: ");
10     scanf("%d", &b);
11     printf("Tong cua 2 so %d va %d la %d.\n", a, b, a+b);
12     return 0;
13 }
```

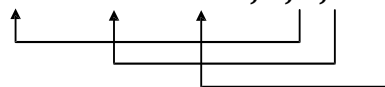
Bạn nhập đoạn chương trình trên vào máy. Dịch, chạy và quan sát kết quả.

Ví dụ 3

→ **Kết quả in ra màn hình**

Nhap vao so a: 4
Nhap vao so b: 14
Tong cua 2 so 4 va 14 la 18.

- Dòng thứ 12: **printf("Tong cua 2 so %d va %d la %d.\n", a, b, a+b);**



Bạn nhập đoạn chương trình trên vào máy. Dịch, chạy và quan sát kết quả.

Ví dụ 4

```

1  /* Chương trình nhập vào bán kính hình tròn. Tính chu vi hình tròn */
2  #include <stdio.h>
3  #include <conio.h>
4
5  #define PI    3.14
6
7  int main()
8  {
9      float fR;
10     printf("Nhập vào bán kính hình tròn: ");
11     scanf("%f", &fR);
12     printf("Chu vi hình tròn: %.2f.\n", 2*PI*fR);
13     return 0;
14 }

```

→ **Kết quả in ra màn hình**

Nhập vào bán kính hình tròn: 1
Chu vi hình tròn: 6.28

■ Dòng thứ 5: **#define PI 3.14**, dùng chỉ thị define để định nghĩa hằng số PI có giá trị 3.14. Trước define phải có dấu # và cuối dòng không có dấu chấm phẩy.

■ Dòng thứ 12: **printf("Chu vi hình tròn: %.2f.\n", 2*PI*fR);**. Hàm này có 2 đối mục. Đối mục thứ nhất là một chuỗi định dạng có chứa chuỗi văn bản **Chu vi hình tròn:** và **%.2f** (ký hiệu khai báo chuyển đổi dạng thức) cho biết dạng số chấm động sẽ được in ra, trong đó **.2** nghĩa là in ra với 2 số lẻ. Đối mục thứ 2 là biểu thức hằng **2*PI*fR**;

Bạn nhập đoạn chương trình trên vào máy. Dịch, chạy và quan sát kết quả.

CÂU HỎI VÀ BÀI TẬP THỰC HÀNH

- Viết chương trình nhập vào 2 số nguyên a và b, tính hiệu hai số đó.
- Viết chương trình nhập vào 2 số nguyên a và b, tính tích hai số đó.
- Viết chương trình nhập vào 2 số nguyên a và b, tính thương hai số đó.
- Viết chương trình nhập bán kính đường tròn, tính diện tích đường tròn.
Với $S=PI*R*R$
- Viết chương trình nhập vào một số nguyên gồm 3 chữ số, đảo ngược số và hiển thị ra màn hình.
Nhập a (3 chữ số)

Chương 3. CÁC THÀNH PHẦN TRONG NGÔN NGỮ C

Học xong chương này, sinh viên sẽ nắm được các vấn đề sau:

- Bộ chữ viết trong C.
- Các từ khóa.
- Danh biểu.
- Các kiểu dữ liệu
- Biến và các biểu thức trong C.
- Cấu trúc của một chương trình viết bằng ngôn ngữ lập trình C

3.1. BỘ KÝ TỰ, TỪ KHÓA, TÊN VÀ LỜI GIẢI THÍCH

3.1.1. Bộ ký tự

Bộ chữ viết trong ngôn ngữ C bao gồm những ký tự, ký hiệu sau: (*phân biệt chữ in hoa và in thường*):

26 chữ cái latin lớn A,B,C...Z

26 chữ cái latin nhỏ a,b,c ...z.

10 chữ số thập phân 0,1,2...9.

Các ký hiệu toán học: +, -, *, /, =, <, >, (,)

Các ký hiệu đặc biệt: ::, ;, ' ' _ @ # \$! ^ [] { } ...

Dấu cách hay khoảng trống.

3.1.2. Các từ khoá trong C

Từ khóa là các từ dành riêng (reserved words) của C mà người lập trình có thể sử dụng nó trong chương trình tùy theo ý nghĩa của từng từ. Ta không được dùng từ khóa để đặt cho các tên của riêng mình. Các từ khóa của Turbo C 3.0 bao gồm:

asm	auto	break	case	cdecl	char
class	const	continue	_cs	default	delete
do	double	_ds	else	enum	_es
extern	_export	far	_fastcall	float	for
friend	goto	huge	if	inline	int
interrupt	_loadds	long	near	new	operator
pascal	private	protected	public	register	return
_saverregs	_seg	short	signed	sizeof	_ss
static	struct	switch	template	this	typedef
union	unsigned	virtual	void	volatile	while

3.1.3. Tên (danh biểu)

Tên hay còn gọi là danh biểu (identifier) được dùng để đặt cho chương trình, hằng, kiểu, biến, chương trình con... Tên có hai loại là tên chuẩn và tên do người lập trình đặt. Tên chuẩn là tên do C đặt sẵn như tên kiểu: int, char, float,...; tên hàm: sin, cos...

Tên do người lập trình đặt để dùng trong chương trình của mình. Sử dụng bộ chữ cái, chữ số và dấu gạch dưới (_) để đặt tên, nhưng phải tuân thủ quy tắc:

- Bắt đầu bằng một chữ cái hoặc dấu gạch dưới.
- Không có khoảng trống ở giữa tên.

- Không được trùng với từ khóa.
- Độ dài tối đa của tên là không giới hạn, tuy nhiên chỉ có 31 ký tự đầu tiên là có ý nghĩa.
- Không cấm việc đặt tên trùng với tên chuẩn nhưng khi đó ý nghĩa của tên chuẩn không còn giá trị nữa.

Ví dụ 1: Tên hợp lệ: Chieu_dai, Chieu_Rong, Chu_Vi, Dien_Tich

Tên không hợp lệ: Do Dai, 12A2,...

3.1.4. Cặp dấu ghi chú thích

Khi viết chương trình đôi lúc ta cần phải có vài lời ghi chú về 1 đoạn chương trình nào đó để dễ nhớ và dễ điều chỉnh sau này; nhất là phần nội dung ghi chú phải không thuộc về chương trình (khi biên dịch phần này bị bỏ qua). Trong ngôn ngữ lập trình C, nội dung chú thích phải được viết trong cặp dấu `/*` và `*/`.

Ví dụ 2 :

```
#include <stdio.h>
#include <conio.h>
int main ()
{
    char ten[50]; /* khai bao bien ten kieu char 50 ky tu */
                /*Xuat chuoi ra man hinh*/
    printf("Xin cho biet ten cua ban: ");
    scanf("%s",ten); /*Doc vao 1 chuoi la ten cua ban*/
    printf("Xin chao ban %s\n",ten);
    printf("Chao mung ban den voi Ngon ngu lap trinh C");
    /*Dung chuong trinh, cho go phim*/
    return 0;
}
```

3.2. CÁC KIỂU DỮ LIỆU CƠ BẢN

Các kiểu dữ liệu sơ cấp chuẩn trong C có thể được chia làm 2 dạng : kiểu số nguyên, kiểu số thực.

TT	Kiểu dữ liệu (Type)	Kích thước (Length)	Miền giá trị (Range)
1	unsigned char	1 byte	0 đến 255
2	char	1 byte	- 128 đến 127
3	enum	2 bytes	- 32,768 đến 32,767
4	unsigned int	2 bytes	0 đến 65,535
5	short int	2 bytes	- 32,768 đến 32,767
6	int	2 bytes	- 32,768 đến 32,767
7	unsigned long	4 bytes	0 đến 4,294,967,295
8	long	4 bytes	- 2,147,483,648 đến 2,147,483,647
9	float	4 bytes	$3.4 * 10^{-38}$ đến $3.4 * 10^{38}$
10	double	8 bytes	$1.7 * 10^{-308}$ đến $1.7 * 10^{308}$
11	long double	10 bytes	$3.4 * 10^{-4932}$ đến $1.1 * 10^{4932}$

3.3. HẰNG (Constant)

Là đại lượng không đổi trong suốt quá trình thực thi của chương trình.

Hằng có thể là một chuỗi ký tự, một ký tự, một con số xác định. Chúng có thể được biểu diễn hay định dạng (Format) với nhiều dạng thức khác nhau.

3.3.1. Khai báo hằng

Có thể thực hiện khai báo theo một trong hai cách sau:

Dạng 1: `const Kiểu Tên_hằng = Giá_trị;`

Dạng 2: `#define Tên_hằng Giá_trị;`

Giá_trị: thông thường là hằng hoặc biểu thức hằng của các hằng được định nghĩa. Đối với cách khai báo `#define` thì Giá_trị có thể nhận bất kì đối tượng nào. Hơn nữa câu lệnh `#define` còn được xem là câu lệnh tiền xử lí, chúng ta có thể đặt chúng ở ngoài các hàm, ở đầu chương trình, bắt đầu của một khối hoặc có thể lẫn giữa các khai báo khác.

Ví dụ 3:

`Const float r=10.51;`

`#define pi 3.14;`

`-123.45E4      = -1234500 ( là -123.45 *104)`

3.3.2. Hằng số nguyên

Số nguyên gồm các kiểu `int` (2 bytes) , `long` (4 bytes) được thể hiện theo những cách sau.

- *Hằng số nguyên 2 bytes (int) hệ thập phân*: Là kiểu số mà chúng ta sử dụng thông thường, hệ thập phân sử dụng các ký số từ 0 đến 9 để biểu diễn một giá trị nguyên.

Ví dụ 4: `123` (một trăm hai mươi ba), `-242` (trừ hai trăm bốn mươi hai).

- *Hằng số nguyên 2 byte (int) hệ bát phân*: Là kiểu số nguyên sử dụng 8 ký số từ 0 đến 7 để biểu diễn một số nguyên.

Cách biểu diễn: `0<các ký số từ 0 đến 7>` Ví dụ : `0345` (số 345 trong hệ bát phân)
`-020` (số -20 trong hệ bát phân)

- *Hằng số nguyên 2 byte (int) hệ thập lục phân*: Là kiểu số nguyên sử dụng 10 ký số từ 0 đến 9 và 6 ký tự A, B, C, D, E ,F để biểu diễn một số nguyên.

Cách biểu diễn: `0x<các ký số từ 0 đến 9 và 6 ký tự từ A đến F>`

Ví dụ 5:

`0x345` (số 345 trong hệ 16)

`0x20` (số 20 trong hệ 16)

`0x2A9` (số 2A9 trong hệ 16)

- *Hằng số nguyên 4 byte (long)*: Số long (số nguyên dài) được biểu diễn như số `int` trong hệ thập phân và kèm theo ký tự `l` hoặc `L`. Một số nguyên nằm ngoài miền giá trị của số `int` (2 bytes) là số `long` (4 bytes).

Ví dụ 6: `45345L` hay `45345l` hay `45345`

- *Các hằng số còn lại*: Viết như cách viết thông thường (không có dấu phân cách giữa 3 số)

Ví dụ 7:

12 (mười hai)

12.45 (mười hai chấm 45)

1345.67 (một ba trăm bốn mươi lăm chấm sáu mươi bảy)

3.3.3. Hằng số thực

Số thực bao gồm các giá trị kiểu float, double, long double được thể hiện theo 2 cách sau:

- *Cách 1*: Sử dụng cách viết thông thường mà chúng ta đã sử dụng trong các môn Toán, Lý, ... Điều cần lưu ý là sử dụng dấu thập phân là dấu chấm (.);

Ví dụ 8: 123.34 -223.333 3.00 -56.0

- *Cách 2*: Sử dụng cách viết theo số mũ hay số khoa học. Một số thực được tách làm 2 phần, cách nhau bằng ký tự e hay E

Phần giá trị: là một số nguyên hay số thực được viết theo cách 1.

Phần mũ: là một số nguyên

Giá trị của số thực là: *Phần giá trị nhân với 10 mũ phần mũ.*

Ví dụ 9: $1234.56e-3 = 1.23456$ (là số $1234.56 * 10^{-3}$)

$0x345=827$, $0x20=32$, $0x2A9= 681$

3.3.4. Hằng ký tự

Hằng ký tự là một ký tự riêng biệt được viết trong cặp dấu nháy đơn ('). Mỗi một ký tự tương ứng với một giá trị trong bảng mã ASCII. Hằng ký tự cũng được xem như trị số nguyên.

Ví dụ 10: 'a', 'A', '0', '9' 9

húng ta có thể thực hiện các phép toán số học trên 2 ký tự (thực chất là thực hiện phép toán trên giá trị ASCII của chúng)

3.3.5. Hằng chuỗi ký tự

Hằng chuỗi ký tự là một chuỗi hay một xâu ký tự được đặt trong cặp dấu nháy kép (").

Ví dụ 11: "Ngon ngu lap trinh C", "Khoa CNTT-DHCT", "NVLinh-DVHieu"

Chú ý:

1. Một chuỗi không có nội dung "" được gọi là chuỗi rỗng.
2. Khi lưu trữ trong bộ nhớ, một chuỗi được kết thúc bằng ký tự NULL ('\0': mã Ascii là 0).
3. Để biểu diễn ký tự đặc biệt bên trong chuỗi ta phải thêm dấu \ phía trước

Ví dụ: "Day la ky tu "dac biet"" phải viết "Day la ky tu \"dac biet\""

3.4. BIẾN

Biến là một đại lượng được người lập trình định nghĩa và được đặt tên thông qua việc khai báo biến. Biến dùng để chứa dữ liệu trong quá trình thực hiện chương trình và giá trị của biến có thể bị thay đổi trong quá trình này. Cách đặt tên biến giống như cách đặt tên đã nói trong phần trên.

Mỗi biến thuộc về một kiểu dữ liệu xác định và có giá trị thuộc kiểu đó.

3.4.1. Cú pháp khai báo biến:

<Kiểu dữ liệu> **Danh sách các biến;**

→ các biến ngăn cách nhau bởi dấu phẩy “,”

Ví dụ 12 :

```
int    a, b, c;           /*Ba biến a, b,c có kiểu int*/
long int chu_vi;         /*Biến chu_vi có kiểu long*/
float   nua_chu_vi;       /*Biến nua_chu_vi có kiểu float*/
double dien_tich;        /*Biến dien_tich có kiểu double*/
```

Lưu ý: Để kết thúc 1 lệnh phải có dấu chấm phẩy (;) ở cuối lệnh.

3.4.2. Vị trí khai báo biến trong C

Trong ngôn ngữ lập trình C, ta phải khai báo biến đúng vị trí. Nếu khai báo (đặt các biến) không đúng vị trí sẽ dẫn đến những sai sót ngoài ý muốn mà người lập trình không lường trước (hiệu ứng lè). Chúng ta có 2 cách đặt vị trí của biến như sau:

a) Khai báo biến ngoài: Các biến này được đặt bên ngoài tất cả các hàm và nó có tác dụng hay ảnh hưởng đến toàn bộ chương trình (còn gọi là biến toàn cục).

Ví dụ 13:

```
int      i;               /*Bien ben ngoai */
float    pi;              /*Bien ben ngoai*/
int main()
{ ... }
```

b) Khai báo biến trong: Các biến được đặt ở bên trong hàm, chương trình chính hay một khối lệnh. Các biến này chỉ có tác dụng hay ảnh hưởng đến hàm, chương trình hay khối lệnh chứa nó. Khi khai báo biến, phải đặt các **biến này ở đầu của khối lệnh**, trước các lệnh gán, ...

Ví dụ 14:

```
#include <stdio.h>
#include<conio.h>
int bienngoai;           /*khai bao bien ngoai*/
int main ()
{   int j,I; /*khai bao bien ben trong chuong trinh chinh*/
    i=1; j=2;
    bienngoai=3;
    printf("\n Gia tri cua i la %d",i);
        /*%d la so nguyen, se biet sau */
    printf("\n Gia tri cua j la %d",j);
    printf("\n Gia tri cua bienngoai la %d",bienngoai);
    return 0;
}
```

Ví dụ 15:

```
#include <stdio.h>
#include<conio.h>
```

```

int main ()
{
    int i, j;
    i=4; j=5;
    printf("\n Gia tri cua i la %d",i);
    printf("\n Gia tri cua j la %d",j);
    if(j>i)
    {
        int hieu=j-i;
        printf("\n Hieu so cua j tru i la %d",hieu);
    }
    else
    {
        int hieu=i-j;
        printf("\n Gia tri cua i tru j la %d",hieu);
    }
    return 0;
}

```

3.5. BIỂU THỨC VÀ PHÉP TOÁN

3.5.1. Biểu thức

Biểu thức là một sự kết hợp giữa các toán tử (operator) và các toán hạng (operand) theo đúng một trật tự nhất định. Mỗi toán hạng có thể là một hằng, một biến hoặc một biểu thức khác. Trong trường hợp, biểu thức có nhiều toán tử, ta dùng cặp dấu ngoặc đơn () để chỉ định toán tử nào được thực hiện trước.

Ví dụ: Biểu thức nghiệm của phương trình bậc hai: $(-b + \sqrt{\Delta})/(2*a)$
 Trong đó 2 là hằng; a, b, Delta là biến.

3.5.2. Các phép toán

3.5.2.1. Các toán tử số học

Trong ngôn ngữ C, các toán tử +, -, *, / làm việc tương tự như khi chúng làm việc trong các ngôn ngữ khác. Ta có thể áp dụng chúng cho đa số kiểu dữ liệu có sẵn được cho phép bởi C. Khi ta áp dụng phép / cho một số nguyên hay một ký tự, bất kỳ phần dư nào cũng bị cắt bỏ. Chẳng hạn, 5/2 bằng 2 trong phép chia nguyên.

Toán tử	Ý nghĩa
+	Cộng
-	Trừ
*	Nhân
/	Chia
%	Chia lấy phần dư
--	Giảm 1 đơn vị
++	Tăng 1 đơn vị

Tăng và giảm (++ & --)

Toán tử ++ thêm 1 vào toán hạng của nó và – trừ bớt 1. Nói cách khác:

$x = x + 1$ giống như ++x

$x = x - 1$ giống như x--

Cả 2 toán tử tăng và giảm đều có thể tiền tố (đặt trước) hay hậu tố (đặt sau) toán hạng.

Ví dụ 16: $x = x + 1$ có thể viết x++ (hay ++x)

Tuy nhiên giữa tiền tố và hậu tố có sự khác biệt khi sử dụng trong 1 biểu thức. Khi 1 toán tử tăng hay giảm đứng trước toán hạng của nó, C thực hiện việc tăng

hay giảm trước khi lấy giá trị dùng trong biểu thức. Nếu toán tử đi sau toán hạng, C lấy giá trị toán hạng trước khi tăng hay giảm nó. Tóm lại:

$x = 10$

$y = ++x \ //y = 11$

Tuy nhiên:

$x = 10$

$x = x++ \ //y = 10$

Thứ tự ưu tiên của các toán tử số học:

$++$ $--$ sau đó là $*$ $/$ $\%$ rồi mới đến $+$ $-$

3.5.2.2. Các toán tử quan hệ và các toán tử Logic

Ý tưởng chính của toán tử quan hệ và toán tử Logic là đúng hoặc sai. Trong C mọi giá trị khác 0 được gọi là đúng, còn sai là 0. Các biểu thức sử dụng các toán tử quan hệ và Logic trả về 0 nếu sai và trả về 1 nếu đúng.

Toán tử	Ý nghĩa
Các toán tử quan hệ	
$>$	Lớn hơn
$>=$	Lớn hơn hoặc bằng
$<$	Nhỏ hơn
$<=$	Nhỏ hơn hoặc bằng
$=$	Bằng
$!=$	Khác
Các toán tử Logic	
$\&\&$	AND
$\ \ $	OR
$!$	NOT

Bảng chân trị cho các toán tử Logic:

P	q	$p\&\&q$	$p\ \ q$	$!p$
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

Các toán tử quan hệ và Logic đều có độ ưu tiên thấp hơn các toán tử số học. Do đó một biểu thức như: $10 > 1 + 12$ sẽ được xem là $10 > (1 + 12)$ và kết quả là sai (0).

Ta có thể kết hợp vài toán tử lại với nhau thành biểu thức như sau:

$10 > 5 \&\& !(10 < 9) \|\| 3 <= 4$ Kết quả là đúng

Thứ tự ưu tiên của các toán tử quan hệ là Logic

Cao nhất: $!$

$>$ $>=$ $<$ $<=$

$==$ $!=$

$\&\&$

Thấp nhất: $\|\|$

3.5.2.3 Cách viết tắt trong C

Có nhiều phép gán khác nhau, đôi khi ta có thể sử dụng viết tắt trong C nữa. Chẳng hạn:

$x = x + 10$ được viết thành $x += 10$

Toán tử $+=$ báo cho chương trình dịch biết để tăng giá trị của x lên 10.

Cách viết này làm việc trên tất cả các toán tử nhị phân (phép toán hai ngôi) của C. Tổng quát:

(Biến) = (Biến) (Toán tử) (Biểu thức)

có thể được viết:

(Biến) (Toán tử) = (Biểu thức)

3.6. CÂU LỆNH TIỀN XỬ LÝ

Trong C, việc dịch (translation) một tập tin nguồn được tiến hành trên hai bước hoàn toàn độc lập với nhau:

- Tiền xử lý.
- Biên dịch.

Hai bước này trong phần lớn thời gian được nối tiếp với nhau một cách tự động theo cách thức mà ta có ấn tượng rằng nó đã được thực hiện như là một xử lý duy nhất. Nói chung, ta thường nói đến việc tồn tại của một bộ tiền xử lý (preprocessor?) nhằm chỉ rõ chương trình thực hiện việc xử lý trước. Ngược lại, các thuật ngữ trình biên dịch hay sự biên dịch vẫn còn nhập nhằng bởi vì nó chỉ ra khi thì toàn bộ hai giai đoạn, khi thì lại là giai đoạn thứ hai.

Bước tiền xử lý tương ứng với việc cập nhật trong văn bản của chương trình nguồn, chủ yếu dựa trên việc diễn giải các mã lệnh rất đặc biệt gọi là các chỉ thị dẫn hướng của bộ tiền xử lý (destination directive of preprocessor); các chỉ thị này được nhận biết bởi chúng bắt đầu bằng ký hiệu (symbol) #.

Hai chỉ thị quan trọng nhất là:

- Chỉ thị sự gộp vào của các tập tin nguồn khác: `#include`
- Chỉ thị việc định nghĩa các macros hoặc ký hiệu: `#define`

Chỉ thị đầu tiên được sử dụng trước hết là nhằm gộp vào nội dung của các tập tin cần có (header file), không thể thiếu trong việc sử dụng một cách tốt nhất các hàm của thư viện chuẩn, phổ biến nhất là:

```
#include <stdio.h>
```

Chỉ thị thứ hai rất hay được sử dụng trong các tập tin thư viện (header file) đã được định nghĩa trước đó và thường được khai thác bởi các lập trình viên trong việc định nghĩa các ký hiệu như là:

```
#define NB_COUPS_MAX 100
```

```
#define SIZE 25
```

CÂU HỎI VÀ BÀI TẬP THỰC HÀNH

Bài 1: Biểu diễn các hằng số nguyên 2 byte sau đây dưới dạng số nhị phân, bát phân, thập lục phân

- a) 12 b) 255 c) 31000 d) 32767 e) -32768

Bài 2: Biểu diễn các hằng ký tự sau đây dưới dạng số nhị phân, bát phân.

a) 'A' b) 'a' c) 'Z' d) 'z'

Bài 3: Viết chương trình đổi một số nguyên hệ 10 sang hệ 2.

Bài 4: Viết chương trình đổi một số nguyên hệ 10 sang hệ 16.

Bài 5: Viết chương trình đọc và 2 số nguyên và in ra kết quả của phép (+), phép trừ (-), phép nhân (*), phép chia (/). Nhận xét kết quả chia 2 số nguyên.

Bài 6: Viết chương trình nhập vào bán kính hình cầu, tính và in ra diện tích, thể tích của hình cầu đó.

Hướng dẫn: $S = 4\pi R^2$ và $V = (4/3)\pi R^3$.

Bài 7: Viết chương trình nhập vào một số a bất kỳ và in ra giá trị bình phương (a^2), lập phương (a^3) của a và giá trị a^4 .

Bài 8: Viết chương trình đọc từ bàn phím 3 số nguyên biểu diễn ngày, tháng, năm và xuất ra màn hình dưới dạng "ngay/thang/nam" (chỉ lấy 2 số cuối của năm).

Bài 9: Viết chương trình nhập vào số giây từ 0 đến 86399, đổi số giây nhập vào thành dạng "gio:phut:giay", mỗi thành phần là một số nguyên có 2 chữ số.

Ví dụ: 02:11:05

Chương 4. NHẬP / XUẤT DỮ LIỆU

4.1. LỆNH GÁN

Lệnh gán (assignment statement) dùng để gán giá trị của một biểu thức cho một biến.

Cú pháp: <Tên biến> = <biểu thức>

Ví dụ 1:

```
int main() {
    int x, y;
    x = 10; /*Gán hằng số 10 cho biến x*/
    y = 2*x; /*Gán giá trị 2*x=2*10=20 cho x*/
    return 0;
}
```

Nguyên tắc khi dùng lệnh gán là kiểu của biến và kiểu của biểu thức phải giống nhau, gọi là có sự tương thích giữa các kiểu dữ liệu. Chẳng hạn ví dụ sau cho thấy một sự không tương thích về kiểu:

```
int main()
{
    int x, y;
    x = 10; /*Gán hằng số 10 cho biến x*/
    y = "Xin chào";
    /*y có kiểu int, còn "Xin chào" có kiểu char* */
    return 0;
}
```

Khi biên dịch chương trình này, C sẽ báo lỗi "*Cannot convert 'char *' to 'int'*" tức là C không thể tự động chuyển đổi kiểu từ char * (chuỗi ký tự) sang int.

Tuy nhiên trong đa số trường hợp sự tự động biến đổi kiểu để sự tương thích về kiểu sẽ được thực hiện.

Ví dụ 2:

```
int main()
{ int x, y;
  float r;
  char ch;
  r = 9000;
  x = 10; /* Gán hằng số 10 cho biến x */
  y = 'd'; /* y có kiểu int, còn 'd' có kiểu char*/
  r = 'e'; /* r có kiểu float, 'e' có kiểu char*/
  ch = 65.7; /* ch có kiểu char, còn 65.7 có kiểu float*/
  return 0;
}
```

Trong nhiều trường hợp để tạo ra sự tương thích về kiểu, ta phải sử dụng đến cách thức chuyển đổi kiểu một cách tường minh. Cú pháp của phép toán này như sau:

(Tên kiểu) <Biểu thức>

Chuyển đổi kiểu của <Biểu thức> thành kiểu mới <Tên kiểu>. Chẳng hạn như:

```
Float f;
f= (float) 10/4; /* f lúc này là 2.5*/
```


Chú ý:

- Khi một biểu thức được gán cho một biến thì giá trị của nó sẽ thay thế giá trị cũ mà biến đã lưu giữ trước đó.

- Trong câu lệnh gán, dấu = là một toán tử; do đó nó có thể được sử dụng là một thành phần của biểu thức. Trong trường hợp này giá trị của biểu thức gán chính là giá trị của biến.

Ví dụ 3:

```
int x, y;  
y = x = 3; /* y lúc này cùng bằng 3*/
```

- Ta có thể gán trị cho biến lúc biến được khai báo theo cách thức sau:

<Tên kiểu> <Tên biến> = <Biểu thức>;

Ví dụ 4: `int x = 10, y=x;`

4.2. XUẤT DỮ LIỆU VỚI HÀM `printf()`

Kết xuất dữ liệu được định dạng.

Cú pháp

`printf("chuỗi định dạng", [đối mục 1, đối mục 2, ...]);`

→ Khi sử dụng hàm phải khai báo tiền xử lý `#include <stdio.h>`

- `printf`: tên hàm, **phải viết bằng chữ thường**.
- đối mục 1, ...: là các mục dữ kiện cần in ra màn hình. Các đối mục này có thể là biến, hằng hoặc biểu thức phải được định trị trước khi in ra.
- chuỗi định dạng: được đặt trong cặp nháy kép (" "), gồm 3 loại:
 - + Đối với chuỗi kí tự ghi như thế nào in ra giống như vậy.
 - + Đối với những kí tự chuyển đổi dạng thức cho phép kết xuất giá trị của các đối mục ra màn hình tạm gọi là mã định dạng. Sau đây là các dấu mô tả định dạng:

`%c` : Kí tự đơn

`%s` : Chuỗi

`%d` : Số nguyên thập phân có dấu

`%f` : Số chấm động (ký hiệu thập phân)

`%e` : Số chấm động (ký hiệu có số mũ)

`%g` : Số chấm động (`%f` hay `%g`)

`%x` : Số nguyên thập phân không dấu

`%u` : Số nguyên hex không dấu

`%o` : Số nguyên bát phân không dấu

`l` : Tiền tố dùng kèm với `%d`, `%u`, `%x`, `%o` để chỉ số nguyên dài (ví dụ `%ld`)

Ví dụ 1: `printf("Bai hoc C dau tien. \n");`

↳ ký tự điều khiển
chuỗi ký tự

Ví dụ 2: `printf("Ma dinh dang \\\\" in ra dau \". \n");`

↳ ký tự điều khiển ký tự đặc biệt
chuỗi ký tự

→ **Kết quả in ra màn hình**

Ma dinh dang "\" in ra dau ".

Ví dụ 3: giả sử biến `i` có giá trị = 5

`printf("So ban vua nhap la: %d . \n", i);`
 xuất giá trị biến i
 đối mục là biến (kiểu int)
 ký tự điều khiển
 chuỗi ký tự
 mã định dạng (kiểu int)

→ **Kết quả in ra màn hình**

So ban vua nhap la: 5.

—

Ví dụ 4: giả sử biến a có giá trị = 7 và b có giá trị = 4

`printf("Tong cua 2 so %d va %d la %d . \n", a, b, a+b);`
 xuất giá trị biểu thức a+b
 xuất giá trị biến b
 xuất giá trị biến a
 đối mục 3 là biểu thức có giá trị là kiểu int
 đối mục 1, 2 là biến (kiểu int)
 ký tự điều khiển
 chuỗi ký tự
 mã định dạng (kiểu int)

→ **Kết quả in ra màn hình**

Tong cua 2 so 7 va 4 la 11.

—

Ví dụ 5: sửa lại ví dụ 4

`printf("Tong cua 2 so %5d va %3d la %1d . \n", a, b, a+b);`
 Bề rộng trường

→ **Kết quả in ra màn hình**

Tong cua 2 so 7 va 4 la 11.

—

2 ký tự (mặc dù định dạng là 1)
 3 ký tự
 5 ký tự

Ví dụ 6: sửa lại ví dụ 5

`printf("Tong cua 2 so %-5d va %-3d la %-1d . \n", a, b, a+b);`

→ **Kết quả in ra màn hình**

Tong cua 2 so 7 va 4 la 11.

—

2 ký tự (mặc dù định dạng là 1)
 3 ký tự
 5 ký tự

→ Dấu trừ trước bề rộng trường sẽ kéo kết quả sang trái

Ví dụ 7: sửa lại ví dụ 4

```
printf("Tong cua 2 so %02d va %02d la %04d . \n", a, b, a+b);
```

→ **Kết quả in ra màn hình**

Tong cua 2 so 07 va 04 la 0011.	
—	
	thêm 2 số 0 trước -> đủ 4 ký tự
	thêm 1 số 0 trước -> đủ 2 ký tự
	thêm 1 số 0 trước -> đủ 2 ký tự

Ví dụ 8: giả sử int a = 6, b = 1234, c = 62

```
printf("%7d%7d%7d.\n", a, b, c);  
printf("%7d%7d%7d.\n", 165, 2, 965);
```

→ **Kết quả in ra màn hình**

6 1234 62 165 2 965 —	Số canh về bên phải bề rộng trường.
-----------------------------	-------------------------------------

```
printf("%-7d%-7d%-7d.\n", a, b, c);  
printf("%-7d%-7d%-7d.\n", 165, 2, 965);
```

→ **Kết quả in ra màn hình**

6 1234 62 165 2 965 —	Số canh về bên trái bề rộng trường.
-----------------------------	-------------------------------------

Ví dụ 9: giả sử float a = 6.4, b = 1234.56, c = 62.3

```
printf("%7.2f%7.2f%7.2f.\n", a, b, c);
```

→ số số lẻ

→ **Kết quả in ra màn hình**

6.40 1234.56 62.30 —	Số canh về bên phải bề rộng trường.
	→ 7 ký tự

→ Bề rộng trường bao gồm: phần nguyên, phần lẻ và dấu chấm động

Ví dụ 10: giả sử float a = 6.4, b = 1234.55, c = 62.34

```
printf("%10.1f%10.1f%10.1f.\n", a, b, c);  
printf("%10.1f%10.1f%10.1f.\n", 165, 2, 965);
```

→ **Kết quả in ra màn hình**

6.4 1234.6 62.3 165.0 2.0 965.0 —	Số canh về bên phải bề rộng trường.
---	-------------------------------------

```
printf("%-10.2f%-10.2f%-10.2f\n", a, b, c);
printf("%-10.2f%-10.2f%-10.2f\n", 165, 2, 965);
```

→ **Kết quả in ra màn hình**

6.40 1234.55 62.34 165.00 2.00 965.00 —	Số canh về bên trái bề rộng trường.
---	-------------------------------------

4.3. NHẬP DỮ LIỆU VỚI HÀM *scanf()*

Định dạng khi nhập liệu.

Cú pháp

scanf ("chuỗi định dạng", [đối mục 1, đối mục 2,...]);

→ Khi sử dụng hàm phải khai báo tiền xử lý **#include <stdio.h>**

- scanf: tên hàm, **phải viết bằng chữ thường**.
- khung định dạng: được đặt trong cặp nháy kép (" ") là hình ảnh dạng dữ liệu nhập vào.
- đối mục 1,...: là danh sách các đối mục cách nhau bởi dấu phẩy, mỗi đối mục sẽ tiếp nhận giá trị nhập vào.

Ví dụ 11: scanf("%d", &i);

└─┬─> đối mục 1
└─┬─> mã định dạng

→ Nhập vào 12abc, biến i chỉ nhận giá trị 12. Nhập 3.4 chỉ nhận giá trị 3.

Ví dụ 12: scanf("%d%d", &a, &b);

→ Nhập vào 2 số a, b phải cách nhau bằng **khoảng trắng** hoặc **enter**.

Ví dụ 13: scanf("%d/%d/%d", &ngay, &thang, &nam);

→ Nhập vào ngày, tháng, năm theo dạng ngày/thang/nam (20/12/2002)

Ví dụ 14: scanf("%d%*c%d%*c%d", &ngay, &thang, &nam);

→ Nhập vào ngày, tháng, năm với dấu phân cách /, -, ...; ngoại trừ số.

Ví dụ 15: scanf("%2d%2d%4d", &ngay, &thang, &nam);

→ Nhập vào ngày, tháng, năm theo dạng dd/mm/yyyy.

CÂU HỎI VÀ BÀI TẬP THỰC HÀNH

1. Viết chương trình đổi một số nguyên hệ 10 sang hệ 2.
2. Viết chương trình đổi một số nguyên hệ 10 sang hệ 16.
3. Viết chương trình đọc và 2 số nguyên và in ra kết quả của phép (+), phép trừ (-), phép nhân (*), phép chia (/). Nhận xét kết quả chia 2 số nguyên.
4. Viết chương trình nhập vào bán kính hình cầu, tính và in ra diện tích, thể tích của hình cầu đó.

Hướng dẫn: $S = 4\pi R^2$ và $V = (4/3)\pi R^3$.

5. Viết chương trình nhập vào một số a bất kỳ và in ra giá trị bình phương (a^2), lập phương (a^3) của a và giá trị a^4 .

6. Viết chương trình đọc từ bàn phím 3 số nguyên biểu diễn ngày, tháng, năm và xuất ra màn hình dưới dạng "ngay/thang/nam" (chỉ lấy 2 số cuối của năm).

7. Viết chương trình nhập vào số giây từ 0 đến 86399, đổi số giây nhập vào thành dạng "gio:phut:giay", mỗi thành phần là một số nguyên có 2 chữ số.

Ví dụ: 02:11:05

Chương 5. CẤU TRÚC RỄ NHÁNH CÓ ĐIỀU KIỆN

Sau khi hoàn tất bài này học viên sẽ hiểu và vận dụng các kiến thức kỹ năng cơ bản sau:

- Ý nghĩa lệnh, khối lệnh.
- Cú pháp, ý nghĩa, cách sử dụng lệnh if, lệnh switch.
- Một số bài toán sử dụng lệnh if, switch thông qua các ví dụ.
- So sánh, đánh giá một số bài toán sử dụng lệnh if hoặc switch.
- Cách sử dụng các cấu trúc lồng nhau.

5.1. LỆNH VÀ KHỐI LỆNH

5.1.1. Lệnh

Là một tác vụ, biểu thức, hàm, cấu trúc điều khiển...

Ví dụ 1:

```
x = x +  
2;  
printf("Day la mot lenh\n");
```

5.1.2. Khối lệnh

Là một dãy các câu lệnh được bọc bởi cặp dấu { }, các lệnh trong khối lệnh phải viết thụt vào 1 tab so với cặp dấu { }

Ví dụ 2:

```
{ //dau  
    khai a  
    = 5;  
    b = 6;  
    printf("Tong %d + %d = %d", a, b, a+b);  
} //cuoi khai
```

} viết thụt vào 1 tab so với cặp { }

→ **Quên dùng cặp dấu { } bao bọc khi sử dụng khối lệnh, hoặc mở dấu { và quên đóng dấu }**

Một khối lệnh có thể chứa bên trong nó nhiều khối lệnh khác gọi là khối lệnh lồng nhau. Sự lồng nhau của các khối lệnh là không hạn chế.

5.2. LỆNH if

Câu lệnh if cho phép lựa chọn một trong hai nhánh tùy thuộc vào giá trị của biểu thức điều kiện là đúng (true) hay sai (false) hoặc khác không hay bằng không.

5.2.1. Dạng 1 (if thiếu)

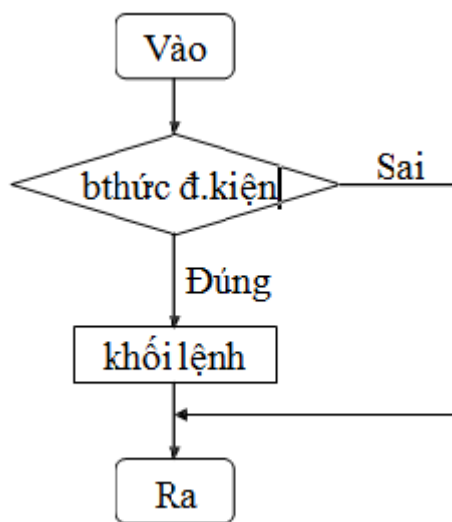
Quyết định sẽ thực hiện hay không một khối lệnh.

Cú pháp lệnh

**if (biểu thức điều kiện)
 khối lệnh;**

→ từ khóa **if** phải viết bằng chữ thường
→ kết quả của **biểu thức điều kiện** phải là
đúng (≠ 0) hoặc **sai (= 0)**

Lưu đồ



→ Nếu **biểu thức điều kiện** đúng thì thực hiện khối lệnh và thoát khỏi if, ngược lại không làm gì cả và thoát khỏi if.

Nếu **khối lệnh** bao gồm từ 2 lệnh trở lên thì phải đặt trong dấu { }

Diễn giải:

+ Khối lệnh là một lệnh ta viết lệnh if như sau:

```
if (biểu thức điều kiện)
    lệnh;
```

+ Khối lệnh bao gồm nhiều lệnh: lệnh 1, lệnh 2..., ta viết lệnh if như sau:

```
if (biểu thức điều kiện)
{
    lệnh 1;
    lệnh 2;
    ...
}
```

→ Không đặt dấu chấm phẩy sau câu lệnh if.

Ví dụ: `if(biểu thức điều kiện);`

→ trình biên dịch không báo lỗi nhưng khối lệnh không được thực hiện cho dù điều kiện đúng hay sai.

Ví dụ 3: Viết chương trình nhập vào 2 số nguyên a, b. Tìm và in ra số lớn nhất.

a. Phác họa lời giải

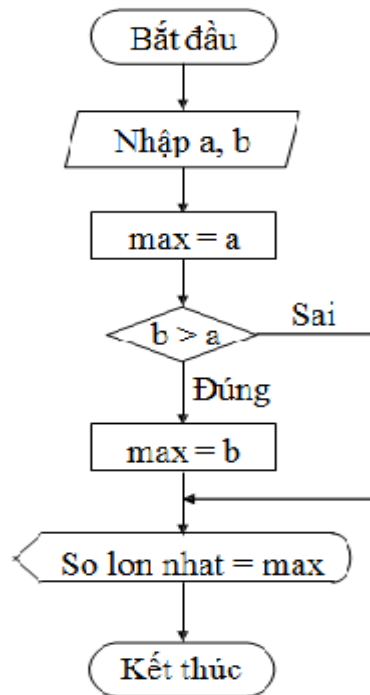
Trước tiên ta cho giá trị **a là giá trị lớn nhất bằng cách gán a cho max** (max là biến được khai báo cùng kiểu dữ liệu với a, b). Sau đó so sánh b với a, **nếu b lớn hơn a ta gán b cho max** và cuối cùng ta **được kết quả max là giá trị lớn nhất**.

b. Mô tả quy trình xử lý (giải thuật)

Ngôn ngữ tự nhiên	Ngôn ngữ
- Khai báo 3 biến a, b, max kiểu số nguyên - Nhập vào giá trị a - Nhập vào giá trị b - Gán a cho max - Nếu $b > a$ thì gán b cho max	<pre>- int ia, ib, imax; - printf("Nhap vao so a: "); scanf("%d", &ia); - printf("Nhap vao so b: "); scanf("%d", &ib); - imax = ia; - if (ib > ia) imax = ib; - printf("So lon nhat = %d.\n", imax);</pre>

→ Biểu thức luận lý phải đặt trong cặp dấu (). if $ib > ia \rightarrow$ báo lỗi

c. Mô tả bằng lưu đồ



d. Viết chương trình

/* Chương trình tìm số lớn nhất từ 2 số nguyên a, b */

```

#include <stdio.h>
#include <conio.h>

int main()
{
    int ia, ib, imax;

    printf("Nhap vao so a: ");
    scanf("%d", &ia);

    printf("Nhap vao so b: ");
    scanf("%d", &ib);
    imax = ia;
    if (ib > ia)
        imax = ib;
    printf("So lon nhat = %d.\n", imax);
}
  
```

→ Kết quả in ra màn hình

Nhap vao so a : 10
 Nhap vao so b : 8
 So lon nhat = 10.

—

Cho chạy lại chương trình và thử lại với:
 a = 7, b = 9
 a = 5, b = 5
 Quan sát và nhận xét kết quả

Ví dụ 4: Viết chương trình nhập vào 2 số nguyên a, b. Nếu a lớn hơn b thì hoán đổi giá trị a và b, ngược lại không hoán đổi. In ra giá trị a, b.

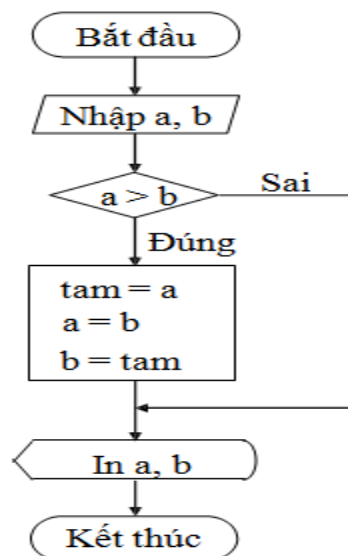
a. Phác họa lời giải

Nếu giá trị a lớn hơn giá trị b, bạn phải hoán chuyển 2 giá trị này cho nhau (nghĩa là a sẽ mang giá trị b và b mang giá trị a) bằng cách đem **giá trị a gởi (gán) cho biến tam** (biến tam được khai báo theo kiểu dữ liệu của a, b), kế đến bạn **gán giá trị b cho a** và cuối cùng bạn **gán giá trị tam cho b**, rồi in ra a, b.

b. Mô tả quy trình thực hiện (giải thuật)

Ngôn ngữ tự nhiên	Ngô
- Khai báo 3 biến a, b, tam kiểu số nguyên - Nhập vào giá trị a - Nhập vào giá trị b - Nếu $a > b$ thì - tam = a; - a = b; - b = tam; - In ra a, b	- int ia, ib, itam; - printf("Nhap vao so a: "); - scanf("%d", &ia); - printf("Nhap vao so b: "); - scanf("%d", &ib); - if (ia > ib) { - itam = ia; - ia = ib; - ib = itam; } - printf("%d, %d\n", ia, ib);

c. Mô tả bằng lưu đồ



d. Viết chương trình

```

/* Chương trình hoán vị 2 số a, b nếu a > b */
#include <stdio.h>
#include <conio.h>
int main()
{
    int ia, ib, itam;
    printf("Nhap vao so a: ");
    scanf("%d", &ia);
    printf("Nhap vao so b:
  
```

```
"); scanf("%d", &ib);
if (ia>ib)
{
    itam = ia;    //hoan vi a va
    b
    ia = ib;
    ib = itam;
}
printf("%d, %d.\n", ia, ib);
return 0;
}
```

→ **Kết quả in ra màn hình**

Nhap vao so a : 10 Nhap vao so b : 8 8, 10 —	Cho chạy lại chương trình và thử lại với: a = 1, b = 8 a = 2, b = 2 Quan sát và nhận kết quả
---	---

5.2.2. Dạng 2 (if đủ)

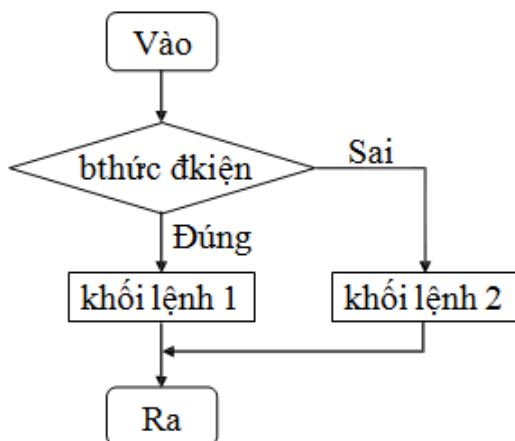
Quyết định sẽ thực hiện 1 trong 2 khối lệnh cho trước.

- Cú pháp lệnh**

```
if (biểu thức điều kiện)
    khối lệnh 1;
else
    khối lệnh 2;
```

→ Từ khóa **if**, **else** phải viết bằng chữ thường
→ kết quả của **biểu thức điều kiện** phải là **đúng (≠ 0)** hoặc **sai (= 0)**

- Lưu đồ**



Nếu **biểu thức điều kiện** đúng thì thực hiện khối lệnh 1 và thoát khỏi if ngược lại, thực hiện khối lệnh 2 và thoát khỏi if.

Nếu **khối lệnh 1**, **khối lệnh 2** bao gồm từ 2 lệnh trở lên thì phải đặt trong dấu { }

Ví dụ 5: Viết chương trình nhập vào 2 số nguyên a, b. In ra thông báo "a bằng b" nếu a = b, ngược lại in ra thông báo "a khác b".

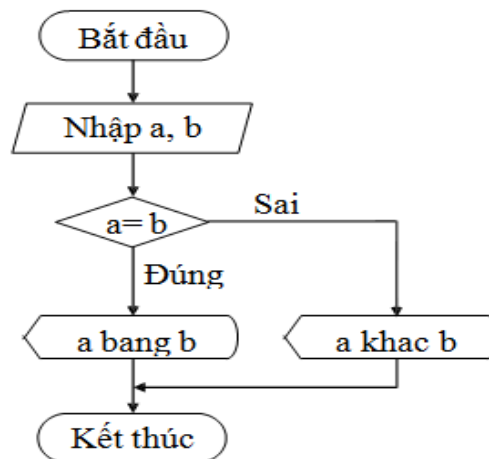
a. Phác họa lời giải

So sánh a với b, nếu a bằng b thì in ra câu thông báo "a bằng b", ngược lại in ra thông báo "a khác b".

b. Mô tả quy trình xử lý (giải thuật)

Ngôn ngữ tự nhiên	Ngôn ngữ C
- Khai báo 2 biến a, b kiểu số nguyên - Nhập vào giá trị a - Nhập vào giá trị b - Nếu a = b thì in ra thông báo "a bằng b" Ngược lại (còn không thì) in ra thông báo "a khác b"	- int ia, ib; - printf("Nhap vao so a: "); scanf("%d", &ia); - printf("Nhap vao so b: "); scanf("%d", &ib); - if (ia == ib) printf("a bang b\n"); else printf("a khac b\n");

c. Mô tả bằng lưu đồ



d. Viết chương trình

```
/* Chương trình in ra thông báo "a bằng b" nếu a = b, ngược lại in ra "a khác b" */
#include <stdio.h>
#include <conio.h>

int main()
{
    int ia, ib;
    printf("Nhap vao so a: ");
    scanf("%d", &ia);
    printf("Nhap vao so b: ");
    scanf("%d", &ib);
    if (ia == ib)
        printf("a bang b\n");
    else
        printf("a khac b\n");
    return 0;
}
```

→ **Kết quả in ra màn hình**

Nhap vào so a : 10 Nhap vào so b : 8 a khác b. _	Cho chạy lại chương trình và thử lại với: a = 6, b = 6 a = 1, b = 5 Quan sát và nhận xét kết quả
---	---

→ Sau else không có dấu chấm phẩy.

Ví dụ: `else; printf('a khác b\n');`

→ **trình biên dịch không báo lỗi, lệnh `printf("a khác b\n");` không thuộc else**

Ví dụ 6: Viết chương trình nhập vào kí tự c. Kiểm tra xem nếu kí tự nhập vào là kí tự thường trong khoảng từ 'a' đến 'z' thì đổi sang chữ in hoa và in ra, ngược lại in ra thông báo "Kí tự bạn vừa nhập là: c".

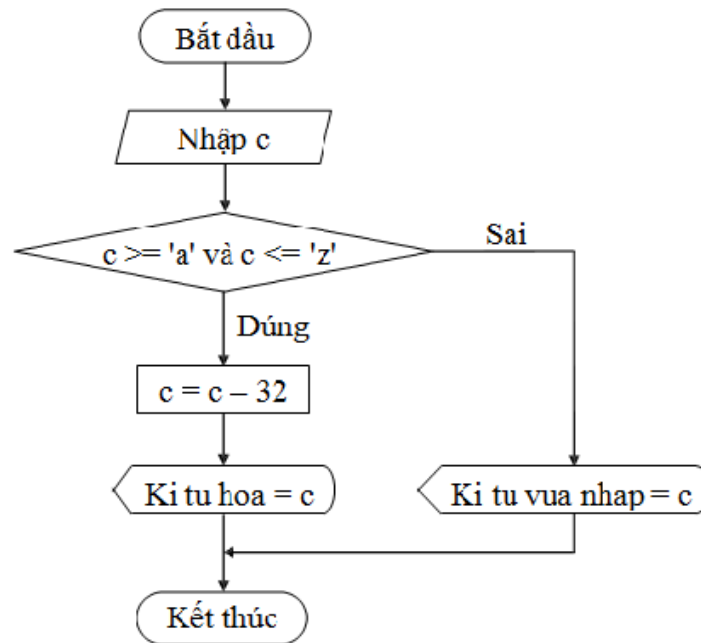
a. Phác họa lời giải

Trước tiên bạn phải kiểm tra xem nếu kí tự c thuộc khoảng 'a' và 'z' thì đổi kí tự c thành chữ in hoa bằng cách lấy kí tự c – 32 rồi gán lại cho chính nó ($c = c - 32$) (vì giữa kí tự thường và in hoa trong bảng mã ASCII cách nhau 32, ví dụ: A trong bảng mã ASCII là 65, B là 66..., còn a là 97, b là 98...), sau khi đổi xong bạn in kí tự c ra. Ngược lại, in câu thông báo "Kí tự bạn vừa nhập là: c".

b. Mô tả quy trình xử lý (giải thuật)

Ngôn ngữ tự nhiên	Ngôn ngữ C
- Khai báo biến c kiểu kí tự - Nhập vào kí tự c - Nếu $c \geq a$ và $c \leq z$ thì $c = c - 32$ in c ra màn hình - Ngược lại in ra thông báo " Kí tự bạn vừa nhập là: c "	- char c; - <code>printf("Nhap vào 1 ki tu: ");</code> <code>scanf("%c", &c);</code> - <code>if (c >= 'a' && c <= 'z')</code> <code>{</code> <code> c = c - 32;</code> <code> printf("Ki tu hoa la: %c.\n", c);</code> <code>}</code> <code>else</code> <code>printf("Ki tu ban vua nhap la: %c.\n", c);</code>

c. Mô tả bằng lưu đồ



d. Viết chương trình

```

/* Chương trình nhập vào ký tự c, nếu c là chữ thường in ra chữ IN HOA */
#include <stdio.h>
#include <conio.h>

int main()
{
    char c;
    printf("Nhập vào 1 ký tự: ");
    scanf("%c", &c);
    if (c >= 97 && c <= 'z')        //hoac if(c >= 97 && c <= 122)
    {
        c = c - 32;                //đổi thành chữ in hoa
        printf("Ký tự hoa là: %c.\n", c);
    }
    else
        printf("Ký tự bạn vừa nhập là: %c.\n", c);
    return 0;
}
  
```

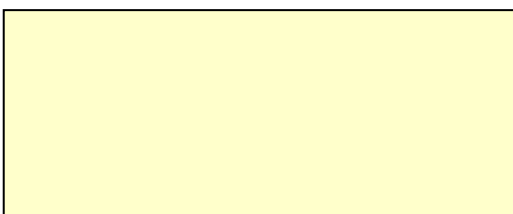
→ **Kết quả in ra màn hình**

Nhập vào một ký tự: g Ký tự hoa là: G. —	Cho chạy lại chương trình và thử lại với: c = '!', c = '2', c = 'A', c = 'u' Quan sát và nhận xét kết quả
--	---

5.2.3. Cấu trúc else if

Quyết định sẽ thực hiện 1 trong n khối lệnh cho trước.

Cú pháp lệnh



if (biểu thức điều kiện 1)
khởi lệnh 1;
 là

→ từ khóa **if, else if, else** phải viết bằng chữ thường
 → kết quả của **biểu thức điều kiện 1, 2...n** phải

else if (biểu thức điều kiện 2)
khởi lệnh 2;

đúng ($\neq 0$) hoặc sai ($= 0$)

...

else if (biểu thức điều kiện n-1)
khởi lệnh n-1;

Nếu **khởi lệnh 1, 2...n** bao gồm từ 2 lệnh trở lên thì phải đặt trong dấu { }

else
khởi lệnh n;

Nếu **biểu thức điều kiện 1** đúng thì thực hiện khởi lệnh 1 và thoát khỏi cấu trúc if

Ngược lại Nếu **biểu thức điều kiện 2** đúng thì thực hiện khởi lệnh 2 và thoát khỏi cấu trúc if

...

Ngược lại Nếu **biểu thức điều kiện n-1** đúng thì thực hiện khởi lệnh n-1 và thoát khỏi cấu trúc if

Ngược lại thì thực hiện khởi lệnh n.

Ví dụ 7: Viết chương trình nhập vào 2 số nguyên a, b. In ra thông báo "a lớn hơn b" nếu $a > b$, in ra thông báo "a nhỏ hơn b" nếu $a < b$, in ra thông báo "a bằng b" nếu $a = b$.

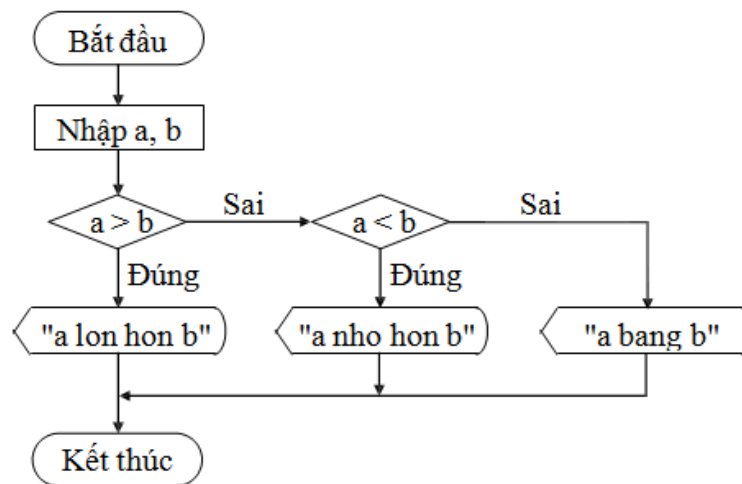
a. Phác họa lời giải

Trước tiên so sánh a với b. Nếu $a > b$ thì in ra thông báo "a lớn hơn b", ngược lại nếu $a < b$ thì in ra thông báo "a nhỏ hơn b", ngược với 2 trường hợp trên thì in ra thông báo "a bằng b".

b. Mô tả quy trình thực hiện (giải thuật)

Ngôn ngữ tự nhiên	Ngôn ngữ C
- Khai báo 2 biến a, b kiểu số nguyên - Nhập vào giá trị a - Nhập vào giá trị b - Nếu $a > b$ thì in ra thông báo "a lớn hơn b" Ngược lại Nếu $a < b$ thì in ra thông báo "a nhỏ hơn b" Ngược lại thì in ra thông báo "a bằng b"	<pre> - int ia, ib; - printf("Nhap vao so a: "); scanf("%d", &ia); - printf("Nhap vao so b: "); scanf("%d", &ib); - if (ia > ib) printf("a lon hon b.\n"); else if (ia < ib) printf("a nho hon b.\n"); else printf("a bang b.\n"); </pre>

c. Mô tả bằng lưu đồ



d. Viết chương trình

```

/* Chương trình nhập vào 2 số nguyên a, b. In ra thông báo a > b, a < b, a = b */
#include <stdio.h>
#include <conio.h>
int main()
{
    int ia, ib;
    printf("Nhập vào số a: ");
    scanf("%d", &ia);
    printf("Nhập vào số b: ");
    scanf("%d", &ib);
    if (ia > ib)
        printf("a lon hon b.\n");
    else if (ia < ib)
        printf("a nho hon b.\n");
    else
        printf("a bang b.\n");
    return 0;
}
  
```

→ **Kết quả in ra màn hình**

Nhập vào số a : 5	Cho chạy lại chương trình và thử lại với:
Nhập vào số b : 7	a = 8, b = 4
a nhỏ hơn b	a = 2, b = 2
—	Quan sát và nhận xét kết quả

Ví dụ 8: Viết chương trình nhập vào kí tự c. Kiểm tra xem nếu kí tự nhập vào là kí tự thường trong khoảng từ 'a' đến 'z' thì đổi sang chữ in hoa và in ra, nếu kí tự in hoa trong khoảng A đến Z thì đổi sang chữ thường và in ra, nếu kí tự là số từ 0 đến 9 thì in ra câu "Kí tự bạn vừa nhập là số ...(in ra kí tự c)", còn lại không phải 3 trường hợp trên in ra thông báo "Bạn đã nhập kí tự...(in ra kí tự c)".

a. Phác họa lời giải

Nhập kí tự c vào, kiểm tra xem nếu kí tự c thuộc khoảng 'a' và 'z' đổi kí tự c thành chữ in hoa bằng cách lấy kí tự c – 32 rồi gán lại cho chính nó (c = c – 32) (vì giữa

kí tự thường và in hoa trong bảng mã ASCII cách nhau 32, ví dụ: A trong bảng mã ASCII là 65, B là 66..., còn a là 97, b là 98...), sau khi đổi xong bạn in kí tự c ra. Ngược lại Nếu kí tự c thuộc khoảng 'A' và 'Z', đổi kí tự c thành chữ thường (theo cách ngược lại) và in ra. Ngược lại Nếu kí tự c thuộc khoảng '0' và '9' thì in ra thông báo "Kí tự bạn vừa nhập là số...". Ngược lại, in câu thông báo "Bạn đã nhập kí tự...".

b. Mô tả quy trình xử lý (giải thuật)

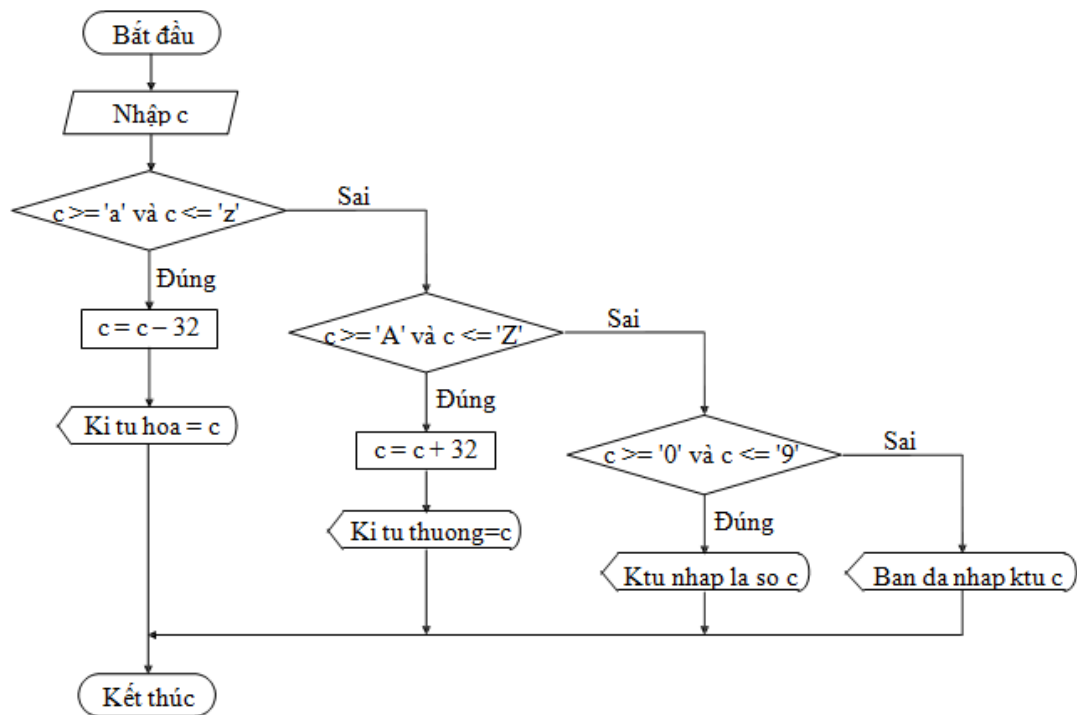
Ngôn ngữ tự nhiên	Ngôn ngữ C
- Khai báo biến c kiểu kí tự - Nhập vào kí tự c - Nếu $c \geq a$ và $c \leq z$ thì $c = c - 32$ in c ra màn hình - Ngược lại Nếu $c \geq A$ và $c \leq Z$ thì $c = c + 32$ in c ra màn hình - Ngược lại Nếu $c \geq 0$ và $c \leq 9$ thì in thông báo "Kí tự bạn vừa nhập là số c" Ngược lại thì in thông báo "Bạn đã nhập kí tự c"	<pre> - char c; - printf("Nhap vao 1 ki tu: "); scanf("%c", &c); - if (c >= 'a' && c <= 'z') { c = c - 32; printf("Ki tu hoa la: %c.\n", c); }; else if(c >= 'A' && c <= 'Z') { c = c + 32; printf("Ki tu thuong la: %c.\n", c); }; else if(c >= '0' && c <= '9') printf("Ki tu Ban vua nhap la so %c.\n", c); else printf("Ban da nhap ki tu %c.\n", c); </pre>

→ Cũng như if, không đặt dấu chấm phẩy sau câu lệnh else if.

Ví dụ: `else if(c >= 'A' && c <= 'Z');`

→ trình biên dịch không báo lỗi nhưng khối lệnh sau else if không được thực hiện.

c. Mô tả bằng lưu đồ



d. Viết chương trình

```

/* Chương trình nhập vào ki tu c. Đổi ra hoa, thường */
#include <stdio.h>
#include <conio.h>
int main()
{
    char c;
    printf("Nhập vào 1 ki tu: ");
    scanf("%c", &c);
    if (c >= 'a' && c <= 'z')          //hoac if(c >= 97 && c <= 122)
    {
        c = c - 32;                    //doi thanh chu in hoa
        printf("Ki tu hoa la: %c.\n", c);
    }
    else if(c >= 'A' && c <= 'Z')      //hoac if(c >= 65 && c <= 90)
    {
        c = c + 32;                    //doi thanh chu thuong
        printf("Ki tu thuong la: %c.\n", c);
    }
    else if(c >= '0' && c <= '9')      //hoac if(c >= 48 && c <= 57)
        printf("Ki tu Ban vua nhap la so %c.\n", c);
    else
        printf("Ban da nhap ki tu %c.\n", c);
    return 0;
}
  
```

→ **Kết quả in ra màn hình**

Nhập vào một ki tu: g

Cho chạy lại chương trình và thử lại với:

Ki tu hoa la: G.	c = '!', c = '2', c = 'a', c = 'Z'
–	Quan sát và nhận xét kết quả

5.2.4. Cấu trúc if lồng

Quyết định sẽ thực hiện 1 trong n khối lệnh cho trước.

- **Cú pháp lệnh**

Cú pháp là một trong 3 dạng trên, nhưng trong 1 hoặc nhiều khối lệnh bên trong phải chứa ít nhất một trong 3 dạng trên gọi là cấu trúc if lồng nhau. Thường cấu trúc if lồng nhau càng nhiều cấp độ phức tạp càng cao, chương trình chạy càng chậm và trong lúc lập trình dễ bị nhầm lẫn.

Lưu ý: Các lệnh **if...else** lồng nhau thì **else** sẽ luôn luôn kết hợp với **if** nào chưa có else gần nhất. Vì vậy khi gặp những lệnh if không có else, Bạn phải đặt chúng trong những **khối lệnh rõ ràng** để tránh bị hiểu sai câu lệnh.

Ví dụ 9: Bạn viết các dòng lệnh sau:

```
...
if (n > 0)
    if (a > b)
        x = a;
else
    x = b;
```

Mặc dù Bạn viết lệnh else thẳng hàng với if (n > 0), nhưng lệnh else ở đây được hiểu đi kèm với if (a > b), vì nó nằm gần với if (a > b) nhất và if (a > b) chưa có else. Để dễ nhìn và dễ hiểu hơn Bạn viết lại như sau:

```
...
if (n > 0)
    if (a > b)
        x = a;
    else
        x = b;
```

Còn nếu Bạn muốn lệnh else là của if (n > 0) thì Bạn phải đặt if (a > b) x = a trong một khối lệnh. Bạn viết lại như sau:

```
...
if (n > 0)
{
    if (a > b)
        x = a;
}
else
    x = b;
...
```

- **Lưu đồ**

Tương tự 3 dạng trên. Nhưng trong mỗi khối lệnh có thể có một (nhiều) cấu trúc if ở 3 dạng trên.

Ví dụ 10: Viết chương trình nhập vào điểm của một học sinh. In ra xếp loại học tập của học sinh đó. (Cách xếp loại. Nếu điểm ≥ 9 , Xuất sắc. Nếu điểm từ 8 đến cận 9, Giỏi. Nếu điểm từ 7 đến cận 8, Khá. Nếu điểm từ 6 đến cận 7, TBKhá. Nếu điểm từ 5 đến

cận 6, TBình. Còn lại là Yếu).

a. Phác họa lời giải

Điểm số nhập vào nếu hợp lệ ($0 \leq \text{điểm} \leq 10$), bạn tiếp tục công việc xếp loại, ngược lại thông báo "Nhập điểm không hợp lệ". Việc xếp loại bạn sử dụng cấu trúc else if.

b. Mô tả quy trình xử lý (giải thuật)

Ngôn ngữ tự nhiên	Ngôn ngữ C
- Khai báo biến điểm kiểu số thực - Nhập vào điểm số - Nếu $\text{điểm} \geq 0$ và $\text{điểm} \leq 10$ thì - Nếu $\text{điểm} \geq 9$ thì in ra xếp loại = Xuất sắc - Ngược lại Nếu $\text{điểm} \geq 8$ thì in ra xếp loại = Giỏi - Ngược lại Nếu $\text{điểm} \geq 7$ thì in ra xếp loại = Khá - Ngược lại Nếu $\text{điểm} \geq 6$ thì in ra xếp loại = TBKhá - Ngược lại Nếu $\text{điểm} \geq 5$ thì in ra xếp loại = TBình - Ngược lại thì in ra xếp loại = Yếu - Ngược lại thì in ra "Bạn nhập điểm không hợp lệ"	- float fdiem; - printf("Nhap vao diem so: "); scanf("%f", &fdiem); - if (fdiem ≥ 0 && fdiem ≤ 10) - if (fdiem ≥ 9) printf("Xep loai = Xuat sac.\n"); - else if (fdiem ≥ 8) printf("Xep loai = Gioi.\n"); - else if (fdiem ≥ 7) printf("Xep loai = Kha.\n"); - else if (fdiem ≥ 6) printf("Xep loai = TBKha.\n"); - else if (fdiem ≥ 5) printf("Xep loai = TBinh.\n"); - else printf("Xep loai = Yeu.\n"); - else printf("Ban nhap diem khong hop le.\n");

c. Mô tả bằng lưu đồ (Tương tự 3 dạng trên).

d. Viết chương trình

<pre> /* Chương trình nhập vào 2 số nguyên a, b. In ra thông báo a > b, a < b, a = b */ #include <stdio.h> #include <conio.h> int main() { float fdiem; printf("Nhap vao diem so: "); scanf("%f", &fdiem); if (fdiem >= 0 && fdiem <= 10) if (fdiem >= 9) printf("Xep loai = Xuat sac.\n"); else if (fdiem >= 8) printf("Xep loai = Gioi.\n"); else if (fdiem >= 7) printf("Xep loai = Kha.\n"); else if (fdiem >= 6) printf("Xep loai = TBKha.\n"); else if (fdiem >= 5) printf("Xep loai = TBinh.\n"); else </pre>

```
        printf("Xep loai = Yeu.\n");  
    else        //if (fdiem>=0 && fdien<=10)  
        printf("Nhap diem khong hop le.\n");  
    return 0;  
}
```

→ **Kết quả in ra màn hình**

Nhap vào diem so: 6.5 Xep loai = TBKha. —	Cho chạy lại chương trình và thử lại với: diem = 4, diem = 9, diem = 7, diem = 12 Quan sát và nhận xét kết quả
---	--

e. Bàn thêm về chương trình

Trong chương trình trên cấu trúc **else if** được lồng vào trong cấu trúc dạng 2, trong cấu trúc else if ta không cần đặt trong khối vì tất cả các if trong cấu trúc này đều có else, nên else printf("Nhap diem khong hop le.\n") đương nhiên là thuộc về if (fdiem >= 0 && fdiem <=10). Giả sử trong cấu trúc else if không có dòng else printf("Xep loai = Yeu.\n") thì khi đó dòng else printf("Nhap diem khong hop le.\n") sẽ thuộc về cấu trúc else if chứ không thuộc về if (fdiem >=0 && fdiem <= 10). Đối với trường hợp đó bạn cần phải đặt cấu trúc else if vào trong {}, thì khi đó dòng else printf("Nhap diem khong hop le.\n") sẽ thuộc về if (fdiem >= 0 && fdiem <= 10).

Ví dụ 11: Viết chương trình nhập vào 3 số nguyên a, b, c. Tìm và in ra số lớn nhất.

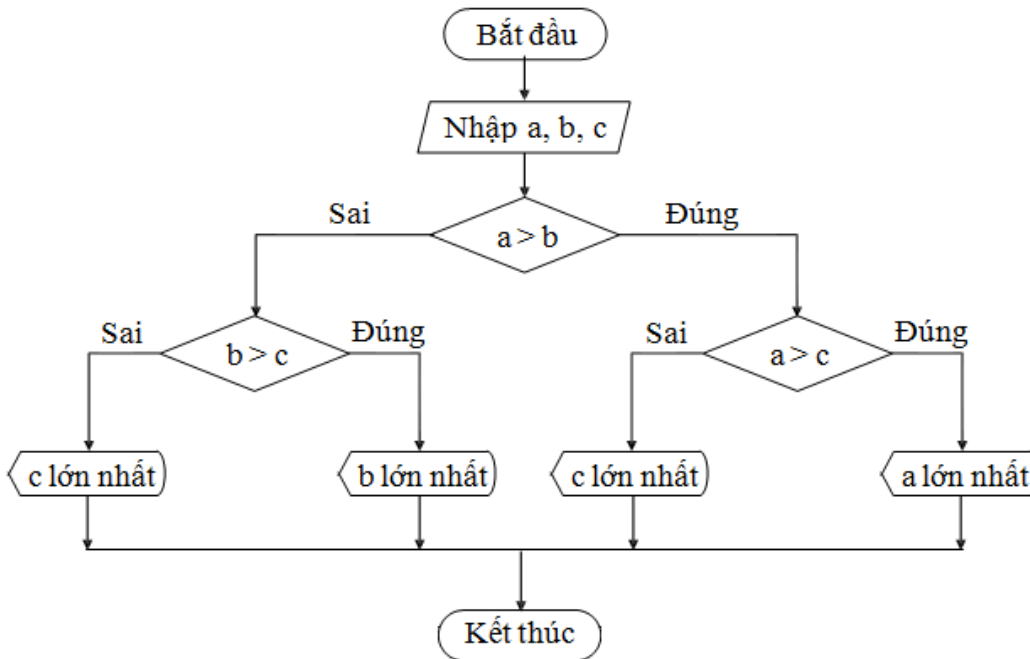
a. Phác họa lời giải

Trước tiên bạn so nếu a>b, mà a>c thì a lớn nhất, ngược lại c lớn nhất, còn nếu a<=b, mà c>b thì b lớn nhất, ngược lại c lớn nhất.

b. Mô tả quy trình xử lý (giải thuật)

Ngôn ngữ tự nhiên	Ngôn ngữ C
- Khai báo 3 biến a, b, c kiểu số nguyên - Nhập vào số a - Nhập vào số b - Nhập vào số c - Nếu a > b thì - Nếu a > c thì a lớn nhất - Ngược lại thì c lớn nhất - Ngược lại - Nếu b > c thì b lớn nhất - Ngược lại thì c lớn nhất	<pre> - int ia, ib, ic; - printf("Nhap vào số a: "); scanf("%d", &ia); - printf("Nhap vào số b: "); scanf("%d", &ib); - printf("Nhap vào số c: "); scanf("%d", &ic); - if (ia > ib) - if (ia > ic) printf("%d lon nhat.\n", ia); else printf("%d lon nhat.\n", ic); else - if (ib > ic) printf("%d lon nhat.\n", ib); else printf("%d lon nhat.\n", ic); </pre>

c. Mô tả bằng lưu đồ



d. Viết chương trình

/* Chương trình nhập vào 2 số nguyên a, b, c. Tìm, in ra số lớn nhất */

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int main()
```

```
{
```

```
    int ia, ib, ic;
```

```
    printf("Nhập vào số a: ");
```

```
    scanf("%d", &ia);
```

```
    printf("Nhập vào số b: ");
```

```
    scanf("%d", &ib);
```

```
    printf("Nhập vào số c: ");
```

```
    scanf("%d", &ic);
```

```
    if (ia > ib)
```

```
        if (ia > ic)
```

```
            printf("%d lớn nhất.\n", ia);
```

```
        else
```

```
            printf("%d lớn nhất.\n", ic);
```

```
    else
```

```
        if (ib > ic)
```

```
            printf("%d lớn nhất.\n", ib);
```

```
        else
```

```
            printf("%d lớn nhất.\n", ic);
```

```
    return 0;
```

```
}
```

→ **Kết quả in ra màn hình**

Nhập vào số a: 4

Nhập vào số b: 5

Nhập vào số c: 3

Cho chạy lại chương trình và thử lại với:

a = 5, b = 4, c = 2

a = 2, b = 1, c = 10

5 lon nhất.

a = 5, b = 5, c = 5

e. Bàn thêm về chương trình

Trong chương trình trên cấu trúc **dạng 2** được lồng vào trong cấu trúc **dạng 2**.

5.3. LỆNH switch

Lệnh switch cũng giống cấu trúc else if, nhưng nó mềm dẻo hơn và linh động hơn nhiều so với sử dụng if. Tuy nhiên, nó cũng có mặt hạn chế là kết quả của biểu thức phải là giá trị hằng nguyên (có giá trị cụ thể). Một bài toán sử dụng lệnh switch thì cũng có thể sử dụng if, nhưng ngược lại còn tùy thuộc vào giải thuật của bài toán.

5.3.1. Cấu trúc switch...case (switch thiếu)

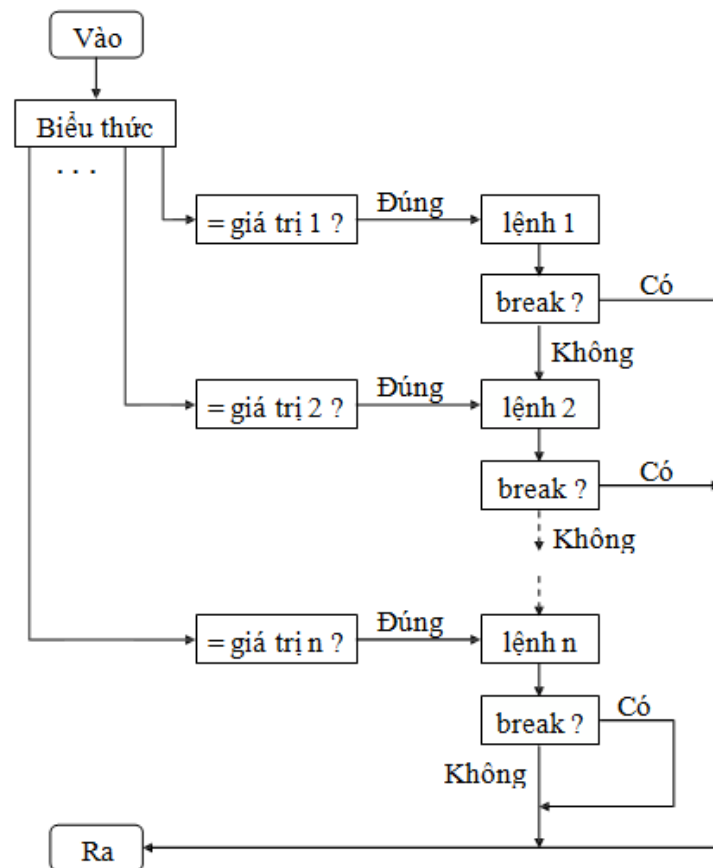
Chọn thực hiện 1 trong n lệnh cho trước.

- **Cú pháp lệnh**

```
switch (biểu thức)
{
    case giá trị 1 : lệnh 1;
                    break;
    case giá trị 2 : lệnh 2;
                    break;
    ...
    case giá trị n : lệnh n;
                    break;
}
```

→ từ khóa **switch, case, break** phải viết bằng chữ thường
→ **biểu thức** phải là có kết quả là **giá trị hằng nguyên (char, int, long,...)**
→ **Lệnh 1, 2...n** có thể gồm nhiều lệnh, nhưng không cần đặt trong cặp dấu { }
→ Khi giá trị của biểu thức bằng giá trị i thì lệnh i sẽ được thực hiện. Nếu sau lệnh i không có lệnh break thì sẽ tiếp tục thực hiện lệnh i + 1... Ngược lại thoát khỏi cấu trúc switch

- **Lưu đồ**



Ví dụ 12: Viết chương trình nhập vào số 1, 2, 3. In ra tương ứng 1, 2, 3 sao.

a. Viết chương trình

/* Chương trình nhập vào số 1, 2, 3. In ra số sao tương ứng */

#include <stdio.h>

#include <conio.h>

int main()

{

int i;

printf("Nhập vào số 1, 2 hoặc 3: ");

scanf("%d", &i);

switch(i)

{

case 3: printf("*"); case

2: printf("*"); case 1:

printf("*");

}

printf("Ấn phím bất kỳ để kết thúc!\n");

return 0;

}

→ **Kết quả in ra màn hình**

Nhập vào số 1, 2 hoặc 3: 2

**

—

Cho chạy lại chương trình và thử lại với:

i = 1, i = 3, i = 0, i = 4

Quan sát và nhận xét kết quả

b. Bàn thêm về chương trình

Trong chương trình trên khi nhập vào i = 2 lệnh printf("*") ở dòng case 2 được thi hành, nhưng do không có lệnh break sau đó nên lệnh printf("*") ở dòng case 1 tiếp

tục được thi hành. Kết quả in ra **.

→ **Không đặt dấu chấm phẩy sau câu lệnh switch.**

Ví dụ: switch(i);

→ **trình biên dịch không báo lỗi nhưng các lệnh trong switch không được thực hiện.**

Ví dụ 13: Viết chương trình nhập vào tháng và in ra quý. (tháng 1 -> quý 1, tháng 10 -> quý 4)

a. Phác họa lời giải

Nhập vào giá trị tháng, kiểm tra xem tháng có hợp lệ (trong khoảng 1 đến 12). Nếu hợp lệ in ra quý tương ứng (1->3: quý 1, 4->6: quý 2, 7->9: quý 3, 10->12: quý 4).

b. Viết chương trình

```
/* Chương trình nhập vào tháng. In ra quý tương ứng */
#include <stdio.h>
#include <conio.h>
int main()
{
    int ithang;
    printf("Nhập vào tháng: ");
    scanf("%d", &ithang);
    if (ithang > 0 && ithang <= 12)
        switch(ithang)
        {
            case 1:
            case 2:
            case 3: printf("Quý 1.\n");
                    break;
            case 4:
            case 5:
            case 6: printf("Quý 2.\n");
                    break;
            case 7:
            case 8:
            case 9: printf("Quý 3.\n");
                    break;
            case 10:
            case 11:
            case 12: printf("Quý 4.\n");
                     break;
        }
    else
        printf("Thang không hợp lệ.\n");
    return 0;
}
```

→ **Kết quả in ra màn hình**

Nhập vào tháng: 4 Quý 2. —	Cho chạy lại chương trình và thử lại với: tháng = 7, tháng = 1, tháng = 13, tháng = -4 Quan sát và nhận xét kết quả
----------------------------------	---

c. Bàn thêm về chương trình

Trong chương trình trên cấu trúc **switch...case** được lồng vào trong cấu trúc **if** dạng

2.

5.3.2. Cấu trúc switch...case...default (switch đủ)

Chọn thực hiện 1 trong $n + 1$ lệnh cho trước.

Cú pháp lệnh

```
switch (biểu thức)
{
    case giá trị 1 : lệnh 1;
                    break;
    case giá trị 2 : lệnh 2;
                    break;
    ...
    case giá trị n : lệnh n;
                    break;
    default       : lệnh;
                    [break;]
}
```

- từ khóa **switch, case, break, default** phải viết bằng chữ thường
- **biểu thức** phải là có kết quả là giá trị nguyên (char, int, long,...)
- **Lệnh 1, 2...n** có thể gồm nhiều lệnh, nhưng không cần đặt trong cặp dấu { }

→ Khi giá trị của biểu thức bằng giá trị i thì lệnh i sẽ được thực hiện. Nếu sau lệnh i không có lệnh break thì sẽ tiếp tục thực hiện lệnh $i + 1$... Ngược lại thoát khỏi cấu trúc switch. Nếu giá trị biểu thức không trùng với bất kỳ giá trị i nào thì lệnh tương ứng với từ khóa default sẽ được thực hiện.

Ví dụ 14: Viết lại chương trình ở ví dụ 12

a. Viết chương trình

```
/* Chương trình nhập vào số 1, 2, 3. In ra số sao tương ứng */
#include <stdio.h>
#include <conio.h>
int main()
{
    int i;
    printf("Nhập vào số 1, 2 hoặc 3: ");
    scanf("%d", &i);
    switch(i)
    {
        case 3: printf("*");
        case 2: printf("*");
        case 1: printf("*");
                break;
        default: printf("Bạn nhập phải nhập vào số 1, 2 hoặc 3.\n");
    }
    return 0;
}
```

→ **Kết quả in ra màn hình**

Nhap vao so 1, 2 hoặc 3: 3 *** —	Cho chạy lại chương trình và thử lại với: i = 1, i = 3, i = 0, i = 4 Quan sát kết quả
--	---

b. Bàn thêm về chương trình

Trong chương trình trên. Nếu bạn nhập vào 1, 2, 3 sẽ in ra số sao tương ứng. Ngoài các số này chương trình sẽ in ra câu thông báo "Bạn phải nhập vào số 1, 2 hoặc 3".

Ví dụ 15: Viết lại chương trình ở ví dụ 13

a. Viết chương trình

```
/* Chương trình nhập vào thang. In ra quy tương ứng */
#include <stdio.h>
#include <conio.h>
int main()
{
    int ithang;
    printf("Nhap vao thang: ");
    scanf("%d", &ithang);
    switch(ithang)
    {
        case 1: case 2: case 3 :    printf("Quy 1.\n");
                                   break;
        case 4: case 5: case 6:    printf("Quy 2.\n");
                                   break;
        case 7: case 8: case 9:    printf("Quy 3.\n");
                                   break;
        case 10: case 11: case 12: printf("Quy 4.\n");
                                   break;
        default                    : printf("Ban phai nhap vao so trong khoang 1..12\n");
    }
    Return 0;
}
```

→ ***Kết quả in ra màn hình***

Nhap vao thang: 4 Quy 2. —	Cho chạy lại chương trình và thử lại với: thang = 7, thang = 1, thang = 13, thang = -4 Quan sát kết quả
----------------------------------	---

c. Bàn thêm về chương trình

Trong chương trình trên. Nếu bạn nhập vào 1 đến 12 sẽ in quý tương ứng. Ngoài các số này chương trình sẽ in ra câu thông báo "Bạn phải nhập vào số trong khoảng 1..12".

5.3.3. Cấu trúc switch lồng

Quyết định sẽ thực hiện 1 trong n khối lệnh cho trước.

• Cú pháp lệnh

Cú pháp là một trong 2 dạng trên, nhưng trong 1 hoặc nhiều lệnh bên trong phải chứa ít nhất một trong 2 dạng trên gọi là cấu trúc switch lồng nhau. Thường cấu trúc switch lồng nhau càng nhiều cấp độ phức tạp càng cao, chương trình chạy càng chậm

và trong lúc lập trình dễ bị nhầm lẫn.

Ví dụ 16: Viết chương trình menu 2 cấp

a. Viết chương trình

```
/* Chuong trinh menu 2 cap */
#include <stdio.h>
#include <conio.h>
int main()
{
    int imenu, isubmenu;
    printf("-----\n");
    printf("    MAIN MENU    \n");
    printf("-----\n");
    printf("1. File\n");
    printf("2. Edit\n");
    printf("3. Search\n");
    printf("Chon muc tuong ung: ");
    scanf("%d", &imenu);
    switch(imenu)
    {
        case 1: printf("-----\n");
                printf("    MENU FILE    \n");
                printf("-----\n");
                printf("1. New\n");
                printf("2. Open\n");
                printf("Chon muc tuong ung: ");
                scanf("%d", &isubmenu);
                switch(isubmenu)
                {
                    case 1: printf("Ban da chon chuc nang New File\n");
                            break;
                    case 2: printf("Ban da chon chuc nang Open File\n");
                            break;
                }
                break; //break cua case 1 – switch(imenu)
        case 2: printf("Ban da chon chuc nang Edit\n");
                break;
        case 3: printf("Ban da chon chuc nang Search\n");
                break;
    }
    return 0;
}
```

→ **Kết quả in ra màn hình**

----- MAIN MENU ----- 1. File 2. Edit 3. Search Chon muc tuong ung: 1	Quan sát kết quả. * Thêm các thành phần sau vào chương trình: - Thêm mục Save vào menu File. - Tạo menu Edit gồm 4 chức năng: Copy, Cut, Paste, Clear. - Tạo menu Search gồm 2 chức năng: Find, Replace.
---	--

----- MENU FILE ----- 1. New 2. Open Chon muc tuong ung: 2 Ban da chon chuc nang Open File —	Chạy lại chương trình và thử với nhiều mục chọn khác nhau. Quan sát kết quả.	C Â U H ỒI VÀ B ÀI
--	--	--

TẬP THỰC HÀNH

- Viết chương trình nhập 3 số từ bàn phím, tìm số lớn nhất trong 3 số đó, in kết quả lên màn hình.
- Viết chương trình tính chu vi, diện tích của tam giác với yêu cầu sau khi nhập 3 số a, b, c phải kiểm tra lại xem a, b, c có tạo thành một tam giác không? Nếu có thì tính chu vi và diện tích. Nếu không thì in ra câu " Không tạo thành tam giác".
- Viết chương trình giải phương trình bậc nhất $ax+b=0$ với a, b nhập từ bàn phím.
- Viết chương trình giải phương trình bậc hai $ax^2+bx + c = 0$ với a, b, c nhập từ bàn phím.
- Viết chương trình nhập từ bàn phím 2 số a, b và một ký tự ch.
Nếu: ch là "+" thì thực hiện phép tính $a + b$ và in kết quả lên màn hình. ch là "-" thì thực hiện phép tính $a - b$ và in kết quả lên màn hình. ch là "*" thì thực hiện phép tính $a * b$ và in kết quả lên màn hình. ch là "/" thì thực hiện phép tính a / b và in kết quả lên màn hình.
- Viết chương trình nhập vào 2 số là tháng và năm của một năm. Xét xem tháng đó có bao nhiêu ngày? Biết rằng:
Nếu tháng là 4, 6, 9, 11 thì số ngày là 30.
Nếu tháng là 1, 3, 5, 7, 8, 10, 12 thì số ngày là 31.
Nếu tháng là 2 và năm nhuận thì số ngày 29, ngược lại thì số ngày là 28.
- Có hai phương thức gửi tiền tiết kiệm: gửi không kỳ hạn lãi suất 2.4%/tháng, mỗi tháng tính lãi một lần, gửi có kỳ hạn 3 tháng lãi suất 4%/tháng, 3 tháng tính lãi một lần. Viết chương trình tính tổng cộng số tiền cả vốn lẫn lãi sau một thời gian gửi nhập từ bàn phím.
- Một số nguyên dương chia hết cho 3 nếu tổng các chữ số của nó chia hết cho 3. Viết chương trình nhập vào một số có 3 chữ số, kiểm tra số đó có chia hết cho 3 dùng tính chất trên.(if)
- Trò chơi "Oẳn tù tì": trò chơi có 2 người chơi mỗi người sẽ dùng tay để biểu thị một trong 3 công cụ sau: Kéo, Bao và Búa.
Nguyên tắc: Kéo thắng bao.
Bao thắng búa.
Búa thắng kéo.
Viết chương trình mô phỏng trò chơi này cho hai người chơi và người chơi với máy. (switch)

10. Viết chương trình tính tiền điện gồm các khoản sau: Định mức sử dụng điện cho mỗi hộ là 50 Kw. Phần định mức tính giá 1.678 đồng /Kwh. Nếu phần vượt định mức ≤ 100 Kw tính giá cho phần này là 2.014 đồng/Kwh. Nếu phần vượt định mức lớn 200 Kw và nhỏ hơn 300Kw tính giá phạt cho phần này là 2.536 đồng/Kwh. Nếu phần vượt định mức lớn hơn hay bằng 300 Kw tính giá phạt cho phần này là 2834 đồng/Kwh .
- Với: chỉ số điện kế cũ và chỉ số điện kế mới nhập vào từ bàn phím. In ra màn hình số tiền trả trong định mức, vượt định mức và tổng của chúng. (if)
11. Viết chương trình nhận vào giờ, phút, giây dạng (hh:mm:ss), từ bàn phím. Cộng thêm một số giây vào và in ra kết quả dưới dạng (hh:mm:ss).
12. Viết chương trình nhập vào ngày tháng năm của một ngày, kiểm tra nó có hợp lệ không.
13. Kiểm tra một ký tự nhập vào thuộc tập hợp nào trong các tập ký tự sau:
- Các ký tự chữ hoa: 'A' ... 'Z'
 - Các ký tự chữ thường: 'a' ... 'z'
 - Các ký tự chữ số : '0' ... '9'
 - Các ký tự khác.
14. Hệ thập lục phân dùng 16 ký số bao gồm các ký tự 0 .. 9 và A, B, C, D, E ,F.
- Các ký số A, B, C, D, E, F có giá trị tương ứng trong hệ thập phân
- Hãy viết chương trình cho nhập vào ký tự biểu diễn một ký số của hệ thập lục phân và cho biết giá trị thập phân tương ứng. Trường hợp ký tự nhập vào không thuộc các ký số trên, đưa ra thông báo lỗi : "Hệ thập lục phân không dùng ký số này"
15. Viết chương trình nhập vào ngày tháng năm của ngày hôm nay, in ra ngày tháng năm của ngày mai.

Chương 6. CẤU TRÚC VÒNG LẶP

Sau khi hoàn tất bài này học viên sẽ hiểu và vận dụng các kiến thức kỹ năng cơ bản sau:

- Ý nghĩa, cách hoạt động của vòng lặp.
- Cú pháp, ý nghĩa, cách sử dụng lệnh for, while, do...while.
- Ý nghĩa và cách sử dụng lệnh break, continue.
- Một số bài toán sử dụng lệnh for, while, do...while thông qua các ví dụ.
- So sánh, đánh giá một số bài toán sử dụng lệnh for, while hoặc do...while.
- Cấu trúc vòng lặp lồng nhau.

6.1. LỆNH for

Lệnh for cho phép lặp lại công việc cho đến khi điều kiện sai.

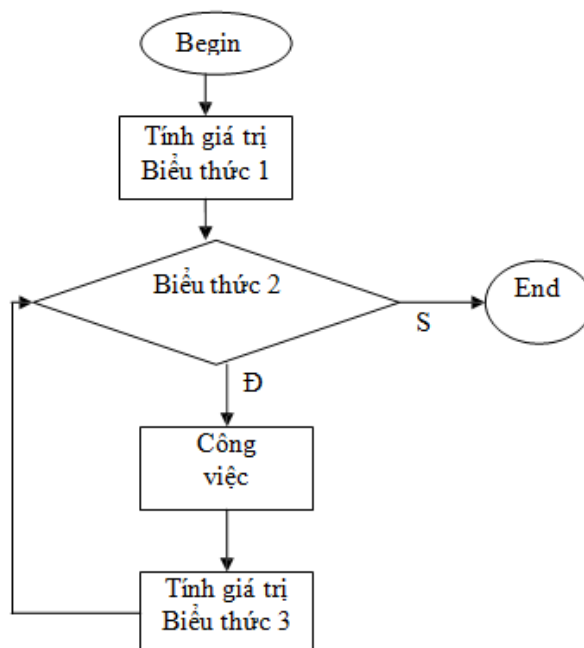
Cú pháp:

for (Biểu thức 1; biểu thức 2; biểu thức 3) <Công việc>

→ từ khóa **for** phải viết bằng chữ thường

Nếu **khối lệnh** bao gồm từ 2 lệnh trở lên thì phải đặt trong dấu { }

Lưu đồ:



Giải thích:

<Công việc>: được thể hiện là 1 câu lệnh hay 1 khối lệnh. Thứ tự thực hiện của câu lệnh for như sau:

B1: Tính giá trị của biểu thức 1.

B2: Tính giá trị của biểu thức 2.

- Nếu giá trị của biểu thức 2 là sai ($=0$): *thoát khỏi câu lệnh for*.
- Nếu giá trị của biểu thức 2 là đúng ($\neq 0$): <Công việc> được thực hiện.

B3: Tính giá trị của biểu thức 3 và quay lại B2.

Một số lưu ý khi sử dụng câu lệnh for:

- Khi biểu thức 2 vắng mặt thì nó được coi là luôn luôn đúng
- Biểu thức 1: thông thường là một phép gán để khởi tạo giá trị ban đầu cho biến điều kiện.
- Biểu thức 2: là một biểu thức kiểm tra điều kiện đúng sai để dừng vòng lặp.
- Biểu thức 3: thông thường là một phép gán để thay đổi giá trị của biến điều kiện.
- Trong mỗi biểu thức có thể có nhiều biểu thức con. Các biểu thức con được phân biệt bởi dấu phẩy.

Ví dụ 1: Viết chương trình in ra câu "Vi dụ su dung vong lap for" 3 lần.

```
1  /* Chuong trinh in ra cau "Vi du su dung vong lap for" 3 lan */
2
3  #include <stdio.h>
4  #include <conio.h>
5
6  #define MSG "Vi du su dung vong lap for.\n"
7
8  int main()
9  {
10     int i;
11     for(i = 1; i<=3; i++)          //hoac for(i = 1; i<=3; i+=1)
12         printf("%s", MSG);
13     return 0;
14 }
```

→ **Kết quả in ra màn hình**

Vi dụ su dung vong lap for.
Vi dụ su dung vong lap for.
Vi dụ su dung vong lap for.

Bạn thay 2 dòng 11 và 12 bằng câu lệnh
for(i=1; i<=3; i++, printf("%s", MSG));
Chạy lại chương trình, quan sát và nhận xét kết quả.

→ **Có dấu chấm phẩy sau lệnh for(i=1; i<=3; i++);** → **các lệnh thuộc vòng lặp for sẽ không được thực hiện.**

Ví dụ 2: Viết chương trình nhập vào 3 số nguyên. Tính và in ra tổng của chúng.

```
1  /* Chuong trinh nhap vao 3 so va tinh tong */
2
3  #include <stdio.h>
4  #include <conio.h>
5
6  int main()
7  {
8     int i, in, is;
9     is = 0;
```


10	for(i = 1; i<=3; i++)
11	{
12	printf("Nhap vao so thu %d :", i);
13	scanf("%d", &in);
14	is = is + in;
15	}
16	printf("Tong: %d", is);
17	return 0;
18	}

→ **Kết quả in ra màn hình**

Nhap vao so thu 1: 5 Nhap vao so thu 2: 4 Nhap vao so thu 3: 2 Tong: 11. —	Bạn thay các dòng từ 9 đến 15 bằng câu lệnh: for(is=0, i=1; i<=3; printf("Nhap vao so thu %d:", i), scanf("%d", &in), i++, is=is+in); Chạy lại chương trình, quan sát và nhận xét kết quả.
--	--

Ví dụ 3: Viết chương trình nhập vào số nguyên n. Tính tổng các giá trị lẻ từ 0 đến n.

1	/* Chuong trinh tinh tong các số lẻ từ 0 đến n*/
2	
3	#include <stdio.h>
4	#include <conio.h>
5	
6	int main()
7	{
8	int i, in, is = 0;
9	printf("Nhap vao so n: ");
10	scanf("%d", &in);
11	is = 0;
12	for(i = 0; i<=in; i++)
13	{
14	if (i % 2 != 0) //neu i la so le
15	is = is + i; //hoac is += i;
16	}
17	printf("Tong: %d", is);
18	return 0;
19	}

→ **Kết quả in ra màn hình**

Nhap vao so n : 5 Tong: 9. —	Bạn thay các dòng từ 11 đến 16 bằng câu lệnh: for(is=0, i=1; i<=n; is=is+i, i+=2); Chạy lại chương trình, quan sát và nhận xét kết quả.
------------------------------------	---

→ **Bạn có thể viết gộp các lệnh trong thân for vào trong lệnh for. Tuy nhiên, khi lập trình bạn nên viết lệnh for có đủ 3 biểu thức đơn và các lệnh thực hiện trong thân for mỗi lệnh một dòng để sau này có thể đọc lại dễ hiểu, dễ sửa chữa.**

Ví dụ 4: Một vài ví dụ thay đổi biến điều khiển vòng lặp.

- Thay đổi biến điều khiển từ 1 đến 100, mỗi lần tăng 1:
for(i = 1; i <= 100; i++)
- Thay đổi biến điều khiển từ 100 đến 1, mỗi lần giảm 1:
for(i = 100; i >= 1; i--)
- Thay đổi biến điều khiển từ 7 đến 77, mỗi lần tăng 7:
for(i = 7; i <= 77; i += 7)
- Thay đổi biến điều khiển từ 20 đến 2, mỗi lần giảm 2:
for(i = 20; i >= 2; i -= 2)

Ví dụ 5: Đọc vào một loạt kí tự trên bàn phím. Kết thúc khi gặp dấu chấm '.'.

1	/* Doc vao 1 loat ktu tren ban phim. Ket thuc khi gap dau cham */
2	
3	#include <stdio.h>
4	
5	#define DAU_CHAM '.'
6	
7	int main()
8	{
9	char c;
10	for(; (c = getchar()) != DAU_CHAM;)
11	putchar(c);
12	return 0;
13	}

→ **Kết quả in ra màn hình**

a	Bạn thay các dòng từ 10 đến 11 bằng câu lệnh: for(; (c = getchar()) != DAU_CHAM; putchar(c)); Chạy lại chương trình, quan sát và nhận xét kết quả.
a	
4	
4	
.	

→ **Vòng lặp for vắng mặt biểu thức 1 và 3.**

Ví dụ 6: Đọc vào một loạt kí tự trên bàn phím, đếm số kí tự nhập vào. Kết thúc khi gặp dấu chấm '.'.

1	/* Doc vao 1 loat ktu tren ban phim. Ket thuc khi gap dau cham */
2	
3	#include <stdio.h>
4	#include <conio.h>
5	
6	#define DAU_CHAM '.'
7	
8	int main()
9	{
10	char c;
11	int idem;

12	for(idem = 0; (c = getchar()) != DAU_CHAM;)
13	idem++;
14	printf("So ki tu: %d.\n", idem);
15	return 0;
16	}

→ **Kết quả in ra màn hình**

afser. So ki tu: 5. _	Bạn thay các dòng từ 12 đến 13 bằng câu lệnh: for(idem = 0; (c = getchar()) != DAU_CHAM; idem++); Chạy lại chương trình, quan sát và nhận x
-----------------------------	--

→ **Vòng lặp for vắng mặt biểu thức 3.**

Ví dụ 7: Đọc vào một loạt kí tự trên bàn phím, đếm số kí tự nhập vào. Kết thúc khi gặp dấu chấm '.'.

1	/* Doc vao 1 loạt ktu tren ban phim, dem so ktu nhap vao. Ket thuc khi gap dau
2	cham */
3	#include <stdio.h>
4	#include <conio.h>
5	
6	#define DAU_CHAM '.'
7	
8	int main()
9	{
10	char c;
11	int idem = 0;
12	for(;;)
13	{
14	c = getchar();
15	if (c == DAU_CHAM) //nhap vao dau cham
16	break; //thoat vong lap
17	idem++;
18	}
19	printf("So ki tu: %d.\n", idem);
20	return 0;
21	}

→ **Kết quả in ra màn hình**

afser. So ki tu: 5.	Chạy lại chương trình, quan sát và nhận xét kết quả.
------------------------	--

→ **Vòng lặp for vắng mặt cả ba biểu thức.**

Ví dụ 8: Nhập vào 1 dãy số nguyên từ bàn phím đến khi gặp số 0 thì dừng. In ra tổng các số nguyên dương.

1	/* Nhap vao 1 day so nguyen tu ban phim den khi gap so 0 thi dung. In ra
2	tong cac so nguyen duong */
3	#include <stdio.h>

```

4  #include <conio.h>
5
6  int main()
7  {
8      int in, itong = 0;
9      for(;;)
10     {
11         printf("Nhap vao 1 so nguyen: ");
12         scanf("%d", &in);
13         if (in < 0)
14             continue;    //in < 0 quay nguoc len dau vong lap
15         if (in == 0)
16             break;        //in = 0 thoat vong lap
17         itong += in;
18     }
19     printf("Tong: %d.\n", itong);
20     return 0;
21 }

```

→ **Kết quả in ra màn hình**

Nhap vao 1 so nguyen: -8 Nhap vao 1 so nguyen: 9 Nhap vao 1 so nguyen: -7 Nhap vao 1 so nguyen: 3 Nhap vao 1 so nguyen: 0 Tong: 12_	Chạy lại chương trình với số liệu khác Quan sát và nhận xét kết quả.
--	---

6.2. LỆNH break

Thông thường lệnh break dùng để thoát khỏi vòng lặp không xác định điều kiện dừng hoặc bạn muốn dừng vòng lặp theo điều kiện do bạn chỉ định. Việc dùng lệnh break để thoát khỏi vòng lặp thường sử dụng phối hợp với lệnh if. Lệnh break dùng trong for, while, do...while, switch. Lệnh break thoát khỏi vòng lặp chứa nó.

Ví dụ 9: Như ví dụ 7, 8 Sử dụng lệnh break trong switch để nhảy bỏ các câu lệnh kế tiếp còn lại.

6.3. LỆNH continue

Được dùng trong vòng lặp for, while, do...while. Khi lệnh continue thi hành quyền điều khiển sẽ trao qua cho biểu thức điều kiện của vòng lặp gần nhất. Nghĩa là lộn ngược lên đầu vòng lặp, tất cả những lệnh đi sau trong vòng lặp chứa continue sẽ bị bỏ qua không thi hành.

Ví dụ 10: Như ví dụ 8.

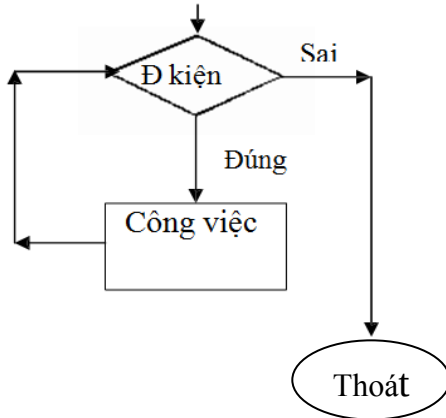
6.4. LỆNH while

Vòng lặp thực hiện lặp lại trong khi biểu thức còn đúng.

Cú pháp lệnh

while (Biểu thức điều kiện) <Công việc>

Lưu đồ



→ Trước tiên **biểu thức** được kiểm tra

- Nếu **sai** thì kết thúc vòng lặp while (khối lệnh không được thi hành 1 lần nào)
- Nếu **đúng** thực hiện khối lệnh; lặp lại kiểm tra biểu thức.

+ Biểu thức: có thể là một biểu thức hoặc nhiều biểu thức con. Nếu là nhiều biểu thức con thì cách nhau bởi dấu phẩy (,) và tính đúng sai của biểu thức được quyết định bởi biểu thức con cuối cùng.

+ Trong thân while (khối lệnh) có thể chứa một hoặc nhiều cấu trúc điều khiển khác.

+ Trong thân while có thể sử dụng lệnh continue để chuyển đến đầu vòng lặp (bỏ qua các câu lệnh còn lại trong thân).

+ Muốn thoát khỏi vòng lặp while tùy ý có thể dùng các lệnh **break**, **goto**, **return** như lệnh **for**.

Ví dụ 11: Viết chương trình in ra câu "Ví dụ sử dụng vòng lặp while" 3 lần.

1	/* Chương trình in ra câu "Ví dụ sử dụng vòng lặp while" 3 lần */
2	
3	#include <stdio.h>
4	#include <conio.h>
5	
6	#define MSG "Ví dụ sử dụng vòng lặp while.\n"
7	
8	int main()
9	{
10	int i = 0;
11	while (i++ < 3)
12	printf("%s", MSG);
13	return 0;
14	}

→ **Kết quả in ra màn hình**

Ví dụ sử dụng vòng lặp while. Ví dụ sử dụng vòng lặp while. Ví dụ sử dụng vòng lặp while.	Bạn thay 2 dòng 11 và 12 bằng câu lệnh while(printf("%s", MSG), ++i < 3); Chạy lại chương trình và quan sát kết quả
---	--

Ví dụ 12: Viết chương trình tính tổng các số nguyên từ 1 đến n, với n được nhập vào từ bàn phím.

1	/* Chương trình tính tổng các số nguyên từ 1 đến n */
2	
3	#include <stdio.h>
4	#include <conio.h>

```

5  int main()
6  {
7      int i = 0, in, is = 0;
8      printf("Nhap vao so n: ");
9      scanf("%d", &in);
10     while (i++ < in)
11         is = is + i;      //hoac is += i;
12     printf("Tong: %d", is);
13     return 0;
14 }
15

```

→ **Kết quả in ra màn hình**

Nhap vao so n : 5 Tong: 15.	Bạn thay các dòng từ 11 đến 12 bằng câu lệnh: while(is = is+i, i++ < in); Chạy lại chương trình, quan sát và nhận xét kết quả.
--------------------------------	--

6.5. LỆNH do...while

Vòng lặp do ... while giống như vòng lặp for, while, dùng để lặp lại một công việc nào đó khi điều kiện còn đúng.

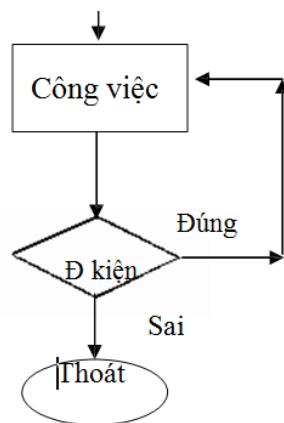
Cú pháp lệnh

```

Do
{
    <Công việc>
}
while (<Biểu thức điều kiện>)

```

Lưu đồ



→ Thực hiện khối lệnh

Kiểm tra biểu thức Nếu **đúng** thì lặp lại thực hiện khối lệnh

Nếu **sai** thì kết thúc vòng lặp (khối lệnh được thi hành 1 lần)

+ Biểu thức: có thể là một biểu thức hoặc nhiều biểu thức con. Nếu là nhiều biểu thức con thì cách nhau bởi dấu phẩy (,) và tính đúng sai của biểu thức được quyết định bởi biểu thức con cuối cùng.

+ Trong thân do...while (khối lệnh) có thể chứa một hoặc nhiều cấu trúc điều khiển khác.

+ Trong thân do...while có thể sử dụng lệnh continue để chuyển đến đầu vòng

lặp (bỏ qua các câu lệnh còn lại trong thân).

+ Muốn thoát khỏi vòng lặp do...while tùy ý có thể dùng các lệnh **break**, **goto**, **return**.

Ví dụ 16: Viết chương trình kiểm tra password.

```
1  /* Chương trình kiểm tra mật khẩu */
2
3  #include <stdio.h>
4
5  # define PASSWORD  12345
6
7  int main()
8  {
9      int in;
10     do
11     {
12         printf("Nhập vào password: ");
13         scanf("%d", &in);
14     } while (in != PASSWORD)
15     Return 0;
16 }
```

→**Kết quả in ra màn hình**

Nhập vào password: 1123 Nhập vào password: 12346 Nhập vào password: 12345 —	Bạn thay các dòng từ 10 đến 14 bằng câu lệnh: do{}while(printf("Nhập vào password: "), scanf("%d", &in), in != PASSWORD); Chạy lại chương trình và quan sát kết quả.
--	---

Ví dụ 17: Viết chương trình nhập vào năm hiện tại, năm sinh. In ra tuổi.

```
1  /* Chương trình in tuổi */
2  #include <stdio.h>
3
4  # define CHUC    "Chúc bạn vui vẻ (: >\n"
5  int main()
6  {
7      unsigned char choi;
8      int inamhtai, inamsinh;
9      do
10     {
11         printf("Nhập vào năm hiện tại: ");
12         scanf("%d", &inamhtai);
13         printf("Nhập vào năm sinh: ");
14         scanf("%d", &inamsinh);
15         printf("Bạn %d tuổi, %s", inamhtai - inamsinh, CHUC);
16         printf("Bạn có muốn tiếp tục? (Y/N)\n");
17         choi = getche();
18     } while (choi == 'y' || choi == 'Y');
19     return 0 ;
20 }
21
```

→**Kết quả in ra màn hình**

Nhập vào năm hiện tại: 2002 Nhập vào năm sinh: 1980 Bạn 22 tuổi, chúc bạn vui vẻ (:> Bạn có muốn tiếp tục? (Y/N) (nếu gõ y hoặc Y tiếp tục thực hiện chương trình, ngược lại gõ các phím khác chương trình sẽ thoát)	Bạn chạy lại chương trình với số liệu khác. Quan sát, đánh giá và nhận xét kết quả.
---	---

6.6. VÒNG LẶP LỒNG NHAU

Ví dụ 18: Vẽ hình chữ nhật đặc bằng các dấu '*'

1	/* Vẽ hình chữ nhật đặc */
2	#include <stdio.h>
3	#include <conio.h>
4	int main()
5	{
6	int i, j, idai, irong;
7	printf("Nhập vào chiều dài: ");
8	scanf("%d", &idai);
9	printf("Nhập vào chiều rộng: ");
10	scanf("%d", &irong);
11	for (i = 1; i <= irong; i++)
12	{
13	for (j = 1; j <= idai; j++) //in một hàng với chiều dài dấu *
14	printf("*");
15	printf("\n"); //xuống dòng khi in xong 1 hàng
16	}
17	return 0;
18	}

→ **Kết quả in ra màn hình**

Nhập vào chiều dài: 10 Nhập vào chiều rộng: 5 *	Bạn chạy lại chương trình với số liệu khác. Quan sát, đánh giá và nhận xét kết quả.
---	---

→

Các lệnh lặp for, while, do...while có thể lồng vào chính nó, hoặc lồng vào lẫn nhau. Nếu không cần thiết không nên lồng vào nhiều cấp để gây nhầm lẫn khi lập trình cũng như kiểm soát chương trình.

So sánh sự khác nhau của các vòng lặp

- Vòng lặp for thường sử dụng khi biết được số lần lặp xác định.
- Vòng lặp thường while, do...while sử dụng khi không biết rõ số lần lặp.
- Khi gọi vòng lặp while, do...while, nếu biểu thức sai vòng lặp while sẽ không được thực hiện lần nào nhưng vòng lặp do...while thực hiện được 1 lần.

→ Số lần thực hiện ít nhất của while là 0 và của do...while là 1

CÂU HỎI VÀ BÀI TẬP THỰC HÀNH

- Viết chương trình in ra bảng mã ASCII
- Viết chương trình tính tổng bậc 3 của N số nguyên đầu tiên.
- Viết chương trình nhập vào một số nguyên rồi in ra tất cả các ước số của số đó.
- Viết chương trình vẽ một tam giác cân bằng các dấu *
- Viết chương trình tính tổng nghịch đảo của N số nguyên đầu tiên theo công thức
$$S = 1 + 1/2 + 1/3 + \dots + 1/N$$
- Viết chương trình tính tổng bình phương các số lẻ từ 1 đến N.
- Viết chương trình nhập vào N số nguyên, tìm số lớn nhất, số nhỏ nhất.
- Viết chương trình nhập vào N rồi tính giai thừa của N.
- Viết chương trình tìm USCLN, BSCNN của 2 số.
- Viết chương trình vẽ một tam giác cân rỗng bằng các dấu *.
- Viết chương trình vẽ hình chữ nhật rỗng bằng các dấu *.
- Viết chương trình nhập vào một số và kiểm tra xem số đó có phải là số nguyên tố hay không?
- Viết chương trình tính số hạng thứ n của dãy Fibonacci.
Dãy Fibonacci là dãy số gồm các số hạng $p(n)$ với:
 $p(n) = p(n-1) + p(n-2)$ với $n > 2$ và $p(1) = p(2) = 1$
Dãy Fibonacci sẽ là: 1 1 2 3 5 8 13 21 34 55 89 144...
- Viết chương trình tính giá trị của đa thức
$$P_n = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x^1 + a_0$$

Hướng dẫn đa thức có thể viết lại
$$P_n = (\dots(a_n x + a_{n-1})x + a_{n-2})x + \dots + a_0$$

Như vậy trước tiên tính $a_n x + a_{n-1}$, lấy kết quả nhân với x, sau đó lấy kết quả nhân với x cộng thêm a_{n-2} , lấy kết quả nhân với x ... n gọi là bậc của đa thức.
- Viết chương trình tính x^n với x, n được nhập vào từ bàn phím.
- Viết chương trình nhập vào 1 số từ 0 đến 9. In ra chữ số tương ứng.
Ví dụ: nhập vào số 5, in ra "Năm".
- Viết chương trình phân tích một số nguyên N thành tích của các thừa số nguyên tố.
- Viết chương trình lặp lại nhiều lần công việc nhập một ký tự và in ra mã ASCII của ký tự đó, khi nào nhập số 0 thì dừng.
- Viết chương trình tìm ước số chung lớn nhất và bội số chung nhỏ nhất của 2 số nguyên.
- Viết chương trình in lá cờ nước Mỹ.
- Viết chương trình tính dân số của một thành phố sau 10 năm nữa, biết rằng dân số hiện nay là 6.000.000, tỉ lệ tăng dân số hàng năm là 1.8% .

Chương 7. HÀM

Học xong chương này, sinh viên sẽ nắm được các vấn đề sau:

- Khái niệm, cách khai báo về hàm.
- Cách truyền tham số, tham biến, tham trị.
- Sử dụng biến cục bộ, toàn cục trong hàm.
- Sử dụng tiền xử lý #define

7.1. KHÁI NIỆM VỀ HÀM TRONG C

Trong những chương trình lớn, có thể có những đoạn chương trình viết lặp đi lặp lại nhiều lần, để tránh rườm rà và mất thời gian khi viết chương trình; người ta thường phân chia chương trình thành nhiều module, mỗi module giải quyết một công việc nào đó. Các module như vậy gọi là các chương trình con.

Một tiện lợi khác của việc sử dụng chương trình con là ta có thể dễ dàng kiểm tra xác định tính đúng đắn của nó trước khi ráp nối vào chương trình chính và do đó việc xác định sai sót để tiến hành hiệu đính trong chương trình chính sẽ thuận lợi hơn.

Trong C, chương trình con được gọi là hàm. Hàm trong C có thể trả về kết quả

thông qua tên hàm hay có thể không trả về kết quả.

Hàm có hai loại: hàm chuẩn và hàm tự định nghĩa. Trong chương này, ta chú trọng đến cách định nghĩa hàm và cách sử dụng các hàm đó.

Một hàm khi được định nghĩa thì có thể sử dụng bất cứ đâu trong chương trình. Trong C, một chương trình bắt đầu thực thi bằng hàm main.

Ví dụ 1:

1	#include <stdio.h>
2	#include <conio.h>
3	
4	// khai báo prototype
5	void line();
6	
7	// ham in 1 dong dau
8	void line()
9	{
10	int i;
11	for(i = 0; i < 19; i++)
12	printf("*");
13	printf("\n");
14	}
15	
16	int main()
17	{
18	line();

19	printf("* Minh hoa ve ham *\n");
20	line();
21	return 0;
22	}

→ **Kết quả in ra màn hình**

* Minh hoa ve ham *

Giải thích chương trình

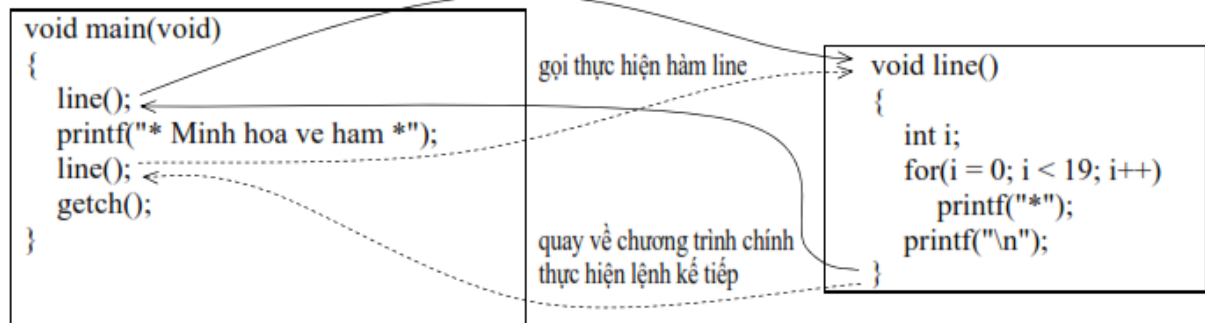
Dòng 8 đến dòng 14: định nghĩa hàm **line**, hàm này không trả về giá trị, thực hiện công việc in ra 19 dấu sao.

Dòng 5: khai báo prototype, sau tên hàm phải có dấu chấm phẩy

Trong hàm line có sử dụng biến i, biến i là biến cục bộ chỉ sử dụng được trong phạm vi hàm line.

Dòng 18 và 20: gọi thực hiện hàm line.

* Trình tự thực hiện chương trình



→ **Không có dấu chấm phẩy sau tên hàm, phải có cặp dấu ngoặc () sau tên hàm nếu hàm không có tham số truyền vào. Phải có dấu chấm phẩy sau tên hàm khai báo prototype. Nên khai báo prototype cho dù hàm được gọi nằm trước hay sau câu lệnh gọi nó.**

Ví dụ 2:

1	#include <stdio.h>
2	#include <conio.h>
3	
4	// khai bao prototype
5	int power(int, int);
6	
7	// ham tinh so mu
8	int power(int ix, int in)
9	{
10	int i, ip = 1;
11	for(i = 1; i <= in; i++)
12	ip *= ix;
13	return ip;
14	}
15	

16	int main()
17	{
18	printf("2 mu 2 = %d.\n", power(2, 2));
19	printf("2 mu 3 = %d.\n", power(2, 3));
20	return 0;
21	}

→ **Kết quả in ra màn hình**

2 mu 2 = 4.
2 mu 3 = 8.

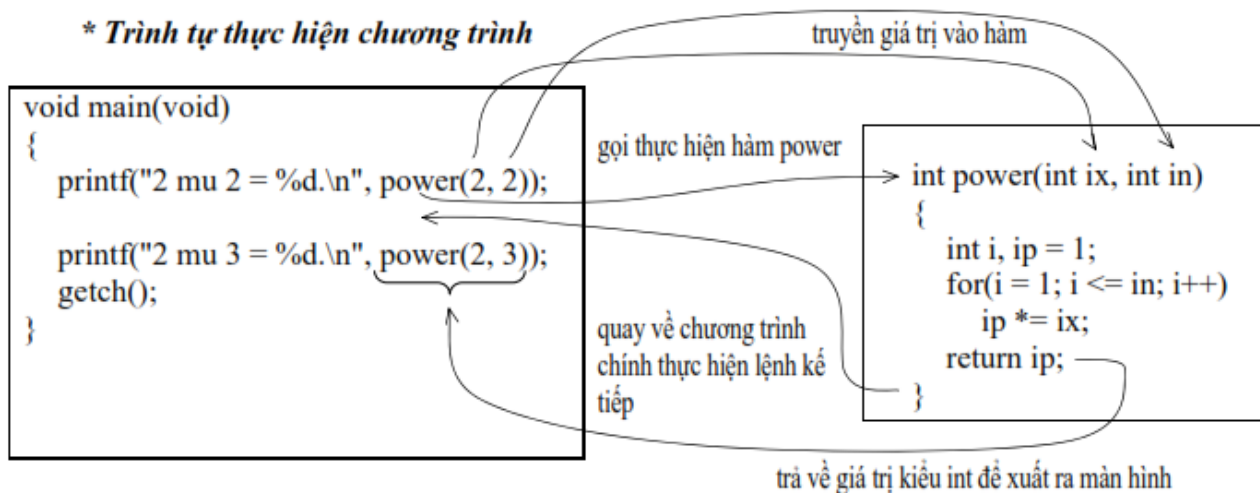
Giải thích chương trình

Hàm **power** có hai tham số truyền vào là ix, in có kiểu int và kiểu trả về cũng có kiểu int.

Dòng 13: return ip: trả về giá trị sau khi tính toán

Dòng 18: đối mục 2 và 3 có kiểu trả về là int sau khi thực hiện gọi power. Hai tham số ix, in của hàm power là dạng truyền tham trị.

* Trình tự thực hiện chương trình



→ Quy tắc đặt tên hàm giống tên biến, hằng... Mỗi đối số cách nhau = dấu phẩy kèm theo kiểu dữ liệu tương ứng.

Ví dụ 3:

1	#include <stdio.h>
2	#include <conio.h>
3	
4	// khai báo prototype
5	void time(int & , int &);
6	
7	// ham doi phut thanh gio:phut
8	void time(int &ig, int &ip)
9	{
10	ig = ip / 60;
11	ip %= 60;
12	}

13	int main()
14	{
15	int igio, iphut;
16	printf("Nhap vao so phut : ");
17	scanf("%d", &iphut); time(igio, iphut);
18	printf("%02d:%02d\n", igio, iphut);
19	return 0;
20	}
21	

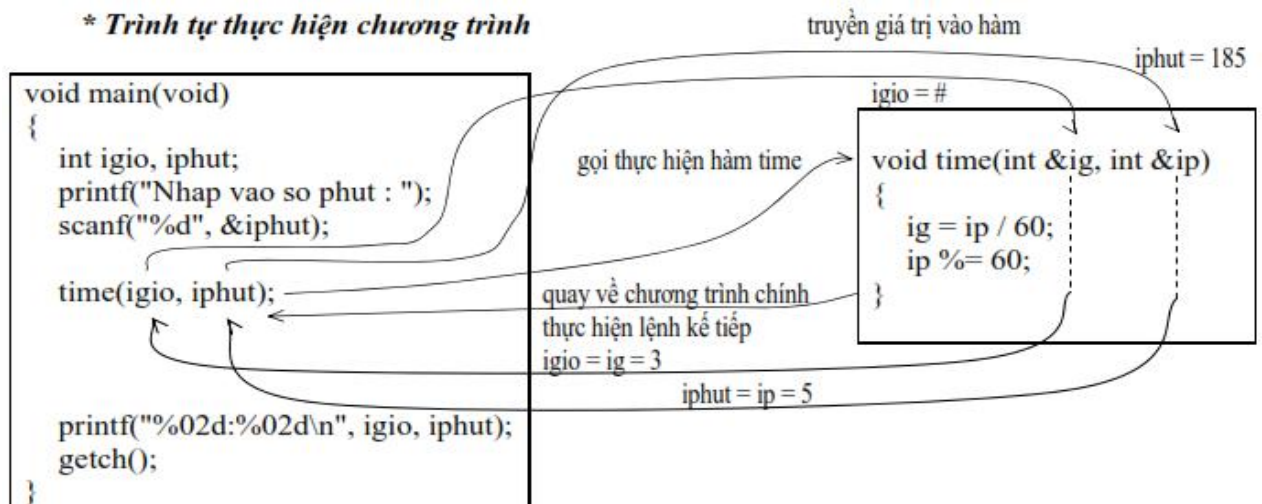
→ **Kết quả in ra màn hình**

Nhap vao so phut: 185
03:05

Giải thích chương trình

Hàm **time** có hai tham số truyền vào là ix, in có kiểu int. 2 tham số này có toán tử địa chỉ & đi trước cho biết 2 tham số này là dạng truyền tham biến.

* Trình tự thực hiện chương trình



7.2. KHAI BÁO VÀ LỜI GỌI HÀM

7.2.1. Định nghĩa hàm

Cấu trúc của một hàm tự thiết kế:

```

<kiểu kết quả> Tên hàm ([<kiểu t số> <tham số>][,<kiểu t số><tham số>][...])
{
    [Khai báo biến cục bộ]
    [các câu lệnh thực hiện hàm]
    [return [<Biểu thức>;]
}

```

Giải thích:

- **Kiểu kết quả:** là kiểu dữ liệu của kết quả trả về, có thể là : int, byte, char, float, void... Một hàm có thể có hoặc không có kết quả trả về. Trong trường hợp *hàm không có kết quả trả về ta nên sử dụng kiểu kết quả là void*.

- **Kiểu t số:** là kiểu dữ liệu của tham số.

- **Tham số:** là tham số truyền dữ liệu vào cho hàm, một hàm có thể có hoặc không có tham số. **Tham số này gọi là tham số hình thức, khi gọi hàm chúng ta phải truyền cho nó các tham số thực tế.** Nếu có nhiều tham số, mỗi tham số phân cách nhau dấu phẩy (,).

- Bên trong thân hàm (phần giới hạn bởi cặp dấu {}) là các khai báo cùng các câu lệnh xử lý. Các khai báo bên trong hàm được gọi là các khai báo cục bộ trong hàm và các khai báo này chỉ tồn tại bên trong hàm mà thôi.

- Khi định nghĩa hàm, ta thường sử dụng câu lệnh return để trả về kết quả thông qua tên hàm.

Lệnh return dùng để thoát khỏi một hàm và có thể trả về một giá trị nào đó.

Cú pháp:

return ;	/*không trả về giá trị*/
return <biểu thức>;	/*Trả về giá trị của biểu thức*/
return (<biểu thức>;	/*Trả về giá trị của biểu thức*/

Nếu hàm có kết quả trả về, ta bắt buộc phải sử dụng câu lệnh return để trả về

kết quả cho hàm.

Ví dụ 4: Viết hàm tìm số lớn giữa 2 số nguyên a và b

```
int max(int a, int b)
{
    return (a>b) ? a:b;
}
```

Ví dụ 5: Viết hàm tìm ước chung lớn nhất giữa 2 số nguyên a, b. Cách tìm: đầu tiên ta giả sử UCLN của hai số là số nhỏ nhất trong hai số đó. Nếu điều đó không đúng thì ta giảm đi một đơn vị và cứ giảm như vậy cho tới khi nào tìm thấy UCLN

```
int ucln(int a, int b)
{
    int u;
    if (a<b) u=a;
    else u=b;
    while ((a%u!=0) || (b%u!=0))
        u--;
    return u;
}
```

7.2.2. Gọi hàm

Một hàm khi định nghĩa thì chúng vẫn chưa được thực thi trừ khi ta có một lời gọi đến hàm đó.

Cú pháp gọi hàm: <Tên hàm>([Danh sách các tham số])

Ví dụ 6: Viết chương trình cho phép tìm ước số chung lớn nhất của hai số tự

nhiên.

```
#include<stdio.h>
unsigned int ucln(unsigned int a, unsigned int b)
{
    unsigned int u;
    if (a<b)
        u=a;
    else
        u=b;
    while ((a%u !=0) || (b%u!=0))
        u--;
    return u;
}
int main()
{
    unsigned int a, b, UC;
    printf("Nhap a,b: ");scanf("%d%d",&a,&b);
    UC = ucln(a,b);
    printf("Uoc chung lon nhat la: %d", UC);
    return 0;
}
```

Lưu ý: Việc gọi hàm là một phép toán, không phải là một phát biểu.

7.2.3. Khai báo nguyên mẫu cho hàm

Một nguyên mẫu hàm là một khai báo hàm trong đó xác định rõ kiểu dữ liệu của các đối số và trị trả về. Thông thường, các hàm được khai báo bằng cách xác định kiểu của giá trị được trả về bởi hàm, và tên hàm. Một hàm `abc()` có hai đối số kiểu `int` là `x` và `y`, và trả về một giá trị kiểu `char`, có thể được khai báo như sau:

```
char abc();
hoặc:
char abc(int x, int y);
```

Cách định nghĩa sau được gọi là **nguyên mẫu hàm**. Khi các nguyên mẫu được sử dụng, C có thể tìm và thông báo bất kỳ kiểu dữ liệu không hợp lệ khi chuyển đổi giữa các đối số được dùng để gọi một hàm với sự định nghĩa kiểu của các tham số. Một lỗi sẽ được thông báo ngay khi có sự khác nhau giữa số lượng các đối số được sử dụng để gọi hàm và số lượng các tham số khi định nghĩa hàm.

Cú pháp tổng quát của một nguyên mẫu hàm:

Kiểu Tên_hàm (*kiểu đối_thứ_1, kiểu đối_thứ_2, kiểu đối_thứ_N*);

Khi hàm được khai báo không có các thông tin nguyên mẫu, trình biên dịch cho rằng không có thông tin về các tham số được đưa ra. Một hàm không có đối số có thể gây ra lỗi khi khai báo không có thông tin nguyên mẫu. Để tránh điều này, khi một hàm không có tham số, nguyên mẫu của nó sử dụng **void** trong cặp dấu ngoặc `()`. Như đã nói ở trên, **void** cũng được sử dụng để khai báo tường minh một hàm không có giá trị trả về.

Ví dụ, nếu một hàm `noparam()` trả về kiểu dữ liệu `char` và không có các tham

số được gọi, có thể được khai báo như sau

```
char noparam(void);
```

Khai báo trên chỉ ra rằng hàm không có tham số, và bất kỳ lời gọi có truyền tham số đến hàm đó là không đúng.

Khi một hàm không nguyên mẫu được gọi tất cả các kiểu **char** được đổi thành kiểu **int** và tất cả kiểu **float** được đổi thành kiểu **double**. Tuy nhiên, nếu một hàm là nguyên mẫu, thì các kiểu đã đưa ra trong nguyên mẫu được giữ nguyên và không có sự tăng cấp kiểu xảy ra.

Nguyên mẫu của hàm được khai báo trước hàm main() (xem lại cấu trúc một chương trình C ở chương 2). Không nhất thiết phải khai báo nguyên mẫu hàm. Nhưng nói chung nên có vì nó cho phép chương trình biên dịch phát hiện lỗi khi gọi hàm hay tự động việc chuyển dạng.

7.2.4. Nguyên tắc hoạt động của hàm

Trong chương trình, khi gặp một lời gọi hàm thì hàm bắt đầu thực hiện bằng cách chuyển các lệnh thi hành đến hàm được gọi. Quá trình diễn ra như sau:

Tên_hàm (Danh sách các tham_số_thực);

Số các **tham_số_thực** thay vào trong danh sách các đối phải bằng số **tham_số_hình_thức** và lần lượt chúng có kiểu tương ứng với nhau. Cần chú ý là các **tham_số_hình_thức** đã khai báo có cùng kiểu dữ liệu với các đối số được sử dụng khi gọi hàm. Trong trường hợp có sai, C có thể không hiển thị lỗi nhưng có thể đưa ra một kết quả không mong muốn. Điều này là vì, C vẫn đưa ra một vài kết quả trong các tình huống khác thường. Người lập trình phải đảm bảo rằng không có các lỗi về sai kiểu.

Khi gặp một lời gọi hàm thì nó sẽ bắt đầu được thực hiện. Nói cách khác, khi máy gặp lời gọi hàm ở một vị trí nào đó trong chương trình, máy sẽ tạm dời chỗ đó và chuyển đến hàm tương ứng. Quá trình đó diễn ra theo trình tự sau :

- Cấp phát bộ nhớ cho các biến cục bộ.
- Gán giá trị của các tham số thực sự cho các đối tượng ứng.
- Thực hiện các câu lệnh trong thân hàm.

Khi gặp câu lệnh return hoặc dấu } cuối cùng của thân hàm thì máy sẽ hủy các đối, biến cục bộ và ra khỏi hàm.

Nếu trở về từ một câu lệnh **return** có chứa biểu thức thì giá trị của biểu thức được gán cho hàm. Giá trị của hàm sẽ được sử dụng trong các biểu thức chứa nó.

7.2.4.1. Truyền theo trị

Mặc nhiên, việc truyền tham số cho hàm trong C là truyền theo giá trị; nghĩa là các giá trị thực (tham số thực) không bị thay đổi giá trị khi truyền cho các tham số hình thức.

Ví dụ 7: Giả sử ta muốn in ra nhiều dòng, mỗi dòng 50 ký tự nào đó. Để đơn giản ta viết một hàm, nhiệm vụ của hàm này là in ra trên một dòng 50 ký tự nào đó. Hàm này có tên là InKT.


```
#include <stdio.h>
#include <conio.h>
void InKT(char ch)
{
    int i;
    for(i=1;i<=50;i++) printf("%c",ch);
    printf("\n");
}
int main()
{
    char c = 'A';
    InKT('*'); /* In ra 50 dau * */
    InKT('+');
    InKT(c);
    return 0;
}
```

- Trong hàm **InKT** ở trên, biến **ch** gọi là tham số hình thức được truyền bằng giá trị (gọi là tham trị của hàm). Các tham trị của hàm coi như là một biến cục bộ trong hàm và chúng được sử dụng như là dữ liệu đầu vào của hàm.

- Khi chương trình con được gọi để thi hành, tham trị được cấp ô nhớ và nhận giá trị là bản sao giá trị của tham số thực. Do đó, mặc dù tham trị cũng là biến, nhưng việc thay đổi giá trị của chúng không có ý nghĩa gì đối với bên ngoài hàm, không ảnh hưởng đến chương trình chính, nghĩa là không làm ảnh hưởng đến tham số thực tương ứng.

Ví dụ 8: *Ta xét chương trình sau đây:*

```
#include <stdio.h>
#include <conio.h>
int hoanvi(int a, int b)
{
    int t; // Biến cục bộ
    t=a;
    a=b;
    b=t;
    printf("\nBen trong ham a=%d    , b=%d",a,b);
    return 0;
}
int main()
{
    int a, b;
    clrscr();
    printf("\n Nhap vao 2 so nguyen a, b:");
    scanf("%d%d",&a,&b);
    printf("\n Truoc khi goi ham hoan vi a=%d    ,b=%d",a,b);
    hoanvi(a,b);
    printf("\n Sau khi goi ham hoan vi a=%d ,b=%d",a,b);
    getch();
    return 0;
}
```

→**Kết quả thực hiện chương trình:**

Nhap vao 2 so nguyen a, b : 6 5

Truoc khi goi ham hoan vi a=6, b=5

Ben trong ham a=5, b=6

Sau khi goi ham hoan vi a=6, b=5

7.2.4.2. Truyền theo địa chỉ

Để thực sự làm thay đổi giá trị các tham biến truyền vào thì phải thực hiện phương thức truyền theo địa chỉ. Trong ví dụ 5.6, nếu ta muốn sau khi kết thúc chương trình con giá trị của a, b thay đổi thì ta phải đặt tham số hình thức là các con trỏ, còn tham số thực tế là địa chỉ của các biến. Lúc này mọi sự thay đổi trên vùng nhớ được quản lý bởi con trỏ là các tham số hình thức của hàm thì sẽ ảnh hưởng đến *vùng nhớ đang được quản lý bởi tham số thực tế tương ứng* (cần đề ý rằng vùng nhớ này chính là các biến ta cần thay đổi giá trị).

Người ta thường áp dụng cách này đối với các dữ liệu đầu ra của hàm.

Ví dụ 9: Xét chương trình sau đây

```
#include <stdio.h>
#include <conio.h>
long hoanvi(long *a, long *b)
/* Khai báo tham số hình thức *a, *b là các con trỏ kiểu long */
{
    long temp;
    temp=*a; /* gán nội dung của a cho temp */
    *a=*b;    /* Gán nội dung của b cho a */
    *b=temp; /* Gán nội dung của temp cho b */
    printf("\n Ben trong ham a=%ld      , b=%ld",*a,*b);
    /*In ra nội dung của a, b*/
    return 0;
}
int main()
{
    long a, b;
    printf("\n Nhap vao 2 so nguyen a, b:");
    scanf("%ld%ld",&a,&b);
    printf("\n Truoc khi goi ham hoan vi a=%ld      ,b=%ld",a,b);
    hoanvi(&a,&b); /* Phải là địa chỉ của a và b */
    printf("\n Sau khi goi ham hoan vi a=%ld,b=%ld",a,b);
    return 0;
}
```

Kết quả thực hiện chương trình:

Nhap vao 2 so nguyen a, b : 6 5

Truoc khi goi ham hoan vi a=6, b=5

Ben trong ham a=5, b=6

Sau khi goi ham hoan vi a=5, b=6

7.3. HÀM *main()*

Mọi chương trình C đều phải có một hàm có tên gọi là *main()*. Hàm này sẽ được gọi thực hiện đầu tiên trong chương trình C. Khi *main()* kết thúc thì chương trình cũng hoàn thành. Trình biên dịch xem hàm *main()* cũng như bất kỳ một hàm nào khác ngoại trừ một điểm đó là: Khi chạy chương trình, môi trường chức năng cấp cho nó hai tham số. Tham số đầu tiên thường được gọi tắt là **argc** có kiểu **int** để biểu diễn số các tham số có trên dòng lệnh (Kể cả tên chương trình) khi chương trình được gọi thực hiện; Tham số thứ hai, được gọi tắt là **argv**, là một mảng các con trỏ trỏ đến các tham số lệnh.

7.4. PHẠM VI HOẠT ĐỘNG CỦA BIẾN

7.4.1. Biến cục bộ

Các biến được khai báo trong thân hàm thì chỉ có tầm hoạt động trong thân hàm đấy, gọi là *biến cục bộ*. Sau khi kết thúc hoạt động của hàm thì các biến này cũng được giải phóng khỏi bộ nhớ. Do đối và biến cục bộ đều có phạm vi hoạt động trong cùng một hàm nên đối và biến cục bộ cần có tên khác nhau.

Ví dụ 10:

```
func(int a, int b)
{
    int x,y;
    ...
}
```

Các biến *x,y* được khai báo là biến cục bộ và chỉ có thể truy xuất ở trong hàm **func** mà thôi. Các hàm khác không thể truy xuất các biến này.

☞ Đối và biến cục bộ có thể trùng tên với các đại lượng ngoài hàm mà không gây ra nhầm lẫn nào. Nếu việc trùng tên xảy ra thì ưu tiên dùng biến cục bộ.

7.4.2. Biến toàn cục

Khi cần dùng đến các biến mà tất cả các hàm dùng trong chương trình đều có thể sử dụng chúng, người ta khai báo các biến này ở ngoài. Các biến như thế được gọi là biến ngoài hay biến toàn cục.

Ví dụ 11:

```
int a; //Biến ngoài
main()
{
    ...
}
func()
{
```

```
...  
}
```

Biến `a` ở trên có thể được sử dụng cả trong hàm **main()** và hàm **func()**. Chúng tồn tại cho đến khi kết thúc chương trình.

Các biến toàn cục được lưu trữ trong các vùng cố định của bộ nhớ. Các biến toàn cục hữu dụng khi nhiều hàm trong chương trình sử dụng cùng dữ liệu. Tuy nhiên, nên tránh sử dụng biến toàn cục nếu không cần thiết, vì chúng giữ bộ nhớ trong suốt thời gian thực hiện chương trình. Vì vậy việc sử dụng một biến toàn cục ở nơi mà một biến cục bộ có khả năng đáp ứng cho hàm sử dụng là không hiệu quả. Ví dụ 5.13 sẽ giúp làm rõ hơn điều này:

Ví dụ 12:

```
void cong2so(int i, int j)  
{  
    return(i + j);  
}  
int i, j;  
void congso(void)  
{  
    return(i + j);  
}
```

Cả hai hàm **cong2so()** và **congso()** đều trả về tổng của các biến **i** và **j**. Tuy nhiên, hàm **cong2so ()** được sử dụng để trả về tổng của hai số bất kỳ; trong khi hàm **congso()** chỉ trả về tổng của các biến toàn cục **i** và **j**.

7.4.3. Biến tĩnh

Trong C có khai báo đặc biệt làm cho biến tồn tại suốt thời gian hoạt động của chương trình, đó là biến tĩnh với từ khóa **static**.

Ví dụ 13:

```
func()  
{  
    static int b;  
}
```

Biến tĩnh chỉ được hàm có định nghĩa nó truy cập đến. Nhưng không giống như biến tự động, nó không biến mất khi hàm kết thúc mà vẫn giữ nguyên trong bộ nhớ. Nếu quyền điều khiển trả lại cho hàm thì giá trị của nó vẫn còn để hàm dùng đến.

Ví dụ 14:

```
void funct1(int a)  
{  
    static int b=0;  
    b+=a;  
    printf("\n b=%d", b);  
}  
int main()  
{
```

```

    funct1 (1) ;
    funct1 (1) ;
    return 0;
}

```

Kết quả: b=1
 b=2

Tại dòng kết quả thứ 2 giá trị của b tăng lên 1 thừa kế kết quả của lần gọi trước cho nên giá trị của nó là 2.

Nếu bỏ từ khóa tại static, thì kết quả là b=1
 b=1

7.5. HÀM TRÊN DÒNG - MACRO

C đưa ra một số cách mở rộng ngôn ngữ bằng các bộ tiền xử lý macro đơn giản. Có hai cách mở rộng chính là #define mà ta đã học và khả năng bao hàm nội dung của các file khác vào file đang được dịch.

Phép thế MACRO :

Định nghĩa có dạng :

```
#define biểu_thức_1 [ biểu_thức_2 ]
```

sẽ gọi tới một macro để thay thế biểu_thức_2 (nếu có) cho biểu_thức_1.

Ví dụ 15:

```
#define YES 1
```

Macro thay biến YES bởi giá trị 1 có nghĩa là hễ có chỗ nào trong chương trình có xuất hiện biến YES thì nó sẽ được thay bởi giá trị 1.

Phạm vi cho tên được định nghĩa bởi #define là từ điểm định nghĩa đến cuối file gốc. Có thể định nghĩa lại tên và một định nghĩa có thể sử dụng các định nghĩa khác trước đó. Phép thế không thực hiện cho các dấu nháy, ví dụ như YES là tên được định nghĩa thì không có việc thay thế nào được thực hiện trong đoạn lệnh có "YES".

Ta cũng có thể định nghĩa các macro có đối, do vậy văn bản thay thế sẽ phụ thuộc vào cách gọi tới macro.

Ví dụ 16:

Định nghĩa macro gọi max như sau :

```
#define max(a,b) ((a)>(b) ? (a) : (b))
```

Việc sử dụng :

```
x=max(p+q, r+s) ;
```

tương đương với :

```
x=((p+q)>(r+s) ? (p+q) : (r+s)) ;
```

Như vậy ta có thể có hàm tính cực đại viết trên một dòng. Chừng nào các đối còn giữ được tính nhất quán thì macro này vẫn có giá trị với mọi kiểu dữ liệu, không cần phải có các loại hàm max khác cho các kiểu dữ liệu khác nhưng vẫn phải có đối cho các

hàm. Nên dùng các dấu ngoặc trong macro để chính xác hóa ý định của mình (tường minh). Ví dụ nếu dùng:

```
#define SUM(x,y) x+y;
...
tong=10*SUM(3,4);
```

thì khi chạy chương trình `tong` sẽ có giá trị là bao nhiêu? Bạn đoán là `SUM(3,4)` có giá trị là 7 và `tong` là 70 ! Hoàn toàn sai, vì phép thay macro sẽ thực hiện là

```
tong=10*3+4;
```

Do phép nhân có độ ưu tiên cao hơn phép cộng nên kết quả của `tong` là 34. Vì thế chính xác là phải làm:

```
#define SUM(x,y) (x+y);
...
```

Và khi thay macro sẽ thực hiện là

```
tong=10*(3+4);
```

Macro dùng thuận tiện hơn hàm, tuy vậy macro chỉ làm được các công việc giản đơn thường chỉ có một phát biểu trên một dòng (inline). Mỗi lần gọi macro thì đoạn mã của nó sẽ thay thế vào văn bản chương trình. Nghĩa là sẽ có nhiều bản sao chép đoạn mã giống nhau. Đối với hàm thì chỉ xuất hiện một lần nên dùng hàm thì tiết kiệm bộ nhớ hơn. Trái lại gọi macro thì không phải chờ đợi gì cả vì không phải truyền đối và chuyển giao điều khiển, do đó dùng macro chạy nhanh hơn hàm.

7.6. SỰ ĐỆ QUY

7.6.1. Định nghĩa

Một hàm được gọi là đệ quy nếu bên trong thân hàm có lệnh gọi đến chính nó.

Ví dụ 17: Người ta định nghĩa giai thừa của một số nguyên dương n như sau:

$$n! = 1 * 2 * 3 * \dots * (n-1) * n = (n-1)! * n \quad (\text{với } 0! = 1)$$

Như vậy, để tính $n!$ ta thấy nếu $n=0$ thì $n!=1$ ngược lại thì $n!=n * (n-1)!$ Với định nghĩa trên thì hàm đệ quy tính $n!$ được viết:

```
#include <stdio.h>
#include <conio.h>
/*Hàm tính n! bằng đệ quy*/
unsigned int giaithua_dequy(int n)
{
    if (n==0)
        return 1;
    else
        return n*giaithua_dequy(n-1);
}
/*Hàm tính n! không đệ quy*/
unsigned int giaithua_khongdequy(int n)
```

```

{
    unsigned int kq=1,i;
    for (i=2;i<=n;i++)
        kq=kq*i;
    return kq;
}
int main()
{
    int n;
    printf("\n Nhap so n can tinh giai thua ");
    scanf("%d",&n);
    printf("\nGoi ham de quy: %d !=
%u",n,giaithua_dequy(n));
    printf("\nGoi ham khong de
    quy:%d!=%u",n,giaithua_khongdequy(n));
    return 0;
}

```

7.6.2. Đặc điểm cần lưu ý khi viết hàm đệ quy

- Hàm đệ quy phải có 2 phần:

- Phần dừng hay phải có trường hợp nguyên tố. Trong ví dụ 17 thì trường hợp $n=0$ là trường hợp nguyên tố.
- Phần đệ quy: là phần có gọi lại hàm đang được định nghĩa. Trong ví dụ 17 thì phần đệ quy là $n>0$ thì $n! = n * (n-1)!$

```

if      ( trường hợp nguyên tố)
    {
        Trình bày cách giải bài toán khi suy biến
    };
else /* Trường hợp tổng quát */
    {
        Gọi đệ qui tới hàm ( đang viết ) với các giá
        trị khác của tham số
    };

```

- Sử dụng hàm đệ quy trong chương trình sẽ làm chương trình dễ đọc, dễ hiểu và vấn đề được nêu bật rõ ràng hơn. Tuy nhiên trong đa số trường hợp thì hàm đệ quy tốn bộ nhớ nhiều hơn và tốc độ thực hiện chương trình chậm hơn không đệ quy. Trong nhiều trường hợp phải khử đệ qui và thay vào đó là các vòng lặp nhằm tăng tốc độ thực hiện chương trình.

- Tùy từng bài có cụ thể mà người lập trình quyết định có nên dùng đệ quy hay không (có những trường hợp không dùng đệ quy thì không giải quyết được bài toán).

7.7. VÍ DỤ VỀ SỬ DỤNG HÀM

Ví dụ 18 về một chương trình có sử dụng các hàm kiểm tra xem ngày tháng năm nhập vào có hợp lệ hay không:

Ví dụ 18:

```
#include <stdio.h>
#include <conio.h>
char nhuan(int nam)
{
    if ( nam % 100 )
        return( !(nam % 4));
    else
        return( !(nam % 400));
}
char kiemtra(int ngay, int thang, int nam)
{
    char f1,f2,f3;
    int f;
    f1=(ngay==30 && thang==2);
    f2=(ngay==29 && thang==2 && !nhuan(nam));
    f=(thang-2)*(thang-4)*(thang-6);
    f=f*(thang-9)*(thang-11);
    f3=(ngay==31 && !f);
    return !(f1 || f2 || f3);
}

tieptuc()
{
    char ch;
    printf("\nCo tiep tục nữa không ? ( C/K )");
    ch=getche();
    ch=toupper(ch);
    return(ch-'K');
}

main()
{
    char tiep;
    int ngay,thang,nam;
    do
    {
        printf("\nCho biết cần kiểm tra ?");
        printf("\nNgày = ");
        scanf("%d",&ngay);
        printf("\nTháng = ");
        scanf("%d",&thang);
        printf("\nTên = ");
        scanf("%d",&nam);
        if(kiemtra(ngay,thang,nam))
            printf("Ngày nay hợp lệ");
        else
            printf("Ngày nay không hợp lệ");
    }
```



```

        tiep=tieptuc();
    }
    while (tiep);
}

```

CÂU HỎI VÀ BÀI TẬP THỰC HÀNH

- Viết hàm tìm số lớn nhất trong hai số. Áp dụng tìm số lớn nhất trong ba số a, b, c với a, b, c nhập từ bàn phím.
- Viết hàm tìm UCLN của hai số a và b. Áp dụng: nhập vào tử và mẫu số của một phân số, kiểm tra xem phân số đó đã tối giản hay chưa.
- Viết hàm in n ký tự c trên một dòng. Viết chương trình cho nhập 5 số nguyên cho biết số lượng hàng bán được của mặt hàng A ở 5 cửa hàng khác nhau. Dùng hàm trên vẽ biểu đồ so sánh 5 giá trị đó, mỗi trị dùng một ký tự riêng.
- Viết một hàm tính tổng các chữ số của một số nguyên. Viết chương trình nhập vào một số nguyên, dùng hàm trên kiểm tra xem số đó có chia hết cho 3 không. Một số chia hết cho 3 khi tổng các chữ số của nó chia hết cho 3.
- Tam giác Pascal là một bảng số, trong đó hàng thứ 0 bằng 1, mỗi một số hạng

của hàng thứ n+1 là một tổ hợp chập k của n ($C_n^k = \frac{n!}{k!(n-k)!}$)

Tam giác Pascal có dạng sau:

```

1                ( hàng 0 )
1 1              ( hàng 1 )
1 2 1            ( hàng 2 )
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
1 6 15 20 15 6 1 (hàng 6)
.....

```

Viết chương trình in lên màn hình tam giác Pascal có n hàng (n nhập vào khi chạy chương trình) bằng cách tạo hai hàm tính giai thừa và tính tổ hợp.

- Yêu cầu như câu 5 nhưng dựa vào tính chất sau của tổ hợp: $C_n^k = C_{n-1}^{k-1} + C_{n-1}^k$ thành thuật toán là: tạo một hàm tổ hợp có hai biến n, k mang tính đệ quy như sau:

$$Tohop(n,k) = \begin{cases} 1 & \text{khi } k=0, k=n \\ Tohop(k-1, n-1) + Tohop(k, n-1) & \text{khi } 1 < k < n \end{cases}$$

- Viết chương trình tính các tổng sau:

a) $S = 1 + x + x^2 + x^3 + \dots + x^n$

b) $S = 1 - x + x^2 - x^3 + \dots + (-1)^n x^n$

c) $S = 1 + x/1! + x^2/2! + x^3/3! + \dots + x^n/n!$

Trong đó n là một số nguyên dương và x là một số bất kỳ được nhập từ bàn phím khi chạy chương trình.

8. Viết chương trình in dãy Fibonacci đã nêu trong bảng phương pháp dùng một hàm Fibonacci F có tính đệ quy.

$$F_n = \begin{cases} 1, & \text{khi } n = 1 \\ 2, & \text{khi } n = 2 \\ F_{n-1} + F_{n-2} \end{cases}$$

9. Bài toán tháp Hà Nội: Có một cái tháp gồm n tầng, tầng trên nhỏ hơn tầng dưới (hình vẽ). Hãy tìm cách chuyển cái tháp này từ vị trí thứ nhất sang vị trí thứ hai thông qua vị trí trung gian thứ ba. Biết rằng chỉ được chuyển mỗi lần một tầng và không được để tầng lớn trên tầng nhỏ.



10. Viết chương trình phân tích một số nguyên dương ra thừa số nguyên tố.

Chương 8. MẢNG VÀ CHUỖI

Sau khi hoàn tất bài này học viên sẽ hiểu và vận dụng các kiến thức kỹ năng cơ bản sau:

- Ý nghĩa, cách khai báo mảng
- Nhập, xuất mảng
- Khởi tạo mảng chuỗi.
- Một số kỹ thuật thao tác trên mảng
- Dùng mảng làm tham số cho hàm.

8.1. MẢNG TRONG C

Là tập hợp các phần tử có cùng dữ liệu. Giả sử bạn muốn lưu n số nguyên để tính trung bình, bạn không thể khai báo n biến để lưu n giá trị rồi sau đó tính trung bình.

Ví dụ 1 : bạn muốn tính trung bình 10 số nguyên nhập vào từ bàn phím, bạn sẽ khai báo 10 biến: a, b, c, d, e, f, g, h, i, j có kiểu int và lập thao tác nhập cho 10 biến này như sau:

```
printf("Nhập vào biến a: ");
```

```
scanf("%d", &a);
```

10 biến bạn sẽ thực hiện 2 lệnh trên 10 lần, sau đó tính trung bình: $(a + b + c + d + e + f + g + h + i + j)/10$

→ Điều này chỉ phù hợp với n nhỏ, còn đối với n lớn thì khó có thể thực hiện được. Vì vậy khái niệm mảng được sử dụng

8.1.1. Mảng một chiều.

8.1.1.1. Khai báo

Có thể khai báo mảng một chiều qua hai cách sau.

Cách 1: Khai báo tường minh

Kiểu Tên_mảng[số_phần_tử]

Tên_mảng: đây là một cái tên đặt đúng theo quy tắc đặt tên của danh biểu. Tên này cũng mang ý nghĩa là tên biến mảng.

Số_phần_tử: là một hằng số nguyên, cho biết số lượng phần tử tối đa trong mảng là bao nhiêu (hay nói khác đi kích thước của mảng là gì).

Kiểu: mỗi phần tử của mảng có dữ liệu thuộc kiểu gì.

Ở đây, ta khai báo một biến mảng gồm có **số_phần_tử** phần tử, phần tử thứ nhất là

tên mảng [0], phần tử cuối cùng là **tên mảng**[**số_phần_tử** -1]

Ví dụ 2:

```
int a[10];  
/* Khai báo biến mảng tên a, phần tử thứ nhất là a[0],  
phần tử cuối cùng là a[9].*/
```

Ta có thể coi mảng a là một dãy liên tiếp các phần tử trong bộ nhớ như sau:

Vị trí	0	1	2	3	4	5	6	7	8	9
Tên phần tử										

Cách 2: Khai báo mảng với số phần tử không xác định (khai báo không tường minh)

Kiểu Tên_mảng []

Khi khai báo, không cho biết rõ số phần tử của mảng, kiểu khai báo này thường được áp dụng trong các trường hợp: vừa khai báo vừa gán giá trị, khai báo mảng là tham số hình thức của hàm.

- Vừa khai báo vừa gán giá trị

Cú pháp:

Kiểu Tên_mảng[] = {Danh sách các giá trị};

Nếu vừa khai báo vừa gán giá trị thì mặc nhiên C sẽ hiểu số phần tử của mảng là số giá trị mà chúng ta gán cho mảng trong cặp dấu {}. **Danh sách các giá trị** được phân cách nhau bằng dấu phẩy. Chúng ta có thể sử dụng hàm sizeof() để lấy số phần tử của mảng như sau:

Số phần tử = sizeof(tên mảng) / sizeof(kiểu)

Ví dụ 3:

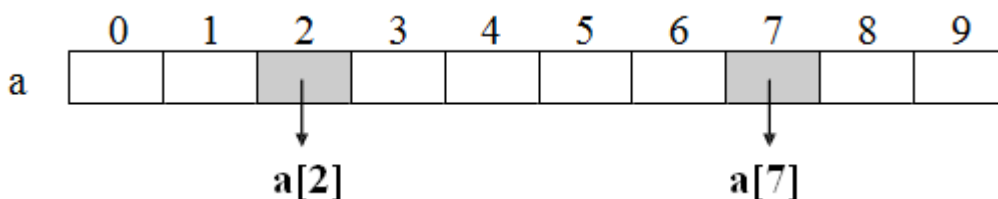
```
int a[] = {1, 5, -2, 22, 45, 12};
```

Mảng a trong ví dụ 6.2 có 6 phần tử được truy xuất từ a[0] đến a[5].

- Khai báo mảng là tham số hình thức của hàm, trong trường hợp này ta không cần chỉ định số phần tử của mảng là bao nhiêu.

8.1.1.2. Truy xuất các phần tử của mảng

Sau khi mảng được khai báo, mỗi phần tử trong mảng đều có chỉ số để tham chiếu. Chỉ số bắt đầu từ 0 đến n-1 (với n là kích thước mảng). Trong ví dụ trên, ta khai báo mảng 10 phần tử thì chỉ số bắt đầu từ 0 đến 9. a[2], a[7]... là phần tử thứ 3, 8... trong mảng xem như là một biến kiểu **int**.



8.1.1.3. Nhập dữ liệu cho mảng

for (i = 0; i < 10; i++) //vòng for có giá trị i chạy từ 0 đến 9

```

    {
        printf("Nhap vao phan tu thu %d: ", i + 1);
        scanf("%d", &ia[i]);
    }

```

8.1.1.4. Đọc dữ liệu từ mảng

```

for(i = 0; i < 10; i++)
    printf("%3d ", ia[i]);

```

Ví dụ 4 : Viết chương trình nhập vào n số nguyên. Tính và in ra trung bình cộng.

1	/* Tính trung bình cộng n số nguyên */
2	
3	#include <stdio.h>
4	#include <conio.h>
5	
6	int main()
7	{
8	int ia[50], i, in, isum = 0;
9	printf("Nhap vao gia tri n: ");
10	scanf("%d", &in);
11	
12	//Nhap du lieu vao mang
13	for(i = 0; i < in; i++)
14	{
15	printf("Nhap vao phan tu thu %d: ", i + 1);
16	scanf("%d", &ia[i]); //Nhap gia tri cho phan tu thu i
17	}
18	
19	//Tinh tong gia tri cac phan tu
20	for(i = 0; i < in; i++)
21	isum += ia[i]; //cong don tung phan tu vao isum
22	
23	printf("Trung binh cong: %.2f\n", (float) isum/in);
24	return 0;
25	}

→ **Kết quả in ra màn hình**

Nhập vào gia trị n: 3 Nhập vào phan tu thu 1: 7 Nhập vào phan tu thu 2: 3 Nhập vào phan tu thu 3: 6 Trung binh cong: 5.33	Bạn có thể gộp 2 lệnh for thành một vừa nhập vừa tính tổng, đưa hàng 21 sau hàng 16 và bỏ các hàng 19, 20, 21. Chạy và quan sát kết quả.
---	---

⚠ Điều gì sẽ xảy ra cho đoạn chương trình trên nếu bạn nhập n > 50 trong khi bạn chỉ khai báo mảng ia tối đa là 50 phần tử. Bạn dùng lệnh if để ngăn chặn điều này trước khi vào thực hiện lệnh for. Thay dòng 9, 10 bằng đoạn lệnh sau :

do

```

{
    printf("Nhap vao gia tri n: ");
    scanf("%d", &in);
} while (in <= 0 || in > 50);    //chi chap nhan gia tri nhap vao trong khoang
1..50

```

→ Chạy chương trình và nhập n với các giá trị -6, 0, 51, 6. Quan sát kết quả.

8.1.1.5. Kỹ thuật Sentinel

Sử dụng kỹ thuật này để nhập liệu giá trị cho các phần tử mảng mà không biết rõ số lượng phần tử sẽ nhập vào là bao nhiêu (không biết số n).

Ví dụ 5 : Viết chương trình nhập vào 1 dãy số dương rồi in tổng các số dương đó.

Phác họa lời giải: Chương trình yêu cầu nhập vào dãy số dương mà không biết trước số lượng phần tử cần nhập là bao nhiêu, vì vậy để chấm dứt nhập liệu khi thỏa mãn bằng cách nhập vào số âm hoặc không.

```

1  /* Nhap vao day so nguyen duong, tinh tong cac so duong */
2
3  #include <stdio.h>
4  #include <conio.h>
5  #define MAX  50
6
7  int main()
8  {
9      float fa[MAX], fsum = 0;
10     int i = 0;
11     do
12     {
13         printf("Nhap vao phan tu thu %d: ", i + 1);
14         scanf("%f", &fa[i]);    //Nhap gia tri cho phan tu thu i
15     } while (fa[i++] > 0);    //con nhap lieu khi gia tri phan tu > 0
16
17     i--;    //giam i di 1 lan cuoi cung tang 1 truoc khi thoat
18     //Tinh tong
19     for(int ij = 0; ij < i; ij++)
20         fsum += fa[ij];    //cong don tung phan tu vao fsum
21
22     printf("Tong : %5.2f\n", fsum);
23     return 0;
24 }

```

→ **Kết quả in ra màn hình**

Nhap vao phan tu thu 1: 1.2 Nhap vao phan tu thu 2: 3 Nhap vao phan tu thu 3: 4.6 Nhap vao phan tu thu 4: -9 Tong : 8.80	Bạn chạy lại chương trình và thử lại với số liệu khác. Quan sát kết quả.
--	--

⚠ Điều gì sẽ xảy ra cho đoạn chương trình trên nếu bạn nhập số lượng phần tử vượt quá 50 trong khi bạn chỉ khai báo mảng fa tối đa là MAX = 50 phần

tử. Bạn dùng lệnh break để thoát khỏi vòng lặp do...while trước khi bước sang phần tử thứ 51. Thêm đoạn lệnh sau vào trước dòng 13:

```

if (i >= MAX)                //kiem traphan tu buoc sang 51
{
    printf("Mang da day!\n"); //thong bao "Mang da day"
    i++;                      //tang i len 1 do dong 17 giam i xuong 1
    break;                    //thoat khoi vong lap do...while
}

```

→ Sửa dòng 5 thành #define MAX 4. Chạy chương trình và nhập các số 1.2, 3.5, 6.5, 4. Quan sát kết quả.

Ví dụ 6 : Có 4 loại tiền 1, 5, 10, 25 và 50 đồng. Hãy viết chương trình nhập vào số tiền sau đó cho biết số số tiền trên gồm mấy loại tiền, mỗi loại bao nhiêu tờ.

Phác họa lời giải: Số tiền là 246 đồng gồm 4 tờ 50 đồng, 1 tờ 25 đồng, 2 tờ 10 đồng, 0 tờ 5 đồng và 1 tờ 1 đồng, Nghĩa là bạn phải xét loại tiền lớn trước, nếu hết khả năng mới xét tiếp loại kế tiếp.

```

1  /* Nhap vao so tien va doi tien ra cac loai 50, 25, 10, 5, 1 */
2  #include <stdio.h>
3  #include <conio.h>
4  #define MAX 5
5  int main()
6  {
7      int itien[MAX] = {50, 25, 10, 5, 1}; //Khai bao va khoi tao mang voi 5 phan tu
8      int i, isotien, ito;
9      printf("Nhap vao so tien: ");
10     scanf("%d", &isotien); //Nhap vao so tien
11     for (i = 0; i < MAX; i++)
12     {
13         ito = isotien/itien[i]; //Tim so to cua loai
14         tien thu i printf("%4d to %2d dong\n", ito,
15         itien[i]);
16         isotien = isotien%itien[i]; //So tien con lai sau khi da loai tru cac loai tien da
17         co
18     }
19     return 0;
20 }

```

→ **Kết quả in ra màn hình**

Nhap vao so tien: 246
 4 tờ 50 đồng
 1 tờ 25 đồng
 2 tờ 10 đồng
 0 tờ 5 đồng
 1 tờ 1 đồng

Bạn chạy lại chương trình và thử lại với số liệu khác.
 Quan sát kết quả.

≠ Điều gì sẽ xảy nếu số phần tử mảng lớn hơn số mục, số phần tử đôi ra không được khởi tạo sẽ điền vào số 0. Nếu số phần tử nhỏ hơn số mục khởi tạo trình

biên dịch sẽ báo lỗi.

Ví dụ 7: `int itien[5] = {50, 25}`, phần tử `itien[0]` sẽ có giá trị 50, `itien[1]` có giá trị 25, `itien[2]`, `itien[3]`, `itien[4]` có giá trị 0.

`int itien[3] = {50, 25, 10, 5, 1}` → trình biên dịch báo lỗi

8.1.1.6. Dùng mảng 1 chiều làm tham số cho hàm

Ví dụ 8: Tìm số lớn nhất trong dãy số nguyên dương.

```
1  /* Chương trình tìm số lớn nhất sử dụng hàm */
2
3  #include <stdio.h>
4  #include <conio.h>
5
6  #define MAX 20
7
8  //Khai báo prototype
9  int max(int, int);
10
11  //hàm tìm số lớn nhất trong mảng 1 chiều
12  int max(int ia[], int in)
13  {
14      int i, imax;
15      for (i = 1; i < in; i++)
16          if (imax < ia[i]) //nếu số đang xét > max
17              imax = ia[i]; //gán số này cho max
18      return imax; //tra về kết quả số lớn nhất
19  }
20
21  int main()
22  {
23      int ia[MAX];
24      int i = 0, inum;
25      do
26      {
27          printf("Nhập vào một số: ");
28          scanf("%d", &ia[i]);
29      } while (ia[i++] != 0);
30      i--;
31      inum = max(ia, i);
32      printf("Số lớn nhất là: %d.\n", inum);
33      return 0;
34  }
```

→ **Kết quả in ra màn hình**

Nhập vào một số: 12 Nhập vào một số: 45 Nhập vào một số: 3 Nhập vào một số: 0 Số lớn nhất là: 45 —	Chạy lại chương trình và thử lại với số liệu khác. Thực hiện một số thay đổi sau: - Di chuyển dòng int a[MAX]; lên sau dòng số 10 - Sửa dòng int max(int, int); thành int max(int); - Sửa dòng int max(int a[], int n); thành int max(int n); - Sửa dòng num = max(a, i); thành num = max(i); Chạy lại chương trình, quan sát, nhận xét và đánh giá kết quả.
---	---

→ **Giải thích chương trình**

Chương trình ban đầu hàm max có hai tham số truyền vào và kết quả trả về là giá trị max có kiểu nguyên, một tham số là mảng 1 chiều kiểu int và một tham số có kiểu int. Với chương trình sau khi sửa hàm max chỉ còn một tham số truyền vào nhưng cho kết quả như nhau. Do sau khi sửa chương trình mảng a[MAX] được khai báo lại là biến toàn cục nên hàm max không cần truyền tham số mảng vào cũng có thể sử dụng được. Tuy vậy, khi lập trình bạn nên viết như chương trình ban đầu là truyền tham số mảng vào (dạng tổng quát) để hàm max có thể thực hiện được trên nhiều mảng khác nhau. Còn với chương trình sửa lại bạn chỉ sử dụng hàm max được với mảng a mà thôi.

Ví dụ 9: Bạn khai báo các mảng sau ia[MAX], ib[MAX], ic[MAX]. Để tìm giá trị lớn nhất của từng mảng. Bạn chỉ cần gọi hàm

```
- imax_a = max(ia, i);  
- imax_b = max(ib, i);  
- imax_c = max(ic, i);
```

Với chương trình sửa lại bạn không thể tìm được số lớn nhất của mảng b và c.

→Bạn lưu ý rằng khi truyền mảng sang hàm, không tạo bản sao mảng mới. Vì vậy mảng truyền sang hàm có dạng tham biến. Nghĩa là giá trị của các phần tử trong mảng sẽ bị ảnh hưởng nếu có sự thay đổi trên chúng.

Ví dụ 10 : Tìm số lớn nhất của 3 mảng a, b, c

```
1  /* Chuong trình tim so lon nhat su dung ham */  
2  
3  #include <stdio.h>  
4  #include <conio.h>  
5  
6  #define MAX  20  
7  
8  //Khai bao prototype int  
9  max(int, int);  
10 int input(int);  
11  
12 //ham tim so lon nhat trong mang 1 chieu  
13 int max(int ia[], int in)  
14 {  
15     int i, imax;  
16     imax = ia[0];           //cho phan tu dau tien la max  
17     for (i = 1; i < in; i++)  
18         if (imax < ia[i])    //neu so dang xet > max  
19             imax = ia[i];    //gan so nay cho max  
20     return imax;           //tra ve ket qua so lon nhat  
21 }  
22  
23 //ham nhap lieu vao mang 1 chieu int  
24 input(int ia[])  
25 {  
26     int i = 0;  
27     do  
28     {  
29         printf("Nhap vao mot so: ");
```

```

30     scanf("%d", &ia[i]);
31     } while (ia[i++] != 0);
32     i--;
33     return i;
34 }
35
36 int main()
37 {
38     int ia[MAX], ib[MAX], ic[MAX];
39     int inum1, inum2, inum3;
40     printf("Nhap lieu cho mang a: \n");
41     inum1 = max(ia, input(ia));
42     printf("Nhap lieu cho mang b: \n");
43     inum2 = max(ib, input(ib));
44     printf("Nhap lieu cho mang c: \n");
45     inum3 = max(ic, input(ic));
46     printf("So lon nhat cua mang a: %d, b: %d, c: %d.\n", inum1, inum2, inum3);
47     return 0;
48 }

```

→ **Kết quả in ra màn hình**

Nhap lieu cho mang a:	Nhap lieu cho mang c: Nhap
Nhap vao mot so: 12	vao mot so: 1
Nhap vao mot so: 45	Nhap vao mot so: 5
Nhap vao mot so: 3	Nhap vao mot so: 4
Nhap vao mot so: 0	Nhap vao mot so: 0
Nhap lieu cho mang b:	So lon nhat cua mang a: 45, b: 15, c: 5.
Nhap vao mot so: 5	—
Nhap vao mot so: 15	
Nhap vao mot so:	

Chạy lại chương trình và thử lại với số liệu khác.

Viết thêm hàm tìm số nhỏ nhất.

// Giải thích chương trình

Hàm input có kiểu trả về là int thông qua biến i (cho biết số lượng phần tử đã nhập vào) và 1 tham số là mảng 1 chiều kiểu int. Dòng 41, 43, 45 lần lượt gọi hàm input với các tham số là mảng a, b, c. Khi hàm input thực hiện việc nhập liệu thì các phần tử trong mảng cũng được cập nhật theo.

8.1.2. Mảng nhiều chiều

Mảng nhiều chiều là mảng có từ 2 chiều trở lên. Điều đó có nghĩa là mỗi phần tử của mảng là một mảng khác. Người ta thường sử dụng mảng nhiều chiều để lưu các ma trận, các tọa độ 2 chiều, 3 chiều...

8.1.2.1. Khai báo

Kiểu Tên_mảng [Số_phần_tử_chiều 1][Số_phần_tử_chiều 2];

Ví dụ 11: Người ta cần lưu trữ thông tin của một ma trận gồm các số thực. Lúc này ta có thể khai báo một mảng 2 chiều như sau:

```
float m[8][9]; /* Khai báo mảng 2 chiều có 8*9 phần tử là số thực*/
```

Trong trường hợp này, ta đã khai báo cho một ma trận có tối đa là 8 dòng, mỗi dòng có tối đa là 9 cột.

	0	1	2	3	4	5	6	7	8
0	m[0][0]	m[0][1]	m[0][2]						
1	m[1][0]								
2	m[2][0]		m[2][2]						
3				m[3][3]					
4									
5									
6									
7									m[7][8]

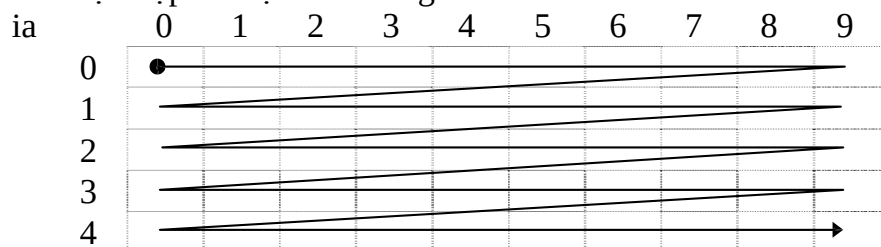
8.1.2.2. Tham chiếu đến từng phần tử của mảng 2 chiều

Sau khi được khai báo, mỗi phần tử trong mảng 2 chiều đều có 2 chỉ số để tham chiếu, chỉ số hàng và chỉ số cột. Chỉ số hàng bắt đầu từ 0 đến số hàng – 1 và chỉ số cột bắt đầu từ 0 đến số cột – 1. Tham chiếu đến một phần tử trong mảng 2 chiều m: m[chỉ số hàng][chỉ số cột] m[3][2] là phần tử tại hàng 3 cột 2 trong mảng 2 chiều xem như là một biến kiểu float.

8.1.2.3. Nhập dữ liệu cho mảng 2 chiều

```
for (i = 0; i < 5; i++) //vòng for có giá trị i chạy từ 0 đến 4 cho
    hàng for (ij = 0; ij < 10; ij++) //vòng for có giá trị ij chạy từ 0 đến 9
        cho cột
        {
            printf("Nhập vào phần tử ia[%d][%d]: ", i + 1, ij + 1);
            scanf("%d", &ia[i][ij]);
        }
```

* Thứ tự nhập dữ liệu vào mảng 2 chiều



8.1.2.4. Đọc dữ liệu từ mảng 2 chiều

Ví dụ 12 : in giá trị các phần tử mảng 2 chiều ra màn hình.

```
for (i = 0; i < 5; i++) //vòng for có giá trị i chạy từ 0 đến 4 cho
    hàng
    {
        for (ij = 0; ij < 10; ij++) //vòng for có giá trị ij chạy từ 0 đến 9 cho
            cột printf("%3d ", ia[i][ij]);
```

```
    printf("\n");           //xuống dòng để in hàng kế tiếp
}
```

Ví dụ 13 : Viết chương trình nhập vào 1 ma trận số nguyên $n \times n$. In ra ma trận vừa nhập vào và ma trận theo thứ tự ngược lại.

```

1  /* nhap va in ma tran */
2
3  #include <stdio.h>
4  #include <conio.h>
5  #define MAX  50
6
7  int main()
8  {
9      int ia[MAX][MAX], i, ij,
10     in; printf("Nhap vao cap
11     ma tran: "); scanf("%d",
12     &in);
13
14     //Nhap du lieu vao ma tran
15     for (i = 0; i < in; i++)      //vòng for có giá trị i chạy từ 0 đến in-1 cho hàng
16         for (ij = 0; ij < in; ij++) //vòng for có giá trị ij chạy từ 0 đến in-1 cho cột
17         {
18             printf("Nhap vao phan tu ia[%d][%d]: ", i + 1, ij + 1);
19             scanf("%d", &ia[i][ij]);
20         }
21
22     //In ma tran
23     for (i = 0; i < in; i++)      //vòng for có giá trị i chạy từ 0 đến in-1 cho hàng
24     {
25         for (ij = 0; ij < in; ij++) //vòng for có giá trị ij chạy từ 0 đến in-1 cho cột
26             printf("%3d ", ia[i][ij]);
27         printf("\n");           //xuống dòng để in hàng kế tiếp
28     }
29     printf("\n");               //Tạo khoảng cách giữa 2 ma tran
30
31     //In ma tran theo thu tu nguoc
32     for (i = in-1; i >= 0; i--)   //vòng for có giá trị i chạy từ in-1 đến 0 cho hàng
33     {
34         //vòng for có giá trị ij chạy từ in-1 đến 0 cho cột
35         for (ij = in-1; ij >= 0 ; ij--) printf("%3d ", ia[i][ij]);
36         printf("\n");           //xuống dòng để in hàng kế tiếp
37     }
38     return 0;
39 }
```

→ **Kết quả in ra màn hình**

Nhap vao cap ma tran: 2	Chạy và thử lại chương trình
Nhap vao phan tu ia[1][1]: 7	với $n = 3, 5$. Quan sát kết
Nhap vao phan tu ia[1][2]: 4	quả.

Nhập vào phần tử <code>ia[2][1]</code> : 6 Nhập vào phần tử <code>ia[2][2]</code> : 15 7 4 6 15 15 6 4 7	- Sửa lại chương trình trên cho phép nhập vào ma trận $m \times n$. Nghĩa là ma trận có m hàng và n cột. Bạn sửa lại chương trình bằng cách cho nhập vào giá trị m và n và sửa lại vòng for cho hàng chạy m lần và vòng for cho cột chạy n lần.
---	--

8.1.2.5. Khởi tạo mảng 2 chiều

Ví dụ 14 : Vẽ chữ H lớn.

1	/* Chương trình vẽ chữ H lớn */
2	
3	#include <stdio.h>
4	#include <conio.h>
5	#define MAX 5
6	
7	int H[MAX][MAX] = {{1, 0, 0, 0, 1},
8	{1, 0, 0, 0, 1},
9	{1, 1, 1, 1, 1},
10	{1, 0, 0, 0, 1},
11	{1, 0, 0, 0, 1}};
12	int main()
13	{
14	int i, ij;
15	for (i = 0; i < MAX; i++)
16	{
17	for (ij = 0; ij < MAX; ij++)
18	if (H[i][ij])
19	printf("!");
20	else
21	printf(" ");
22	printf("\n");
23	}
24	return 0;
25	}

→ **Kết quả in ra màn hình**

! ! ! ! !!!!! ! ! ! ! _	Bạn sửa lại chương trình để in ra chữ C, B...
--	---

9.1.2.6 Dùng mảng 2 chiều làm tham số cho hàm

Ví dụ 15 : Nhập vào 2 ma trận vuông cấp n số thập phân. Cộng 2 ma trận này lưu vào ma trận thứ 3 và tìm số lớn nhất trên ma trận thứ 3.

1	/* cộng ma trận */
2	
3	#include <stdio.h>

```

4  #include <conio.h>
5
6  #define MAX  20
7
8  //Khai bao prototype
9  void input(float);
10 void output(float);
11 void add(float, float, float);
12 float max(float);
13
14 //khai bao bien toan cuc
15 int in;
16
17 //ham tim so lon nhat trong mang 2 chieu
18 float max(float fa[][MAX])
19 {
20     float fmax;
21     fmax = fa[0][0];           //cho phan tu dau tien la max
22     for (int i = 0; i < in; i++)
23         for (int ij = 0; ij < in; ij++)
24             if (fmax < fa[i][ij]) //neu so dang xet > max
25                 fmax = fa[i][ij]; //gan so nay cho max
26     return fmax;               //tra ve ket qua so lon nhat
27 }
28
29 //ham nhap lieu mang 2 chieu
30 void input(float fa[][MAX])
31 {
32     for (int i = 0; i < in; i++)
33         for (int ij = 0; ij < in; ij++)
34             {
35                 printf("Nhap vao ptu[%d][%d]: ", i, ij);
36                 scanf("%f", &fa[i][ij]);
37             }
38 }
39 //ham in mang 2 chieu ra man hinh
40 void output(float fa[][MAX])
41 {
42     for (int i = 0; i < in; i++)
43     {
44         for (int ij = 0; ij < in; ij++)
45             printf("%5.2f", fa[i][ij]);
46         printf("\n");
47     }
48 }
49
50 //ham cong 2 mang 2 chieu
51 void add(float fa[][MAX], float fb[][MAX], float fc[][MAX])
52 {
53     for (int i = 0; i < in; i++)
54         for (int ij = 0; ij < in; ij++)
55             fc[i][ij] = fa[i][ij] + fb[i][ij];
56 }

```

```

56
57 int main()
58 {
59     float fa[MAX][MAX], fb[MAX][MAX], fc[MAX][MAX];
60     printf("Nhap vao cap ma tran: ");
61     scanf("%d", &in);
62     printf("Nhap lieu ma tran a: \n");
63     input(fa);
64     printf("Nhap lieu ma tran b: \n");
65     input(fb);
66     add(fa, fb, fc);
67     printf("Ma tran a: \n");
68     output(fa);
69     printf("Ma tran b: \n");
70     output(fb);
71     printf("Ma tran c: \n");
72     output(fc);
73     printf("So lon nhat cua ma tran c la: %5.2f.\n", max(fc));
74     return 0;
75 }
76

```

→ **Kết quả in ra màn hình**

Nhap vao cap ma tran : 2	Ma tran a:
Nhap lieu ma tran a: Nhap	5.20 4.00
vao ptu[0][0] : 5.2	7.10 9.00
Nhap vao ptu[0][1] : 4	Ma tran b:
Nhap vao ptu[1][0] : 7.1	12.00 3.40
Nhap vao ptu[1][1] : 9	9.60 11.00
Nhap lieu ma tran b: Nhap	Ma tran c:
vao ptu[0][0] : 12	17.20 7.40
Nhap vao ptu[0][1] : 3.4	16.70 20.00
Nhap vao ptu[1][0] : 9.6	So lon nhat cua ma tran c la: 20.00
Nhap vao ptu[1][1] : 11	_

→ **Giải thích chương trình**

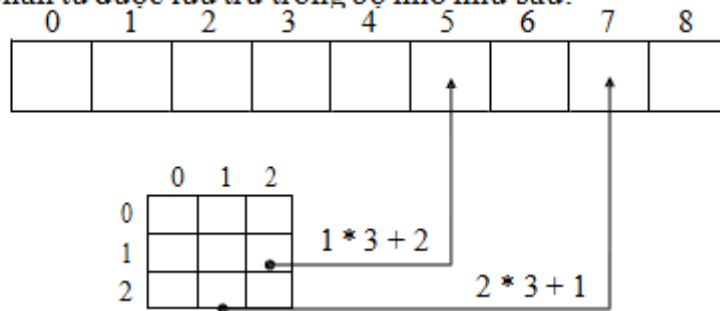
Trong chương trình khai báo biến in toàn cục do biến này sử dụng trong suốt quá trình chạy chương trình. Tham số truyền vào hàm là mảng hai chiều dưới dạng `a[][MAX]` vì hàm không dành chỗ cho mảng, hàm chỉ cần biết số cột để tham khảo đến các phần tử.

Ví dụ 16 : Mảng 2 chiều được khai báo `int ia[3][3]`

Truyền tham số vào hàm: `ia[][3]`,
để tham khảo đến `ptu[2][1]`,
hàm tính như sau:

$$2 * 3 + 1 = 7 \text{ (chỉ số hàng * số cột + chỉ số cột)}$$

ia[3][3] gồm 9 phần tử được lưu trữ trong bộ nhớ như sau:



→ Giống như mảng 1 chiều khi truyền mảng 2 chiều sang hàm cũng không tạo bản sao mới.

8.2. CHUỖI KÝ TỰ

8.2.1. Khai báo và khởi tạo chuỗi

Chuỗi ký tự là một dãy gồm các ký tự hoặc một mảng các ký tự được kết thúc bằng ký tự '\0' (còn được gọi là ký tự NULL trong bảng mã Ascii).

Các hằng chuỗi ký tự được đặt trong cặp dấu nháy kép "".

8.2.1.1. Khai báo theo mảng

Cú pháp: **char <Biến> [Chiều dài tối đa]**

Ví dụ 17: Trong chương trình, ta có khai báo: char Ten[12];

Trong khai báo này, bộ nhớ sẽ cung cấp 12+1 bytes để lưu trữ nội dung của chuỗi ký tự Ten; byte cuối cùng lưu trữ ký tự '\0' để chấm dứt chuỗi.

Ghi chú:

- Chiều dài tối đa của biến chuỗi là một hằng nguyên nằm trong khoảng từ 1 đến 255 bytes.
- Chiều dài tối đa không nên khai báo thừa để tránh lãng phí bộ nhớ, nhưng cũng không nên khai báo thiếu.

Ví dụ 18: Nhập vào in ra tên

1	/* Chuong trinh nhap va in ra ten*/
2	
3	#include <stdio.h>
4	#include <conio.h>
5	
6	int main()
7	{
8	char cname[30];
9	printf("Cho biet ten cua ban: ");
10	scanf("%s", cname);
11	printf("Chao ban %s\n", cname);

12	return 0;
13	}

→ **Kết quả in ra màn hình**

Cho biet ten cua ban: Minh Chao ban Minh –	Chạy lại chương trình và thử nhập tên: Mai Lan, Thanh Nhi Quan sát kết quả.
--	---

→ **Lưu ý: không cần sử dụng toán tử địa chỉ & trong cname trong lệnh scanf("%s", fname), vì bản thân fname đã là địa chỉ.**

Dùng hàm scanf để nhập chuỗi có hạn chế như sau: Khi bạn thử lại chương trình trên với dữ liệu nhập vào là Mai Lan, nhưng khi in ra bạn chỉ nhận được Mai. Vì hàm scanf nhận vào dữ liệu đến khi gặp khoảng trắng thì kết thúc.

8.2.1.2. Khai báo theo con trỏ

Cú pháp: **char *<Biến>**

Ví dụ: Trong chương trình, ta có khai báo:

```
char
*Ten;
```

Trong khai báo này, bộ nhớ sẽ dành 2 byte để lưu trữ địa chỉ của biến con trỏ Ten đang chỉ đến, chưa cung cấp nơi để lưu trữ dữ liệu. Muốn có chỗ để lưu trữ dữ liệu, ta phải gọi đến hàm malloc() hoặc calloc() có trong “alloc.h”, sau đó mới gán dữ liệu cho biến

8.2.1.3. Vừa khai báo vừa gán giá trị

1	/* Chuong trinh nhap va in ra ten*/
2	
3	#include <stdio.h>
4	#include <conio.h>
5	
6	int main()
7	{
8	char cname[30];
9	char chao[] = "Chao ban";
10	printf("Cho biet ten cua ban: ");
11	gets(cname);
12	printf("%s %s.\n", chao, cname);
13	return 0;
14	}

→ **Kết quả in ra màn hình**

Cho biet ten cua ban: Mai Lan Chao ban Mai Lan –	Chạy lại chương trình vào thử nhập tên: Doan Trang Quan sát kết quả.
--	---

*** Ghi chú:** Chuỗi được khai báo là một mảng các ký tự nên các thao tác trên mảng có thể áp dụng đối với chuỗi ký tự.

8.2.2. Nhập / xuất chuỗi

8.2.2.1. Nhập chuỗi từ bàn phím

Để nhập một chuỗi ký tự từ bàn phím, ta sử dụng hàm gets()

Cú pháp: **gets(<Biến chuỗi>)**

Ví dụ: char Ten[20];
gets(Ten
);

Ta cũng có thể sử dụng hàm scanf() để nhập dữ liệu cho biến chuỗi, tuy nhiên lúc này ta chỉ có thể nhập được một chuỗi không có dấu khoảng trắng.

Ngoài ra, hàm cgets() (trong conio.h) cũng được sử dụng để nhập chuỗi.

8.2.2.2. Xuất chuỗi lên màn hình

Để xuất một chuỗi (biểu thức chuỗi) lên màn hình, ta sử dụng hàm puts().

Cú pháp: **puts(<Biểu thức chuỗi>)**

Ví dụ: Nhập vào một chuỗi và hiển thị trên màn hình chuỗi vừa nhập.

1	/* Chương trình nhập và in ra tên */
2	
3	#include <stdio.h>
4	#include <conio.h>
5	
6	int main()
7	{
8	char cname[30];
9	puts("Cho biết tên của bạn: ");
10	gets(cname);
11	puts("Chào bạn ");
12	puts(cname);
13	return 0;
14	}

→ **Kết quả in ra màn hình**

Cho biết tên của bạn:	Sửa dòng 9 thành printf("Cho biết tên của bạn: "); và từ dòng 11 đến 12 thành printf("Chào bạn %s.\n", cname);
Mai Lan	
Chào bạn	Chạy lại chương trình vào thử nhập tên: Tuan Anh, Thanh Lan
Mai Lan	
—	Quan sát kết quả.

8.2.3. Một số hàm xử lý chuỗi

Các hàm về xử lý chuỗi có trong thư viện string.h

8.2.3.1. Cộng chuỗi - Hàm strcat()

Cú pháp: **char *strcat(char *des, const char *source)**

Hàm này có tác dụng ghép chuỗi nguồn vào chuỗi đích.

Ví dụ 19: Nhập vào họ lót và tên của một người, sau đó in cả họ và tên của họ lên màn hình.

```
#include<conio.h>
#include<stdio.h>
#include<string.h>
int main()
{
    char HoLot[30], Ten[12];
    printf("Nhap Ho Lot: ");
    gets(HoLot);
    printf("Nhap Ten: ");
    gets(Ten);
    strcat(HoLot,Ten); /* Ghep Ten vao HoLot*/
    printf("Ho ten la: ");puts(HoLot);
    return 0;
}
```

8.2.3.2. Xác định độ dài chuỗi - Hàm strlen()

Cú pháp: **int strlen(const char* s)**

Ví dụ 20: Sử dụng hàm strlen xác định độ dài một chuỗi nhập từ bàn phím.

```
#include<conio.h>
#include<stdio.h>
#include<string.h>

int main()
{
    char Chuoi[255];
    int Dodai;
    printf("Nhap chuoi: ");
    gets(Chuoi);
    Dodai = strlen(Chuoi);
    printf("Chuoi vua nhap: ");
    puts(Chuoi);
    printf("Co do dai %d",Dodai);
    return 0;
}
```

8.2.3.3. Đổi một ký tự thường thành ký tự hoa - Hàm toupper()

Hàm toupper() (trong ctype.h) được dùng để chuyển đổi một ký tự thường thành ký tự hoa.

Cú pháp: **char toupper(char c)**

8.2.3.4. Đổi chuỗi chữ thường thành chuỗi chữ hoa, hàm strupr()

Hàm strupr() được dùng để chuyển đổi chuỗi chữ thường thành chuỗi chữ hoa, kết quả trả về của hàm là một con trỏ chỉ đến địa chỉ chuỗi được chuyển đổi.

Cú pháp: **char *strupr(char *s)**

Ví dụ 21: Viết chương trình nhập vào một chuỗi ký tự từ bàn phím. Sau đó sử dụng hàm strupr() để chuyển đổi chúng thành chuỗi chữ hoa.

```
#include<conio.h>
#include<stdio.h>
#include<string.h>
```

```

int main()
{
    char Chuoi[255], *s;
    printf("Nhap chuoi: "); gets(Chuoi);
    s=strupr(Chuoi) ;
    printf("Chuoi chu hoa: "); puts(s);
    return 0;
}

```

8.2.3.5. Đổi chuỗi chữ hoa thành chuỗi chữ thường, hàm `strlwr()`

Muốn chuyển đổi chuỗi chữ hoa thành chuỗi toàn chữ thường, ta sử dụng hàm `strlwr()`, các tham số của hàm tương tự như hàm `strupr()`

Cú pháp: **char *strlwr(char *s)**

8.2.3.6. Sao chép chuỗi, hàm `strcpy()`

Hàm này được dùng để sao chép toàn bộ nội dung của chuỗi nguồn vào chuỗi đích.

Cú pháp: **char *strcpy(char *Des, const char *Source)**

Ví dụ 22 : Viết chương trình cho phép chép toàn bộ chuỗi nguồn vào chuỗi

đích.

```

#include<conio.h>
#include<stdio.h>
#include<string.h>
int main()
{
    char Chuoi[255], s[255];
    printf("Nhap chuoi: "); gets(Chuoi);
    strcpy(s, Chuoi);
    printf("Chuoi dich: "); puts(s);
    return 0;
}

```

8.2.3.7. Sao chép một phần chuỗi, hàm `strncpy()`

Hàm này cho phép chép n ký tự đầu tiên của chuỗi nguồn sang chuỗi đích.

Cú pháp: **char *strncpy(char *Des, const char *Source, size_t n)**

8.2.3.8. Trích một phần chuỗi, hàm `strchr()`

Để trích một chuỗi con của một chuỗi ký tự bắt đầu từ một ký tự được chỉ định trong chuỗi cho đến hết chuỗi, ta sử dụng hàm `strchr()`.

Cú pháp : **char *strchr(const char *str, int c)**

Ghi chú:

- Nếu ký tự đã chỉ định không có trong chuỗi, kết quả trả về là NULL.
- Kết quả trả về của hàm là một con trỏ, con trỏ này chỉ đến ký tự c được tìm thấy đầu tiên trong chuỗi str.

8.2.3.9. Tìm kiếm nội dung chuỗi, hàm `strstr()`

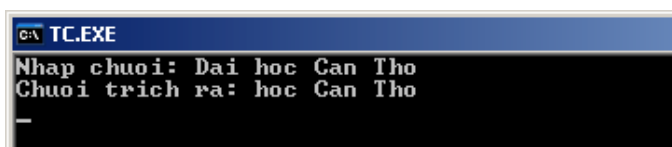
Hàm `strstr()` được sử dụng để tìm kiếm sự xuất hiện đầu tiên của chuỗi s2 trong chuỗi s1.

Cú pháp: **char *strstr(const char *s1, const char *s2)**

Kết quả trả về của hàm là một con trỏ chỉ đến phần tử đầu tiên của chuỗi s1 có chứa chuỗi s2 hoặc giá trị NULL nếu chuỗi s2 không có trong chuỗi s1.

Ví dụ 23: Viết chương trình sử dụng hàm strstr() để lấy ra một phần của chuỗi gốc bắt đầu từ chuỗi “hoc”.

```
#include<conio.h>
#include<stdio.h>
#include<string.h>
int main()
{
    char Chuoi[255], *s;
    printf("Nhap chuoi: "); gets(Chuoi);
    s=strstr(Chuoi, "hoc");
    printf("Chuoi trich ra: "); puts(s);
    return 0;
}
```



8.2.3.10. So sánh chuỗi, hàm strcmp()

Để so sánh hai chuỗi theo từng ký tự trong bảng mã Ascii, ta có thể sử dụng hàm strcmp().

Cú pháp: **int strcmp(const char *s1, const char *s2)**

Hai chuỗi s1 và s2 được so sánh với nhau, kết quả trả về là một số nguyên (số này có được bằng cách lấy ký tự của s1 trừ ký tự của s2 tại vị trí đầu tiên xảy ra sự khác nhau).

- Nếu kết quả là số âm, chuỗi s1 nhỏ hơn chuỗi s2.
- Nếu kết quả là 0, hai chuỗi bằng nhau.
- Nếu kết quả là số dương, chuỗi s1 lớn hơn chuỗi s2.

8.2.3.11. So sánh chuỗi, hàm stricmp()

Hàm này thực hiện việc so sánh trong n ký tự đầu tiên của 2 chuỗi s1 và s2, giữa chữ thường và chữ hoa không phân biệt.

Cú pháp: **int stricmp(const char *s1, const char *s2)**

Kết quả trả về tương tự như kết quả trả về của hàm strcmp()

8.2.3.12. Khởi tạo chuỗi, hàm memset()

Hàm này được sử dụng để đặt n ký tự đầu tiên của chuỗi là ký tự c.

Cú pháp: **memset(char *Des, int c, size_t n)**

8.2.3.13. Đổi từ chuỗi ra số, hàm atoi(), atof(), atol() (trong stdlib.h)

Để chuyển đổi chuỗi ra số, ta sử dụng các hàm trên.

Cú pháp : **int atoi(const char *s)** : chuyển chuỗi thành số nguyên

long atol(const char *s): chuyển chuỗi thành số nguyên dài

float atof(const char *s): chuyển chuỗi thành số thực

Nếu chuyển đổi không thành công, kết quả trả về của các hàm là 0.

Ngoài ra, thư viện *string.h* còn hỗ trợ các hàm xử lý chuỗi khác, ta có thể đọc thêm trong phần trợ giúp

8.2.4. Ví dụ về xử lý chuỗi

```
1  /* Nhập danh sách tên và sắp xếp theo thu tu tăng dần*/
2
3  #include <stdio.h>
4  #include <conio.h>
5  #include <string.h>
6
7  #define MAXNUM 5
8  #define MAXLEN 10
9
10 int main()
11 {
12     char cname[MAXNUM][MAXLEN]; //mang chuỗi
13     char *cptr[MAXNUM];          //mang con trỏ trỏ đến chuỗi
14     char *ctemp;
15     int i, ij, icount = 0;
16
17     //nhập danh sách tên
18     while (icount < MAXNUM)
19     {
20         printf("Nhập vào tên người thứ %d: ", icount + 1);
21         gets(cname[icount]);
22         cptr[icount++] = cname[icount]; //con trỏ đến tên
23     }
24
25     //sắp xếp danh sách theo thu tu tăng dần
26     for (i = 0; i < icount - 1; i++)
27         for (ij = i + 1; ij < icount; ij++)
28             if (strcmp(cptr[i], cptr[ij]) > 0)
29             {
30                 ctemp = cptr[i];
31                 cptr[i] = cptr[ij];
32                 cptr[ij] = ctemp;
33             }
34
35     //In danh sách đã sắp xếp
36     printf("Danh sách sau khi sắp xếp:\n");
37     for (i = 0; i < icount; i++)
38         printf("Tên người thứ %d : %s\n", i + 1, cptr[i]);
39     return 0;
40 }
```

→ **Kết quả in ra màn hình**

Nhập vào ten người thu 1: Minh Nhập vào ten người thu 2: Lan Nhập vào ten người thu 3: Anh Nhập vào ten người thu 4: Trang Nhập vào ten người thu 5: Quan Danh sách sau khi sắp xếp: Ten người thu 1: Anh Ten người thu 2: Lan Ten người thu 3: Minh Ten người thu 4: Quan Ten người thu 5: Trang	Chạy lại chương trình vào thử nhập vào các tháng khác Quan sát kết quả.
---	--

→ **Giải thích chương trình**

Trong chương trình dùng cả mảng chuỗi char cname[MAXNUM][MAXLEN] và mảng con trỏ trỏ đến chuỗi char *cptr[MAXNUM];.

cptr[0]	1010	→	1010	M	i	n	h	\0	
cptr[1]	1016	→	1016	L	a	n	\0		
cptr[2]	1022	→	1022	A	n	h	\0		
cptr[3]	1028	→	1028	T	r	a	n	g	\0
cptr[4]	1034	→	1034	Q	u	a	n	\0	

↑ Mảng các con trỏ trỏ đến chuỗi trước khi sắp xếp

cptr[0]	1022	→	1010	M	i	n	h	\0	
cptr[1]	1016	→	1016	L	a	n	\0		
cptr[2]	1010	→	1022	A	n	h	\0		
cptr[3]	1034	→	1028	T	r	a	n	g	\0
cptr[4]	1028	→	1034	Q	u	a	n	\0	

↑ Mảng các con trỏ trỏ đến chuỗi sau khi sắp xếp

CÂU HỎI VÀ BÀI TẬP THỰC HÀNH

- Viết hàm tìm số lớn nhất, nhỏ nhất trong một mảng n số nguyên.
- Viết hàm sắp xếp tăng dần, giảm dần của một dãy số cho trước.
- Viết hàm tách tên và họ lót từ một chuỗi cho trước.
- Viết hàm cắt bỏ khoảng trắng thừa ở giữa, hai đầu.
- Viết hàm chuyển đổi 1 chuỗi sang chữ thường và 1 hàm chuyển đổi sang chữ HOA.
- Viết hàm chuyển đổi 1 chuỗi sang dạng Title Case (kí tự đầu của mỗi từ là chữ HOA, các kí tự còn lại chữ thường)
- Viết chương trình nhập vào 1 chuỗi và in ra chuỗi đảo ngược.
Ví dụ: Nhập vào chuỗi "Lap trinh C can ban" In ra "nab nac C hnirt paL"
- Viết chương trình nhập vào một chuỗi ký tự rồi đếm xem trong chuỗi đó có bao nhiêu chữ 'th'.
- Biết rằng năm 0 là năm Canh thân (năm ky nhau có chu kì là 3, năm hợp nhau có chu kì là 4). Hãy viết chương trình cho phép gõ vào năm dương lịch (ví dụ

1997), xuất ra năm âm lịch (Đinh Sửu) và các năm kỷ và hợp.

Có 12 chi: Tý, Sửu, Dần, Mão, Thìn, Ty, Ngọ, Mùi, Thân, Dậu, Tuất, Hợi.

Có 10 can: Giáp, Ất, Bính, Đinh, Mậu, Kỷ, Canh, Tân, Nhâm, Quý.

10. Viết chương trình nhập vào 3 chữ số (305, 6, 28). Cho biết dòng chữ mô tả giá trị con số đó. Ví dụ 305 -> ba trăm lẻ năm.

11. Viết chương trình nhập vào một chuỗi sau đó in ra màn hình mỗi dòng là một từ. Ví dụ chuỗi "Lap trinh C". Kết quả in ra

Lap
trinh
C

12. Viết chương trình nhập vào một chuỗi các kí tự, ký số, khoảng trắng và dấu chấm câu. Cho biết chuỗi trên gồm bao nhiêu từ.

13. Viết chương trình nhập vào một chuỗi ký tự. Kiểm tra xem chuỗi đó có đối xứng không?

14. Viết chương trình nhập vào một chuỗi gồm các chữ cái (a -> z, A -> Z). Hãy đếm xem có bao nhiêu nguyên âm a, i, e, o, u.

15. Giả sử số phòng trong một khách sạn được cho bởi hằng số NUM_ROOM.
Viết:

- Một khai báo dãy thích hợp để theo dõi phòng nào còn trống.
- Một hàm tìm phòng nào còn trống.
- Viết chương trình đơn giản để quản lý phòng khách sạn theo dạng một trình đơn chọn công việc gồm có 4 mục như sau:
 - Tìm phòng trống.
 - Trả phòng.
 - Liệt kê những phòng còn trống.
 - Liệt kê những phòng đã thuê.
 - Kết thúc.

16. Viết chương trình mô tả văn bản của một bức điện tín. Nhập liệu bao gồm 1 hay nhiều dòng chứa một số từ, mỗi từ cách nhau khoảng trắng. In ra hóa đơn tính tiền với mỗi từ giá 100 đồng, phí trả thêm 50 đồng cho từ dài quá 8 kí tự. Hóa đơn có dạng sau:

So tu : 10

So tu co kích thước bình thường : 8 x 100 = 800 đồng

So tu có kích thước > 8 kí tự : 2 x 150 = 300 đồng

Tổng cộng : 1100 đồng

17. Viết chương trình thống kê xem có bao nhiêu người họ "Ly", "Tran"... trong 1 danh sách cho trước. Nếu không có thông báo "Không có người nào thuộc họ".

18. Viết chương trình nhập vào 1 chuỗi, sau đó chép sang chuỗi khác một chuỗi con từ chuỗi ban đầu có số kí tự chỉ định.

Ví dụ: Chuỗi ban đầu "Le Thuy Doan Trang". Nếu số kí tự chỉ định là 2 thì chuỗi đích sẽ là "Le"

19. Viết chương trình nhập vào 1 chuỗi, sau đó chép sang chuỗi khác một chuỗi con từ chuỗi ban đầu với vị trí bắt đầu và số kí tự chỉ định.

Ví dụ: Chuỗi ban đầu "Le Thuy Doan Trang". Nếu vị trí ban đầu là 14 và

số kí tự chỉ định là 5 thì chuỗi đích sẽ là "Trang"

20. Viết chương trình nhập vào 1 chuỗi nguồn, ví dụ "Nguyen Minh Long", sau đó nhập vào 1 chuỗi con, ví dụ "Minh", chương trình sẽ xác định vị trí bắt đầu của chuỗi con ở vị trí nào trong chuỗi nguồn. Kết quả in ra màn hình như sau:

- Chuoi nguon la : Nguyen Minh Long
- Chuoi con la : Minh
- Vi tri bat dau cua chuoi con la : 8

21. Viết chương thực hiện các yêu cầu sau:

- Nhập vào 1 chuỗi bất kỳ, ví dụ : "Nguyen Minh Long
- Muốn xóa từ vị trí nào, ví dụ : 8
- Muốn xóa bao nhiêu kí tự, ví dụ : 5

Kết quả in ra màn hình:

- Chuoi nguon la : Nguyen Minh Long
- Chuoi sau khi xoa : Nguyen Long

Chương 9. CON TRỎ

9.1. KHÁI NIỆM

Các biến chúng ta đã biết và sử dụng trước đây đều là biến có kích thước và kiểu dữ liệu xác định. Người ta gọi các biến kiểu này là biến tĩnh. Khi khai báo biến tĩnh, một lượng ô nhớ cho các biến này sẽ được cấp phát mà không cần biết trong quá trình thực thi chương trình có sử dụng hết lượng ô nhớ này hay không. Mặt khác, các biến tĩnh dạng này sẽ tồn tại trong suốt thời gian thực thi chương trình dù có những biến mà chương trình chỉ sử dụng 1 lần rồi bỏ. Một số hạn chế có thể gặp phải khi sử dụng các biến tĩnh:

- o Cấp phát ô nhớ dư, gây ra lãng phí ô nhớ.
- o Cấp phát ô nhớ thiếu, chương trình thực thi bị lỗi.

Để tránh những hạn chế trên, ngôn ngữ C cung cấp cho ta một loại biến đặc biệt gọi là biến động với các đặc điểm sau:

- o Chỉ phát sinh trong quá trình thực hiện chương trình chứ không phát sinh lúc bắt đầu chương trình.
- o Khi chạy chương trình, kích thước của biến, vùng nhớ và địa chỉ vùng nhớ được cấp phát cho biến có thể thay đổi.
- o Sau khi sử dụng xong có thể giải phóng để tiết kiệm chỗ trong bộ nhớ.

Tuy nhiên các biến động không có địa chỉ nhất định nên ta không thể truy cập đến chúng được. Vì thế, ngôn ngữ C lại cung cấp cho ta một loại biến đặc biệt nữa để khắc phục tình trạng này, đó là biến con trỏ (pointer) với các đặc điểm:

- o Biến con trỏ không chứa dữ liệu mà chỉ chứa địa chỉ của dữ liệu hay chứa địa chỉ của ô nhớ chứa dữ liệu.
- o Kích thước của biến con trỏ không phụ thuộc vào kiểu dữ liệu, luôn có kích thước cố định là 2 byte.

Một **con_trỏ** là một biến, nó chứa địa chỉ vùng nhớ của một biến khác, chứ không lưu trữ giá trị của biến đó. Nếu một biến chứa địa chỉ của một biến khác, thì biến này được gọi là **con_trỏ** đến biến thứ hai kia. Một con trỏ cung cấp phương thức gián tiếp để truy xuất giá trị của các phần tử dữ liệu. Xét hai biến `var1` và `var2`, `var1` có giá trị 500 và được lưu tại địa chỉ 1000 trong bộ nhớ. Nếu `var2` được khai báo như là một con trỏ tới biến `var1`, sự biểu diễn sẽ như sau:

Vị trí Bộ nhớ	Giá trị lưu trữ	Tên biến
1000	500	var1
1001		
1002		
.		
.		
1108	1000	var2

Ở đây, `var2` chứa giá trị 1000, đó là địa chỉ của biến `var1`.

9.2. KHAI BÁO BIẾN CON TRỎ

Cú pháp:

Kiểu *Tên_con_trỏ;

Ví dụ 1:

1	/* Cong hang so */
2	
3	#include <stdio.h>
4	#include <conio.h>
5	int main()
6	{
7	int x = 6, y = 7;
8	int *px, *py;
9	printf("x = %d, y = %d\n", x, y);
10	px = &x;
11	py = &y;
12	*px += 10;
13	*py += 10;
14	printf("x = %d, y = %d\n", x, y);
15	return 0;
16	}

→ **Kết quả in ra màn hình**

x = 6, y = 7 x = 16, y = 17	Chạy chương trình và quan sát kết quả.
--------------------------------	--

→ **Giải thích chương trình**

Khai báo ở dòng 9 cấp phát 2 bytes để lưu giữ địa chỉ của biến nguyên và vùng nhớ đó có tên là px, tương tự cho py. Dấu * cho biết biến này chứa địa chỉ chứ không phải giá trị, int cho biết địa chỉ đó sẽ trỏ đến các biến nguyên.

→ Ví dụ trên cho thấy *px và *py là 2 biến con trỏ trỏ đến địa chỉ của 2 biến x và y (dòng 11 và 12), vì vậy khi nội dung của biến con trỏ *px và *py thay đổi thì nội dung của x, y cũng thay đổi theo.

Địa chỉ vùng nhớ

1203	6	x
1204		
1205		
1206		
1207	7	y
1208		
1209		
1210		

Sau phép gán gán px = &x và py = &y thì giá trị của px = 1203 và py = 1207

Địa chỉ vùng nhớ

1215	6	px
1216		
1217		
1218	7	py
1219		
1220		
1221		
1222		

*px là nội dung của px và *py là nội dung của py. Nên khi thực hiện 2 phép cộng gán *px += 10 và *py += 10 thì giá trị của x sẽ là 16 và giá trị y sẽ là 17.

9.3. TRUYỀN ĐỊA CHỈ SANG HÀM

Ví dụ 2:

1	/* Khoi tao 2 so */
2	
3	#include <stdio.h>
4	#include <conio.h>
5	
6	void init (int, int); //Khai bao nguyen mau
7	
8	void init(int *px, int *py)
9	{
10	*px = 3; //gan 3 cho noi dung cua px
11	*py = 5; //gan 5 cho noi dung cua py
12	}
13	
14	int main()
15	{
16	int x, y;
17	init(&x, &y);
18	printf("x = %d, y = %d\n",x, y);
19	return 0;
20	}

→ **Kết quả in ra màn hình**

x = 3, y = 5	Chạy chương trình và quan sát kết quả.
—	

→ **Giải thích chương trình**

Ở dòng 17, gọi hàm init truyền 2 tham số là địa chỉ của biến x và y, nên khi nội dung của 2 biến con trỏ *px và *py thay đổi thì x và y của chương trình chính cũng thay đổi theo. Hàm main() đã sử dụng cách truy cập biến khác với hàm init, hàm main(void) gọi chúng là x, y còn hàm init gọi chúng là *px, *py. Hàm init đọc giá trị của biến con trỏ *px, *py từ vùng địa chỉ của chương trình gọi, sau khi thực hiện và trả kết quả về chương trình gọi.

9.4. CON TRỎ VÀ MẢNG

9.4.1. Con trỏ và mảng 1 chiều

Giữa mảng và con trỏ có một sự liên hệ rất chặt chẽ. Những phần tử của mảng có thể được xác định bằng chỉ số trong mảng, bên cạnh đó chúng cũng có thể được xác lập qua biến con trỏ.

9.4.1.1. Truy cập các phần tử mảng theo dạng con trỏ

Ta có các quy tắc sau:

&<Tên mảng>[0] tương đương với **<Tên mảng>**

&<Tên mảng> [<Vị trí>] tương đương với **<Tên mảng> + <Vị trí>**

<Tên mảng>[<Vị trí>] tương đương với ***(<Tên mảng> + <Vị trí>)**

Ví dụ 3: Cho 1 mảng 1 chiều các số nguyên a có 5 phần tử, truy cập các phần tử theo kiểu mảng và theo kiểu con trỏ.

1	#include <stdio.h>
2	#include <conio.h>
3	/* Nhập mảng bình thường*/
4	void NhapMang(int a[], int N)
5	{
6	int i;
7	for(i=0;i<N;i++)
8	{
9	printf("Phan tu thu %d: ",i);scanf("%d",&a[i]);
10	}
11	}
12	/* Nhập mảng theo dạng con trỏ*/
13	void NhapContro(int a[], int N)
14	{
15	int i;
16	for(i=0;i<N;i++){
17	printf("Phan tu thu %d: ",i);scanf("%d",a+i);
18	}
19	}
20	int main()
21	{
22	int a[20],N,i;
23	printf("So phan tu N= ");scanf("%d",&N);
24	NhapMang(a,N); /* NhapContro(a,N)*/
25	printf("Truy cap theo kieu mang: ");
26	for(i=0;i<N;i++)
27	printf("%d ",a[i]);
28	printf("\nTruy cap theo kieu con tro: ");
29	for(i=0;i<N;i++)
30	printf("%d ",*(a+i));
31	return 0;
32	}

→ **Kết quả in ra màn hình**

So phần tử N=4	Truy cap theo kieu mang: 2 3 4 5
Phan tu thu 0: 2	Truy cap theo kieu con tro: 2 3 4 5 _
Phan tu thu 1: 3	
Phan tu thu 2: 4	
Phan tu thu 3: 5	

9.4.1.2. Truy xuất từng phần tử đang được quản lý bởi con trỏ theo dạng mảng

<Tên biến>[<Vị trí>] tương đương với *(<Tên biến> + <Vị trí>)

&<Tên biến>[<Vị trí>] tương đương với (<Tên biến> + <Vị trí>)

Trong đó <Tên biến> là biến con trỏ, <Vị trí> là 1 biểu thức số nguyên.

Ví dụ 4: Giả sử có khai báo:

1	#include <stdio.h>
2	#include <malloc.h>
3	#include <conio.h>
4	
5	int main()
6	{
7	int *a;
8	int i;
9	a=(int*)malloc(sizeof(int)*10);
10	for(i=0;i<10;i++)
11	a[i] = 2*i;
12	printf("Truy cap theo kieu mang: ");
13	for(i=0;i<10;i++)
14	printf("%d ",a[i]);
15	printf("\nTruy cap theo kieu con tro: ");
16	for(i=0;i<10;i++)
17	printf("%d ",*(a+i));
18	return 0;
19	}

→ **Kết quả in ra màn hình**

Truy cap theo kieu mang: 0 2 4 6 8 10 12 14 16 18
Truy cap theo kieu con tro: 0 2 4 6 8 10 12 14 16 18 _

	0	1	2	3	4	5	6	7	8	9
	0	2	4	6	8	10	12	14	16	18
a	2 byte									

9.4.1.3. Con trỏ chỉ đến phần tử mảng

Giả sử con trỏ ptr chỉ đến phần tử a[i] nào đó của mảng a thì:

ptr + j chỉ đến phần tử thứ j sau a[i], tức a[i+j]

ptr - j chỉ đến phần tử đứng trước a[i], tức a[i-j]

Ví dụ 5: Giả sử có 1 mảng mang_int, cho con trỏ contro_int chỉ đến phần tử thứ 5 trong mảng. In ra các phần tử của contro_int & mang_int.

```

1  #include <stdio.h>
2  #include <malloc.h>
3  #include <conio.h>
4
5  int main()
6  {
7      int
8      i,mang_int[10];
9      int *contro_int;
10     for(i=0;i<=9;i++)
11         mang_int[i]=i*2;
12     contro_int=&mang_int[5];
13     printf("\nNoi dung cua mang_int ban dau=");
14     for (i=0;i<=9;i++)
15         printf("%d ",mang_int[i]);
16     printf("\nNoi dung cua contro_int ban dau =");
17     for (i=0;i<5;i++)
18         printf("%d ",contro_int[i]);
19     for(i=0;i<5;i++)
20         contro_int[i]++;
21     printf("\n-----");
22     printf("\nNoi dung cua mang_int sau khi tang 1=");
23     for (i=0;i<=9;i++)
24         printf("%d ",mang_int[i]);
25     printf("\nNoi dung cua contro_int sau khi tang 1=");
26     for (i=0;i<5;i++)
27         printf("%d ",contro_int[i]);
28     if (contro_int!=NULL)
29         free(contro_int);
30     return 0;
31 }

```

→ **Kết quả in ra màn hình**

```

Noi dung cua mang_int ban dau= 0 2 4 6 8 10 12 14 16 18
Noi dung cua contro_int ban dau =10 12 14 16 18
-----
Noi dung cua mang_int sau khi tang 1= 0 2 4 6 8 11 13 15 17 19
Noi dung cua contro_int sau khi tang 1=11 13 15 17 19_

```

9.4.2. Con trỏ và mảng nhiều chiều

Ta có thể sử dụng con trỏ thay cho mảng nhiều chiều như sau: Giả sử ta có mảng 2 chiều và biến con trỏ như sau:

```
int a[n][m];
```

```
int *contro_int;
```

Thực hiện phép gán contro_int=a;

Khi đó phần tử $a[0][0]$ được quản lý bởi `contro_int`;
 $a[0][1]$ được quản lý bởi `contro_int+1`;
 $a[0][2]$ được quản lý bởi `contro_int+2`;
 ...
 $a[1][0]$ được quản lý bởi `contro_int+m`;
 $a[1][1]$ được quản lý bởi `contro_int+m+1`;
 ...
 $a[n][m]$ được quản lý bởi `contro_int+n*m`;

Tương tự như thế đối với mảng nhiều hơn 2 chiều.

Ví dụ 6: Sự tương đương giữa mảng 2 chiều và con trỏ.

```

1  #include <stdio.h>
2  #include <malloc.h>
3  #include <conio.h>
4  int main()
5  {
6      int i,j;
7      int mang_int[4][5]={1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20};
8      int *contro_int;
9      contro_int=(int*)mang_int;
10     printf("\nNội dung của mang_int ban đầu=");
11     for (i=0;i<4;i++)
12     {
13         printf("\n");
14         for (j=0;j<5;j++)
15             printf("%d\t",mang_int[i][j]);
16     }
17     printf("\n-----");
18     printf("\nNội dung của contro_int ban đầu \n");
19     for (i=0;i<20;i++)
20         printf("%d ",contro_int[i]);
21
22     for(i=0;i<20;i++) contro_int[i]++ ;
23     printf("\n-----");
24     printf("\nNội dung của mang_int sau khi tang 1=");
25     for (i=0;i<4;i++)
26     {
27         printf("\n");
28         for (j=0;j<5;j++) printf("%d\t",mang_int[i][j]);
29     }
30     printf("\nNội dung của contro_int sau khi tang 1=\n");
31     for (i=0;i<20;i++)
32         printf("%d ",contro_int[i]);
33     if (contro_int!=NULL) free(contro_int);
34     return 0;
35 }
```


→ **Kết quả in ra màn hình**

Nội dung của mảng_int ban đầu=																			
1	2	3	4	5															
6	7	8	9	10															
11	12	13	14	15															
16	17	18	19	20															

Nội dung của mảng_int ban đầu																			
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Nội dung của mảng_int sau khi tăng 1=																			
2		3		4		5		6											
7		8		9		10		11											
12		13		14		15		16											
17		18		19		20		21											
Nội dung của mảng_int sau khi tăng 1=																			
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21

9.4.3. Cấp phát vùng nhớ cho biến con trỏ

Trước khi sử dụng biến con trỏ, ta nên cấp phát vùng nhớ cho biến con trỏ này quản lý địa chỉ. Việc cấp phát được thực hiện nhờ các hàm malloc(), calloc() trong thư viện alloc.h.

Cú pháp các hàm:

void *malloc(size_t size): Cấp phát vùng nhớ có kích thước là size.

void *calloc(size_t nitems, size_t size): Cấp phát vùng nhớ có kích thước là nitems*size.

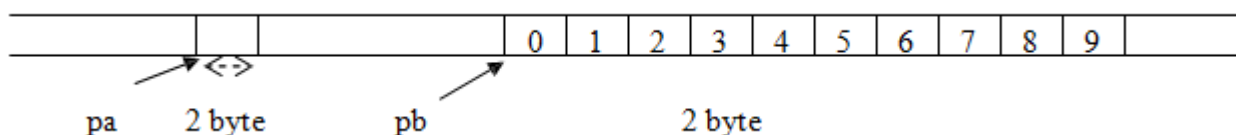
Ví dụ 7: Giả sử ta có khai báo:

```
int a, *pa, *pb;
```

```
pa = (int*)malloc(sizeof(int)); /* Cấp phát vùng nhớ có kích thước
bằng với kích thước của một số nguyên */
```

```
pb = (int*)calloc(10, sizeof(int)); /* Cấp phát vùng nhớ có thể
chứa được 10 số nguyên */
```

Lúc này hình ảnh trong bộ nhớ như sau:



Lưu ý: Khi sử dụng hàm malloc() hay calloc(), ta phải ép kiểu vì nguyên mẫu các hàm này trả về con trỏ kiểu void.

9.4.4 Giải phóng vùng nhớ cho biến con trỏ

Một vùng nhớ đã cấp phát cho biến con trỏ, khi không còn sử dụng nữa, ta sẽ thu hồi lại vùng nhớ này nhờ hàm free().

Cú pháp: `void free(void *block)`

Ý nghĩa: Giải phóng vùng nhớ được quản lý bởi con trỏ block.

Ví dụ 8: Ở ví dụ trên, sau khi thực hiện xong, ta giải phóng vùng nhớ cho 2 biến con trỏ pa & pb:

```
free(pa);
free(pb);
```

9.4.5. Cấp phát lại vùng nhớ cho biến con trỏ

Trong quá trình thao tác trên biến con trỏ, nếu ta cần cấp phát thêm vùng nhớ có kích thước lớn hơn vùng nhớ đã cấp phát, ta sử dụng hàm `realloc()`.

Cú pháp: `void *realloc(void *block, size_t size)`

Ý nghĩa: Cấp phát lại 1 vùng nhớ cho con trỏ *block* quản lý, vùng nhớ này có kích thước mới là *size*; khi cấp phát lại thì nội dung vùng nhớ trước đó vẫn tồn tại.

- Kết quả trả về của hàm là địa chỉ đầu tiên của vùng nhớ mới. Địa chỉ này có thể khác với địa chỉ được chỉ ra khi cấp phát ban đầu.

Ví dụ 9: Trong ví dụ trên ta có thể cấp phát lại vùng nhớ do con trỏ pa quản lý như sau:

```
int a, *pa;
pa=(int*)malloc(sizeof(int)); /*Cấp phát vùng nhớ có kích thước 2 byte*/
pa = realloc(pa, 6); /* Cấp phát lại vùng nhớ có kích thước 6 byte*/
```

9.4.6. Một số phép toán trên con trỏ

a. Phép gán con trỏ: Hai con trỏ cùng kiểu có thể gán cho nhau.

Ví dụ 10:

```
int a, *p, *a ; float *f;
a = 5 ; p = &a ; q = p ; /* đúng */
f = p ; /* sai do khác kiểu */
```

Ta cũng có thể ép kiểu con trỏ theo cú pháp:

(<Kiểu kết quả>*)<Tên con trỏ>

Chẳng hạn, ví dụ trên được viết

lại:

```
int a, *p, *a ; float *f;
a = 5 ; p = &a ; q = p ; /* đúng */
f = (float*)p; /* Đúng nhờ ép kiểu*/
```

b. Cộng, trừ con trỏ với một số nguyên

Ta có thể cộng (+), trừ (-) 1 con trỏ với 1 số nguyên N nào đó; kết quả trả về là 1 con trỏ. Con trỏ này chỉ đến vùng nhớ cách vùng nhớ của con trỏ hiện tại N phân tử.

Ví dụ 11: Cho đoạn chương trình

sau:

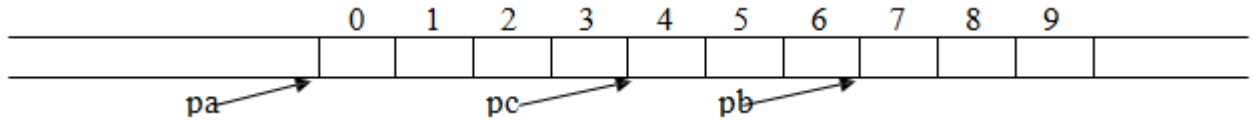
```
int
*pa;
pa = (int*) malloc(20); /* Cấp phát vùng nhớ 20 byte=10 số nguyên*/
```

```

int *pb, *pc;
pb = pa +
7; pc = pb -
3;

```

Lúc này hình ảnh của pa, pb, pc như sau:



c. *Con trỏ NULL*: là con trỏ không chứa địa chỉ nào cả. Ta có thể gán giá trị NULL cho 1 con trỏ có kiểu bất kỳ.

d. *Lưu ý*:

- Ta không thể cộng 2 con trỏ với nhau.
- Phép trừ 2 con trỏ cùng kiểu sẽ trả về 1 giá trị nguyên (int). Đây chính là khoảng cách (số phần tử) giữa 2 con trỏ đó. *Chẳng hạn*, trong ví dụ trên $pc - pa = 4$.

9.5. CON TRỎ VÀ CHUỖI

9.5.1. Khởi tạo chuỗi bằng con trỏ

Ví dụ 12:

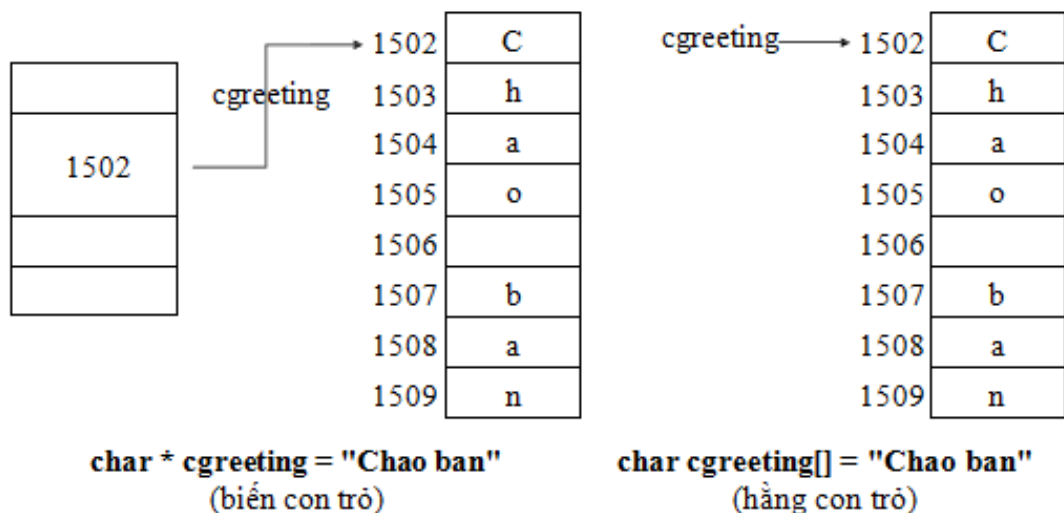
1	/* Chương trình nhập và in ra ten*/
2	
3	#include <stdio.h>
4	#include <conio.h>
5	
6	int main()
7	{
8	char *cgreeting = "Chao ban";
9	char cname[30];
10	puts("Cho biet ten cua ban: ");
11	gets(cname);
12	puts(cgreeting);
13	puts(cname);
14	return 0;
15	}

→ **Kết quả in ra màn hình**

Cho biet ten cua ban: Minh Chao ban Minh	Chạy chương trình và quan sát kết quả.
--	--

→ **Giải thích chương trình**

cgreeting là biến con trỏ được khởi tạo bằng phát biểu `char *cgreeting = "Chao ban"` thay vì `char cgreeting[] = "Chao ban"`. Cả hai cách đều cho cùng kết quả và đều dành số byte cho chuỗi kèm theo kí tự null. Đối với mảng địa chỉ của kí tự đầu tiên của mảng sẽ là tên mảng, còn con trỏ sẽ có thêm biến con trỏ trỏ đến tên cgreeting.



9.5.2. Khởi tạo mảng con trỏ trỏ đến chuỗi

Ví dụ 13:

```

1  /* Chương trình nhập tháng (số) và in ra tháng (chữ) tương ứng */
2
3  #include <stdio.h>
4  #include <conio.h>
5
6  int main()
7  {
8      char *cthang[12] = {"January", "February", "March",
9                          "April", "May", "June", "July",
10                         "August", "September", "October",
11                         "November", "December"};
12
13     int ithang;
14     printf("Nhập vào tháng (1-12): ");
15     scanf("%d", &ithang); printf("%s.\n", cthang[ithang-1]);
16     return 0;
17 }
```

→ **Kết quả in ra màn hình**

Nhập vào tháng (1-12): 2 February	Chạy lại chương trình vào thử nhập vào các tháng khác Quan sát kết quả.
--------------------------------------	--

→ **Giải thích chương trình**

Khai báo `char *cthang[12]` có ý nghĩa như sau: `cthang` là tên gọi, dấu `*` là kiểu con trỏ trỏ đến ký tự (char).

	Địa chỉ	0	1	2	3	4	5	6	7	8	9
cthang[0]	→ 1010	J	a	n	u	a	r	y	\0		
cthang[1]	→ 1020	F	e	b	r	u	a	r	y	\0	
cthang[2]	→ 1030	M	a	r	c	h	\0				
cthang[3]	→ 1040	A	p	r	i	l	\0				
cthang[4]	→ 1050	M	a	y	\0						
cthang[5]	→ 1060	J	u	n	e	\0					
cthang[6]	→ 1070	J	u	l	y	\0					
cthang[7]	→ 1080	A	u	g	u	s	t	\0			
cthang[8]	→ 1090	S	e	p	t	e	m	b	e	r	\0
cthang[9]	→ 1100	O	c	t	o	b	e	r	\0		
cthang[10]	→ 1110	N	o	v	e	m	b	e	r	\0	
cthang[11]	→ 1120	D	e	c	e	m	b	e	r	\0	

Mảng các chuỗi **char cthang[12][10]**

cthang[0]	1010	→ 1010	J	a	n	u	a	r	y	\0	
cthang[1]	1018	→ 1018	F	e	b	r	u	a	r	y	\0
cthang[2]	1027	→ 1027	M	a	r	c	h	\0			
cthang[3]	1033	→ 1033	A	p	r	i	l	\0			
cthang[4]	1039	→ 1039	M	a	y	\0					
cthang[5]	1043	→ 1043	J	u	n	e	\0				
cthang[6]	1048	→ 1048	J	u	l	y	\0				
cthang[7]	1053	→ 1053	A	u	g	u	s	t	\0		
cthang[8]	1060	→ 1060	S	e	p	t	e	m	b	e	r \0
cthang[9]	1070	→ 1070	O	c	t	o	b	e	r	\0	
cthang[10]	1078	→ 1078	N	o	v	e	m	b	e	r	\0
cthang[11]	1087	→ 1087	D	e	c	e	m	b	e	r	\0

Mảng các con trỏ trỏ đến các chuỗi **char *cthang[12]**

→ Khởi tạo mảng các con trỏ trỏ đến các chuỗi chiếm ít bộ nhớ hơn khởi tạo mảng chuỗi.

9.6. CON TRỎ TRỞ ĐẾN CON TRỎ

Ví dụ 14:

```
1  /* In ma trận*/
2
3  #include <stdio.h>
4  #include <conio.h>
5
6  #define ROWS 4
7  #define COLS 5
8
9  int main()
10 {
11     int itable[ROWS][COLS] = {{10, 12, 14, 16, 18},
12                               {11, 13, 15, 17, 19},
13                               {20, 22, 24, 26, 28},
14                               {21, 23, 25, 27, 29}};
15     int i, ij, ix = 10;
16
17     for (i = 0; i < ROWS; i++)
18         for (ij = 0; ij < COLS; ij++)
19             *(*(table + i) + ij) += ix;
20
21     for (i = 0; i < ROWS; i++)
22     {
23         for (ij = 0; ij < COLS; ij++)
24             printf("%4d", *(*(table + i) + ij));
25         printf("\n");
26     }
27     return 0;
28 }
```

→ **Kết quả in ra màn hình**

20 22 24 26 28 21 23 25 27 29 30 32 34 36 38 31 33 35 37 39	Chạy chương trình và quan sát kết quả.
--	--

→ **Giải thích chương trình**

Trong chương trình dùng cả mảng chuỗi char cname[MAXNUM][MAXLEN] và mảng con trỏ trỏ đến chuỗi char *cptr[MAXNUM];.

CÂU HỎI VÀ BÀI TẬP THỰC HÀNH

Làm lại các bài tập ở bài Mảng và chuỗi sử dụng biến con trỏ

.....

MỤC LỤC

Chương 1. NGÔN NGỮ LẬP TRÌNH VÀ PHƯƠNG PHÁP LẬP TRÌNH	1
1.1. NGÔN NGỮ LẬP TRÌNH (Programming Language)	1
1.1.1. Thuật giải (Algorithm)	1
1.1.2. Chương trình (Program)	2
1.1.3. Ngôn ngữ lập trình (Programming language)	2
1.2. CÁC BƯỚC GIẢI BÀI TOÁN TRÊN MÁY TÍNH	2
1.3. KỸ THUẬT LẬP TRÌNH	3
1.3.1. I-P-O Cycle (Input-Process-Output Cycle) (Quy trình nhập-xử lý-xuất)	3
1.3.2. Ngôn ngữ tự nhiên (Ngôn ngữ mẹ đẻ)	4
1.3.3. Lưu đồ - sơ đồ khối	4
1.3.4. Ngôn ngữ lập trình (Mã giả)	6
CÂU HỎI ÔN TẬP CHƯƠNG 1	7
Chương 2. LÀM QUEN LẬP TRÌNH C QUA CÁC VÍ DỤ ĐƠN GIẢN	8
2.1. KHỞI ĐỘNG VÀ THOÁT KHỎI CHƯƠNG TRÌNH C	8
2.1.1. Khởi động C	9
2.1.2. Bắt đầu một dự án C	9
2.1.3. Lưu dự án	10
2.1.4. Thoát khỏi CodeBlock	10
2.2. CÁC VÍ DỤ C ĐƠN GIẢN	10
CÂU HỎI VÀ BÀI TẬP THỰC HÀNH	14
Chương 3. CÁC THÀNH PHẦN TRONG NGÔN NGỮ C	15
3.1. BỘ KÝ TỰ, TỪ KHÓA, TÊN VÀ LỜI GIẢI THÍCH	15
3.1.1. Bộ ký tự	15
3.1.2. Các từ khóa trong C	15
3.1.3. Tên (danh biểu)	15
3.1.4. Cặp dấu ghi chú thích	16
3.2. CÁC KIỂU DỮ LIỆU CƠ BẢN	16
3.3. HẰNG (Constant)	16
3.3.1. Khai báo hằng	17
3.3.2. Hằng số nguyên	17
3.3.3. Hằng số thực	18
3.3.4. Hằng ký tự	18
3.3.5. Hằng chuỗi ký tự	18
3.4. BIẾN	19
3.4.1. Cú pháp khai báo biến	19
3.4.2. Vị trí khai báo biến trong C	19
3.5. BIỂU THỨC VÀ PHÉP TOÁN	20
3.5.1. Biểu thức	20
3.5.2. Các phép toán	20
3.6. CÂU LỆNH TIỀN XỬ LÝ	22
CÂU HỎI VÀ BÀI TẬP THỰC HÀNH	22
Chương 4. NHẬP / XUẤT DỮ LIỆU	24
4.1. LỆNH GÁN	24
4.2. XUẤT DỮ LIỆU VỚI HÀM printf()	25
4.3. NHẬP DỮ LIỆU VỚI HÀM scanf()	28

CÂU HỎI VÀ BÀI TẬP THỰC HÀNH	28
Chương 5. CẤU TRÚC RỄ NHÁNH CÓ ĐIỀU KIỆN.....	30
5.1. LỆNH VÀ KHỐI LỆNH.....	30
5.1.1. Lệnh.....	30
5.1.2. Khối lệnh	30
5.2. LỆNH if.....	30
5.2.1. Dạng 1 (if thiếu).....	30
5.2.2. Dạng 2 (if đủ).....	34
5.2.3. Cấu trúc else if.....	37
5.2.4. Cấu trúc if lồng.....	42
5.3. LỆNH switch.....	47
5.3.1. Cấu trúc switch...case (switch thiếu)	47
5.3.2. Cấu trúc switch...case...default (switch đủ)	50
5.3.3. Cấu trúc switch lồng	51
CÂU HỎI VÀ BÀI TẬP THỰC HÀNH	53
Chương 6. CẤU TRÚC VÒNG LẶP.....	55
6.1. LỆNH for.....	55
6.2. LỆNH break	60
6.3. LỆNH continue.....	60
6.4. LỆNH while	60
6.5. LỆNH do...while	62
6.6. VÒNG LẶP LỒNG NHAU	64
CÂU HỎI VÀ BÀI TẬP THỰC HÀNH	65
Chương 7. HÀM.....	66
7.1. KHÁI NIỆM VỀ HÀM TRONG C	66
7.2. KHAI BÁO VÀ LỜI GỌI HÀM.....	69
7.2.1. Định nghĩa hàm	69
7.2.2. Gọi hàm	70
7.2.3. Khai báo nguyên mẫu cho hàm	71
7.2.4. Nguyên tắc hoạt động của hàm.....	72
7.3. HÀM main().....	75
7.4. PHẠM VI HOẠT ĐỘNG CỦA BIẾN.....	75
7.4.1. Biến cục bộ	75
7.4.2. Biến toàn cục	75
7.4.3. Biến tĩnh	76
7.5. HÀM TRÊN DÒNG - MACRO.....	77
7.6. SỰ ĐỆ QUY.....	78
7.6.1. Định nghĩa	78
7.6.2. Đặc điểm cần lưu ý khi viết hàm đệ quy	79
7.7. VÍ DỤ VỀ SỬ DỤNG HÀM	79
CÂU HỎI VÀ BÀI TẬP THỰC HÀNH	81
Chương 8. MẢNG VÀ CHUỖI.....	83
8.1. MẢNG TRONG C	83
8.1.1. Mảng một chiều.....	83
8.1.2. Mảng nhiều chiều	90

8.2. CHUỖI KÝ TỰ.....	96
8.2.1. Khai báo và khởi tạo chuỗi.....	96
8.2.2. Nhập / xuất chuỗi	98
8.2.3. Một số hàm xử lý chuỗi	98
8.2.4. Ví dụ về xử lý chuỗi.....	102
CÂU HỎI VÀ BÀI TẬP THỰC HÀNH	103
Chương 9. CON TRỎ.....	106
9.1. KHÁI NIỆM	106
9.2. KHAI BÁO BIẾN CON TRỎ	107
9.3. TRUYỀN ĐỊA CHỈ SANG HÀM	108
9.4. CON TRỎ VÀ MẢNG	109
9.4.1. Con trỏ và mảng 1 chiều.....	109
9.4.2. Con trỏ và mảng nhiều chiều	111
9.4.3. Cấp phát vùng nhớ cho biến con trỏ	113
9.4.4. Giải phóng vùng nhớ cho biến con trỏ	113
9.4.5. Cấp phát lại vùng nhớ cho biến con trỏ	114
9.4.6. Một số phép toán trên con trỏ	114
9.5. CON TRỎ VÀ CHUỖI.....	115
9.5.1. Khởi tạo chuỗi bằng con trỏ	115
Ví dụ 12:.....	115
9.5.2. Khởi tạo mảng con trỏ trỏ đến chuỗi.....	116
Ví dụ 13:.....	116
9.6. CON TRỎ TRỎ ĐẾN CON TRỎ	118
CÂU HỎI VÀ BÀI TẬP THỰC HÀNH	118